

UNIVERSITY OF MACEDONIA
SCHOOL OF INFORMATION SCIENCES
DEPARTMENT OF APPLIED INFORMATICS



Title

Verifying the Validity of Academic Qualifications with Blockchain Technology

BSc Thesis

Georgios Michoulis

Supervisor

Assistant Professor Sofia Petridou

Thessaloniki, 29 June 2020

Acknowledgements

I would like to warmly thank my supervisor, Sofia Petridou, for her valuable help with this thesis, for her advice, and for the patience and support she showed me throughout. I also thank Professor Konstantinos Vergidis, whose contribution to the preparation and final form of this thesis was very helpful and productive.

I would also like to thank my family, who have supported me in everything I have attempted over all these years.

Abstract

Interest in Internet security keeps growing and concerns more and more people. Together with advances in cryptography, which have turned researchers' attention to the problem of preserving security, this interest gave rise to blockchain technology. In 2008 Satoshi Nakamoto published the Bitcoin paper, proposing a new way of carrying out transactions between participants who cannot trust one another. The rapid growth of Internet applications, the huge volume of data and the urgent need to protect and safeguard users' transactions make the use of blockchain all the more necessary.

This thesis is built around the study of blockchain technology, with the aim of understanding it in both theory and practice. More specifically, it sets out to give a clear, illustrated account of blockchain technology and how it works. To this end, Chapter 2 describes the cryptographic background of blockchain so that we can study the mathematics behind the technology; we do this by applying the mathematics in the open-source software Sage. Chapter 3 follows. It describes the structure of blockchain and how the cryptosystems of the previous chapter are used, introduces concepts such as transactions, and presents the structure of a block, the way blocks are linked into a chain, and the concept of forking. The chapter also aims to draw a line between blockchain technology and Bitcoin: although the reference model it describes is the blockchain model that came from Bitcoin, the technology is not limited to the financial sector.

Chapter 4 describes the best-known consensus models and compares them as part of the research. Since Bitcoin is not the only blockchain, however, Chapter 5 focuses on Ethereum: its structure, a high-level comparison with Bitcoin, the technology behind digital wallets and smart contracts, and some examples of decentralized applications.

The forgery of academic qualifications is a problem that worries modern society. This problem, combined with the knowledge described above, led us to write Chapter 6, in which we design and implement Verde, a decentralized web application. Verde is a tool for countering the forgery of academic qualifications using the Ethereum blockchain. The chapter therefore describes the architecture of our application and how it works, using smart contracts that we implemented, the MetaMask digital wallet, and two interfaces that we built: a) one for registering the student's data and b) one for verifying the student's data. The chapter ends by comparing Verde with ten other similar applications. It also compares the Ethereum platform itself with RSK, the corresponding smart contract execution platform for Bitcoin, and analyzes which platform is better for developing Verde.

In summary, the contribution of this thesis lies mainly in the study of the technology and in the research and implementation of a decentralized application whose ultimate

aim is to prevent and counter the forgery of academic qualifications. The study therefore includes a literature review and comparative research, and it extends to a software implementation.

Keywords: Blockchain, Bitcoin, Ethereum, Smart Contracts, Hash Functions, Elliptic Curves, Merkle Tree, Decentralized Applications, Digital Wallets, Countering Degree Forgery.

Contents

1	Introduction	13
1.1	Objectives	17
1.2	Problem description	17
1.3	Methodology	17
1.4	Structure	18
2	Cryptographic background	21
2.1	Hash functions	21
2.1.1	Constructing hash functions	23
2.1.2	The Merkle-Damgård method	23
2.1.3	Birthday attack	24
2.1.4	The MD family	25
2.1.5	Authentication with hash functions	27
2.2	Elliptic curves	28
3	The blockchain model	35
3.1	Blockchain layers	35
3.2	Architecture	37
3.2.1	Peer-to-peer systems	39
3.3	Ownership	39
3.4	The double-spending problem	42
3.5	Transactions	42
3.6	Blocks	44
3.7	Linking the blocks	49
3.8	How the blockchain works: Bitcoin	49
3.9	Forking	51
3.9.1	Soft forks	51
3.9.2	Hard forks	51
4	Consensus models	53
4.1	The Proof of Work consensus model	54
4.2	The Proof of Stake (PoS) consensus model	55
4.3	More consensus models	56
4.4	Ledger conflicts and resolutions	58
4.5	Comparison table of consensus algorithms	59

5	Decentralized technologies on the Ethereum blockchain	61
5.1	Ethereum	61
5.2	Digital wallets	62
5.2.1	Nondeterministic wallets	63
5.2.2	Hierarchical Deterministic (HD) wallets (BIP-32, BIP-44)	64
5.2.3	Seeds and mnemonic codes (BIP-39)	64
5.3	Accounts	65
5.4	Transactions	66
5.4.1	Message transactions	66
5.4.2	Smart contract creation transactions	67
5.5	The block in Ethereum	67
5.6	EVM	69
5.7	Smart contracts	72
5.7.1	Data storage	73
5.8	Decentralized applications (dApps)	74
5.8.1	Tools for building a smart contract	75
5.8.2	Gas	77
5.9	Decentralized applications	79
6	The Verde decentralized application	83
6.1	The architecture of Verde	84
6.1.1	Purpose of the application	84
6.1.2	Architecture of the registration interface	85
6.1.3	Architecture of the verification interface	86
6.1.4	Architecture of the smart contract	87
6.2	Implementation of Verde	88
6.2.1	Implementation of the registration interface	88
6.2.2	Implementation of the verification interface	91
6.2.3	Implementation of the smart contract	93
6.3	Comparing Verde with similar applications	95
6.3.1	Comparison table of the proposals	102
6.4	Comparing development platforms for Verde	105
6.4.1	RSK	105
6.4.2	Comparison with Ethereum	106
7	Conclusions	111
7.1	Conclusion	111
7.2	Future research directions	112
	Appendix A: Terminology	115

References**117**

List of Tables

4.1	Comparison of consensus models [1, 2, 3, 4, 5].	60
6.1	Comparison of the proposals [6, 7, 8, 9, 10, 11, 12, 13, 14, 15].	104
6.2	Comparison of development platforms for Verde [16, 17].	109
A.1	Translation table.	115

List of Figures

2.1	The categories of hash functions by the kind of resistance they offer. .	23
2.2	The Merkle-Damgård construction.	24
2.3	Elliptic curves over the set of rational numbers. (a) Elliptic curve $y^2 = x^3 - 2x + 1$. (b) Elliptic curve $y^2 = x^3 - 2x + 4$	29
2.4	Elliptic curve over the finite field $\mathbb{F}_{47}$	31
3.1	The layers of the blockchain.	36
3.2	Distributed system.	38
3.3	Hybrid system.	39
3.4	Ownership.	40
3.5	The entities of a ledger.	41
3.6	Example of a transaction.	43
3.7	Structure of a block.	45
3.8	Structure of a Merkle tree.	46
3.9	The blockchain with the Merkle tree.	48
3.10	A chain of blocks.	49
3.11	A transaction will be added to a group of unused transactions.	50
4.1	The first block.	53
4.2	The outlink matrix that computes the importance of users in PoI [5]. .	57
4.3	Transaction conflict (A and B correspond to block(X) and block(Y) in the text).	59
4.4	Conflict resolution.	59
5.1	The structure of Ethereum.	61
5.2	The Ethereum “ecosystem”.	62
5.3	A complete diagram of how the Ethereum Virtual Machine works [18]. .	70
5.4	An abstract diagram of how the Ethereum Virtual Machine works. . .	71

6.1	The levels of the application.	84
6.2	The abstract structure of the application.	85
6.3	Registration of the student's details by the student's academic department.	86
6.4	Verification of a graduate's academic qualification.	87
6.5	Forms 1 and 2 of the interface and the retrieval of data from the DB. .	89
6.6	Forms 3 and 4 and the sending of data to the blockchain.	89
6.7	The ID of a successful transaction (transaction id) in hexadecimal form.	89
6.8	Sending personal details to Ethereum.	90
6.9	Sending the details of the courses the student passed.	90
6.10	Flow diagram of the registration interface.	91
6.11	The verification interface before "Search" is pressed.	92
6.12	Displaying the graduate's personal details.	92
6.13	Displaying the details of the graduate's courses.	92
6.14	The UomToken contract in UML.	94
6.15	The StudentGrades contract in UML.	95
6.16	Contract deployment.	107
6.17	Sending the student's details.	107
6.18	Sending the student's courses.	108

Chapter 1

Introduction

This thesis studies blockchain technology and the cryptosystems related to it, with the help of open-source software. In the end, we developed a proposal for countering the forgery of academic qualifications with this technology: the Verde application. We first define basic concepts of cryptography and the mathematical background they require. We then explain the structure of blockchain and the consensus models under which different blockchains operate. Finally, we present the decentralized application we built and compare it with similar studies. After a short account of the modern cryptosystems that blockchain uses, we turn to the technology itself. The main focus of the study is the abstract structure of the first blockchain, Bitcoin. We then compare Bitcoin's consensus model with other, more recent models. The thesis also explains the Ethereum blockchain and its smart contracts, compares Ethereum with the first blockchain at a high level, and implements our proposal on Ethereum.

Etymologically, the word blockchain is a compound of “block” and “chain”, and it conveys what the concept means: an unbreakable chain of blocks. Blocks that contain transactions are linked to one another by cryptographic information, which is very hard to tamper with because it relies on hash functions. Blockchain is one of the technologies that will flourish in the Fourth Industrial Revolution. Cryptography is the study of mathematical techniques for transmitting information securely over insecure communication channels. This is also what gives blockchain its decentralized character, so that no central authority controls its transactions. In essence, blockchain is a decentralized network of ledgers, and its nodes communicate with each other peer-to-peer. There is no gateway between the nodes and the blockchain, since communication across the network is torrent-like. The goal of blockchain is to let untrusted nodes communicate, transact and store data in a decentralized way, meaning that no third party controls the network.

The core ideas behind blockchain technology emerged in the late 1980s and early 1990s. In 1989, Leslie Lamport developed the Paxos protocol, and in 1990 he submitted the paper “The Part-Time Parliament” to *ACM Transactions on Computer Systems*

[19]; the paper was finally published in a 1998 issue. It describes a consensus model for reaching agreement on an outcome in a network of computers in which the computers, or the network itself, may be unreliable. In 1991, a chain of information was used as an electronic register for digitally signing documents, in a way that could easily show that none of the signed documents in the collection had been changed. These concepts were combined and applied to electronic payments in 2008. They were described in the paper “Bitcoin: A Peer-to-Peer Electronic Cash System”, which Satoshi Nakamoto published under a pseudonym, and later in 2008 they took shape in the founding of the Bitcoin blockchain cryptocurrency network [20].

Satoshi Nakamoto’s idea of the blockchain is also the solution to the Byzantine Generals Problem [21]. An example makes the Generals Problem clearer. Each general has his own army, and the generals are stationed at different locations around the city they are besieging. They must agree either to attack or to retreat. It does not matter whether they attack or retreat, as long as all the generals reach consensus. They therefore communicate with each other by letter to decide their next move, but they cannot know whether a message they send or receive has arrived intact or is genuine. Nakamoto’s solution uses hash functions and a random single-use value, the nonce. General A sends General B a message to attack. The message contains the text (plaintext) and a suitable nonce, chosen so that the hash of the two meets the criterion (hash target) that B has set. B then computes the same hash and, if it meets the criterion B set, accepts A’s message as authentic. The process General A follows is called Proof of Work [22], and it requires a lot of computing power. If there were many A generals, they would place their messages in a “block” and find a single nonce for the whole block they hash. Before hashing, however, Nakamoto proposed adding other data to the block, to make its verification stronger and the hash harder to compute. One of these items is the hash of the previous block. This creates a chain of blocks, the blockchain, which cannot be falsified.

An incomplete definition we could give for blockchain is the following:

Blockchain is a distributed system of peer nodes that uses an algorithm to handle the information held in ordered, linked blocks of data by means of cryptography and computer security, with the aim of ensuring and maintaining the integrity of that information [23].

Blockchain is a technology with many applications today, and we should mention the main networks currently in operation: Bitcoin, Litecoin, Ripple, Ethereum and Hyperledger.

Bitcoin

Bitcoin is a digital money system and the first system to use blockchain technology. A new block is created roughly every 10 minutes, and blocks are linked to each other with the SHA-256 hash function. Bitcoin uses the PoW consensus model, in which the mining nodes (miners) must find a nonce, a random single-use value, to add to the new block so that the block's hash value is lower than a predefined difficulty value. The difficulty is adjusted up or down to keep block creation on the 10-minute target. In Bitcoin, transaction fees are very small, since miners earn most of their funds by publishing new blocks. The fee is designed to charge a small amount for each transaction, although it may grow when pending transactions are delayed. Paying a higher transaction fee can give a transaction higher priority for being added to the blockchain. Originally, miners received 50 BTC for each block. In July 2016 the reward for mining a block was 12.5 BTC, and since May 2020 it has been 6.5 BTC because of the halving. Under the protocol, this reward is halved every 210,000 blocks (about four years) and will drop to zero once 21 million BTC have been created. Bitcoin mining will continue beyond that point, but the reward will come entirely from transaction fees. Finally, a Bitcoin transaction contains code written in a language called Script. Bitcoin transactions today use only a small part of the functions Script makes available. In practice, most Bitcoin transactions use very little Script code to move funds between the parties [1].

Litecoin

Litecoin is similar to Bitcoin but aims to offer faster confirmation times. Litecoin has implemented SegWit, which splits transactions into two parts and in effect allows a larger block size. The “witness” signature is separated from the Merkle tree. Another difference between Bitcoin and Litecoin is that Litecoin uses the scrypt cryptographic algorithm for hashing instead of SHA-256. The scrypt algorithm is harder to solve than SHA-256 because it uses more memory, which makes it harder to develop application-specific integrated circuits for it. Litecoin allows more coins to be mined (84 million Litecoins). Litecoin complements Bitcoin, with larger transaction volumes, and was not designed to replace it [1], since it is meant for everyday transactions.

Ripple

Ripple aims to make use of Bitcoin while connecting different payment systems. It has a fixed supply of 100 billion XRP, half of which is intended for circulation. Clients do not need to download the whole blockchain, since they can easily join in seconds. There is also no reward for mining, because each transaction costs a small amount of Ripple, similar to gas in Ethereum [1].

Ethereum

Ethereum is a platform focused on executing smart contracts. Smart contracts are programs that live on the blockchain and that Ethereum users can access. They can both receive and send funds while performing computations. A contract can act as a trusted third party in transactions, because its code is public. Ethereum also uses a Turing-complete programming language (e.g. Solidity). Nodes earn rewards from mining fees and from transaction fees. Ethereum also has a concept called “gas”, which is used as the reward for performing computations for a contract. There is a maximum gas limit per smart contract, because all mining nodes must execute the transactions in parallel. The state produced by executing the contract is stored on the blockchain by the user who publishes the next block. In 2015, Ethereum suffered the DAO hack, which led to a hard fork. Many users rejected the hard fork, and this produced a new platform, Ethereum Classic, which in effect keeps running the Ethereum platform as it was before the fork.

Hyperledger

Hyperledger is a group of projects that aim to build distributed ledgers. It is hosted and supported by the Linux Foundation. Many blockchain platforms use Hyperledger, and each of them solves specific problems. There are many types of Hyperledger, such as the following [1]:

- Fabric. This is a permissioned blockchain that can execute smart contracts (chaincode). It was created by Digital Asset and IBM.
- Sawtooth. This is a distributed system that uses PoET as its consensus model. Each participant in the system requests a “wait” time from a piece of hardware, which assigns waiting times at random. The participant that gets the shortest time creates the next block in the chain. Sawtooth was created by Intel.
- Iroha. It works as a “Know Your Customer” (KYC) identity service using blockchain technology, which allows institutions to share data and manage identities. It was created by Soramitsu, Hitachi, NTT Data and Colu.
- Burrow. This is a platform that runs smart contracts. It accepts Ethereum smart contract code. It was created by Monax and co-funded by Intel.
- Indy. This is an independent platform that provides components for secure transactions and digital identities. It supports three important privacy features: a) Decentralized Identifiers (DID), b) pointers to off-ledger sources, so that no personal data is written to the ledger, and c) zero-knowledge proofs. It was created by the Sovrin Foundation.

1.1 Objectives

The aim of this thesis is to study blockchain technology and to implement a decentralized application in a blockchain environment. The first part of this work involves understanding the cryptographic background, the structure of blockchain and the consensus models, and researching and comparing these technologies. Next, we show how blockchain can be applied in academic institutions, develop an application, and compare it in two ways: with respect to the blockchain it uses, and against similar earlier studies. Implementing Verde gives a practical picture of how decentralized applications are structured and developed, and of the satisfactory solution our idea offers, which can be put into practice right away. In short, this thesis studies blockchain technology in both theory and practice, and it presents a research-based, realistic proposal for preventing the forgery of academic qualifications.

1.2 Problem description

News reports appear quite often these days about people who occupy critical positions in the private or public sector while holding forged academic qualifications. Typical cases of degree forgery involve administrative staff, teachers, researchers and, more worryingly still, doctors and surgeons [24, 25, 26, 27]. Concern about data integrity, however, extends beyond education to every sector of our society, including computer science itself, as the many security and privacy issues raised by Internet of Things technology show. These cases led us to consider how digital technology could help counter the problem. Considering that (i) obtaining an academic qualification is a “public transaction” between an academic institution and a graduating student, (ii) there are communities of organizations interested in accessing the record of these transactions, and (iii) no transaction can be altered once it has been created, we concluded that blockchain is a suitable solution [20]. Other features, such as the distributed storage of information and the absence of a central authority, also make it attractive from an implementation point of view. Finally, the lack of literature in Greek led us to write this study in Greek.

1.3 Methodology

This thesis is both a literature review and a research study. Our analysis of blockchain technology and of cryptography draws on authoritative literature. The work began with a study of the literature, to understand the theoretical background of blockchain and the tools used in blockchain systems. The next challenge was to implement a range of technologies: specific cryptosystems and mathematical operations in computer algebra systems; the decentralized web application and its interfaces, using programming

languages; and the smart contracts. Once all of this had been implemented in an open-source environment, we recorded the results, which this thesis presents. The research part consists of investigating, studying, observing and finally comparing the consensus models, the platform on which Verde is developed, and similar proposals.

1.4 Structure

Chapter 2 presents the basic cryptographic models on which blockchain technology is built. The chapter aims to explain the theory of hash functions and elliptic curves, and how they are applied in practice with the help of the open-source software Sage. Each section comes with suitable examples and applications of the mathematical theory in Sage, to make clear what the relevant definitions, theorems and corollaries mean.

Chapter 3 covers the architecture of blockchain based on the reference model, that is, Bitcoin. It first introduces the layers of blockchain, peer-to-peer systems and the concept of ownership. It then describes transactions, the structure of a block and the chain of blocks in detail, and closes with the significance of forking. The purpose of the chapter is to explain blockchain technology, and above all how it works, through illustrative figures and an analysis of these concepts.

Chapter 4 then studies consensus models. They concern a different layer of blockchain, which is why they have a chapter of their own. The chapter aims to describe the best-known, widely used consensus models and then to compare them. It also gives an overview of existing models.

Chapter 5 follows, with a presentation of the Ethereum blockchain. Ethereum differs from the reference model because it has concepts such as the EVM, smart contracts and gas, and its block structure and network architecture are different as well. The chapter explains the significance of digital wallets and decentralized applications. It goes into more technical aspects of Ethereum and describes Ethereum applications that are already on the market, which leads into the next chapter.

Chapter 6 describes the decentralized web application Verde, compares it with similar proposals, and investigates which blockchain platform is the best choice for it. The aim is to present the decentralized application VerDe (Verified Degrees), which is based on blockchain technology and smart contracts and verifies the validity of an academic qualification. What is new about the application is that it uses the Ethereum blockchain for verification, with “pseudo-cryptocurrencies” as the means of verifying academic qualifications.

Finally, Chapter 7 brings together the conclusions of this thesis about blockchain technology and the Verde application, offers some thoughts on blockchain, and discusses future research and extensions of the application and of blockchain.

To complete the thesis, Appendix A presents a dictionary table in which we propose Greek terms for a number of concepts.

Chapter 2

Cryptographic background

In this chapter we study the cryptographic systems that blockchain uses, and we give mathematical examples using the open-source software Sage.

Cryptography aims to achieve the following goals of communication security, which are its cornerstone in both theory and practice:

1. Confidentiality, which keeps data secret and hides information from unauthorized users. The term is also closely tied to privacy.
2. Non-repudiation, which prevents the sender from denying that they sent the message, and the recipient from denying that they received it.
3. Privacy, which aims to keep information confidential and to control access to it.
4. Authenticity, which assures the recipient that the message they received really comes from the sender who sent it and not from a third party.
5. Integrity, which ensures that the information being transmitted is not altered.

We study the cryptographic models that follow with the Sage tool. Sage is free, open-source mathematics software that emphasizes collaborative development and free access to the source code. It supports research and teaching in cryptography, number theory, algebra, geometry and other related fields.

2.1 Hash functions

In cryptography, a hash function is a function H that takes a message of any size and returns a string of fixed length (128, 160, 256 or 512 bits). The output of the function is called the “fingerprint” or “digest”. The most important use of hash functions is message authentication. For example, if Alice wants to send a message to Bob in a symmetric cryptosystem, then:

1. Alice hashes her message m to produce $y = H(m)$ and encrypts it with the key k to obtain $e = E(y, k)$.
2. She places e at the end of the message and sends it to Bob.

3. Bob separates the ciphertext from the message he received (m'). He decrypts it to produce $y = D(e, k)$ and then hashes the message he received to produce $y' = H(m')$.
4. If $y = y'$, he knows that the message he received is genuine.

The most common application of cryptographic hash functions is the “encryption” of passwords [28]. When a user enters their password `pwd`, the system computes the hash value $h(\text{pwd})$ and stores this value instead of the password itself. When the user enters the password again, the system recomputes the hash value and compares it with the stored one. A hash function must be fast to compute, and it must also meet the following requirements [29]:

- It must be infeasible for anyone to find a message m such that $H(m) = y$ [30]. That is, given a value $y \in \{0, 1\}^n$, the function must be one-way (or preimage-resistant).
- It must have the following property: given a message m , it is computationally infeasible to find a second message m' with $m' \neq m$ and $h(m') = h(m)$. This property is called second-preimage resistance, or weak collision resistance.
- A pair of messages (m, m') with $m \neq m'$ and $h(m) = h(m')$ is called a collision of h . If it is computationally infeasible to find a collision (m, m') of h , then h is said to be collision-resistant, or strongly collision-resistant.

Collision-resistant hash functions are sometimes called collision-free, but this is misleading. The function h maps a set with infinitely many elements to a set with finitely many elements. So there are many collisions (in fact, infinitely many) [28]. Collision resistance simply means that collisions are hard to find.

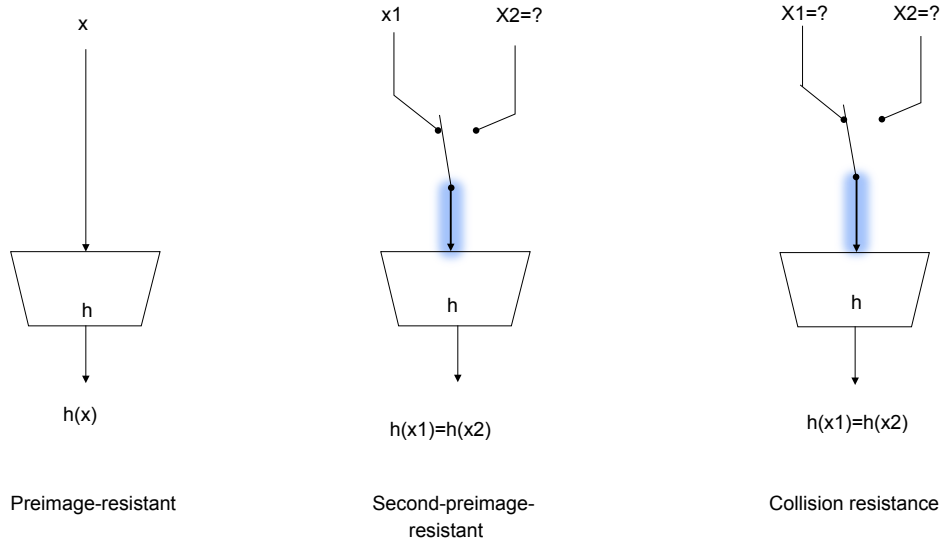


Figure 2.1: The categories of hash functions by the kind of resistance they offer.

2.1.1 Constructing hash functions

The example of Chaum, van Heijst and Pfitzmann helps us understand the structure of a hash function better [28]. Let p be a large prime such that $q = (p - 1)/2$ is prime. Also let α and β be two primitive roots of p (generators of $\mathbb{Z}_p^*$). The value $\log_\alpha \beta$ is not public, and we assume that computing it is computationally infeasible. The hash function $h : \{0, \dots, q - 1\} \times \{0, \dots, q - 1\} \rightarrow \mathbb{Z}_p^*$ is defined as follows:

$$h(x_1, x_2) = \alpha^{x_1} \beta^{x_2} \bmod p$$

This function maps integers modulo q^2 to integers modulo p . As a result, the message digest has roughly half as many bits as the message. So, if the discrete logarithm $\log_\alpha \beta$ in $\mathbb{Z}_p^*$ cannot be computed, the function is collision-resistant [28] [31].

2.1.2 The Merkle-Damgård method

One of the most efficient and practical ways to construct cryptographic hash functions is the method of Merkle and Damgård, also known as Merkle's meta-method [28]. The method, *which is also called the compression method*, reduces the problem of designing a collision-resistant hash function $h : \{0, 1\}^* \rightarrow \{0, 1\}^n$ to the problem of constructing a collision-resistant function $f : \{0, 1\}^{n+r} \rightarrow \{0, 1\}^n$ ($r \in \mathbb{N}$, $r > 0$) with the finite domain $\{0, 1\}^{n+r}$. In essence, a message m of length $n + r$ is reduced to a compressed message $f(m)$ of length n , where r is the compression rate. The figure shows how this method works. Specifically, m is a message of any length, and the hash function works iteratively to compute $h(m)$. The message is split into blocks of length r . The blocks are taken from m one after another, concatenated with the current value v and compressed by f

to give a new n -bit value. Every message m is padded with a string of the form $100\dots 0$, even if the length of the original message m is a multiple of r . This way, the bits added during padding can be told apart from the bits of the original message, and so they can be removed more easily [31].

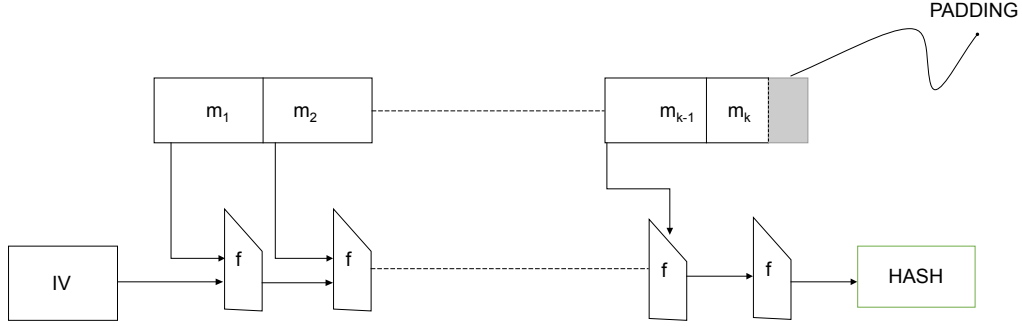


Figure 2.2: The Merkle-Damgård construction.

After padding, the message is decomposed as $m = m_1 \parallel m_2 \parallel \dots \parallel m_k$, $m_i \in \{0, 1\}^r$, $1 \leq i \leq k$, into blocks m_i of length r . The remaining bits of m_{k+1} are filled with zeros:

$$m = m_1 \parallel m_2 \parallel \dots \parallel m_k \parallel m_{k+1}$$

Starting with the initial value $v_0 \in \{0, 1\}^n$, we set recursively

$$v_i := f(v_{i-1} \parallel m_i), \quad 1 \leq i \leq k+1$$

Therefore $h(m) := v_{k+1}$.

2.1.3 Birthday attack

In hash function security, the level of security is measured by how hard it is to find a collision between two different messages that end up producing the same digest (hash value) [32] [28]. The birthday attack is a brute-force attack against collision resistance. For the probability to be above 50%, 23 people must be in the same room. This follows from the equation [28]:

$$\prod_{i=1}^{k-1} \frac{365-i}{365} = \left(1 - \frac{1}{365}\right) \left(1 - \frac{2}{365}\right) \dots \left(1 - \frac{k-1}{365}\right)$$

In practice, this means that hash functions have security of the order of $2^{n/2}$, that is, a 50% probability of finding a collision. For example, if we chose a 128-bit hash

function, it would have only 64-bit security [28]. That is very little, especially for modern computers, since 80 bits is considered the minimum for a brute-force attack to be ineffective. Therefore hash functions should all be at least 160 bits long [32] [28].

2.1.4 The MD family

This family consists of the cryptographic hash functions MD2, MD4 and MD5. The most popular of them is MD5, which was designed to fix flaws in its predecessor, MD4 [32] [28]. However, its internal compression function was found to have weaknesses, so it is not a secure function. MD5 also hashes to 128-bit values, which are no longer considered adequate for security today [32] [33]. **RIPEMD** (Research and Development in Advanced Communications Technologies in Europe = RACE + Integrity Primitives Evaluation = RIPE + Message Digest = RIPEMD) is the most modern form of hash functions. RIPEMD-160 produces a 160-bit digest. It comes from the MD4 family. Although MD4 is now considered outdated technology, RIPE is regarded as one of the most modern and strongest hash functions. **SHA-1**. The Secure Hashing Algorithm was developed by the National Institute of Standards and Technology (NIST) together with the US National Security Agency (NSA). It produces 160-bit values, but since February 2005 it has no longer been considered secure [31] [28]. **SHA-2** (Secure Hash Algorithm 2) is a family of six cryptographic hash functions with hash values of 224, 256, 384 or 512 bits, respectively: SHA-224, SHA-256, SHA-384, SHA-512, SHA-512/224 and SHA-512/256. Sage has libraries with which we can produce hash functions [31].

Sage:

```
sage: import hashlib
sage: hashlib.sha1('').hexdigest()
'da39a3ee5e6b4b0d3255bfef95601890afd80709'
sage: hashlib.sha1('a').hexdigest()
'86f7e437faa5a7fce15d1ddcb9eaaaea377667b8'
sage: hashlib.sha1('abc').hexdigest()
'a9993e364706816aba3e25717850c26c9cd0d89d'
sage: hashlib.sha1('abd').hexdigest()
'cb4cc28df0fdba0ecf9d9662e294b118092a5735'
sage: hashlib.sha1('Today is Christmas!').hexdigest()
'36836f719949547d3006aa46e6c5b64424d49eb3'
sage: hashlib.sha1('Today is Cristmas!').hexdigest()
'cfa8971d311749fb38e8b16db41e66d76a2fd098'
sage: hashlib.sha224('Today is Christmas!').hexdigest()
'0f452a8c03228ec8c64c7fb1edf687cceca11f313a86979243bd1c07'
sage: hashlib.sha256('Today is Christmas!').hexdigest()
'e01a896262d5bf8850f1a2f9cc5019a4b1c1f20fac6943974f25d433a518a504'
sage: hashlib.algorithms ('md5', 'sha1',
'sha224', 'sha256', 'sha384', 'sha512')
sage: hashlib.algorithms_available
```

```

'DSA',
'DSA-SHA',
'MD4',
'MD5',
'RIPEMD160',
'SHA',
'SHA1',
'SHA224',
'SHA256',
'SHA384',
'SHA512',
'dsaEncryption',
'dsaWithSHA',
'ecdsa-with-SHA1',
'md4',
'md5',
'ripemd160',
'sha',
'sha1',
'sha224',
'sha256',
'sha384',
'sha512',
'whirlpool'
sage: hashlib.sha1('a').hexdigest()
'86f7e437faa5a7fce15d1ddcb9eaeaea377667b8'
sage: hashlib.ripemd160('a').hexdigest()

```

```

-----
AttributeError Traceback (most recent call last)
<ipython-input-1-8b15933388b5> in <module>()
    1 import hashlib
----> 2 hashlib.ripemd160('a').hexdigest()
AttributeError: 'module' object has no attribute 'ripemd160'
-----

```

```

sage: r = hashlib.new('ripemd160')
sage:r.update("a")
sage:r.hexdigest()
'0bdc9d2d256b3ee9daae347be6f4dc835a467ffe'

```

The important point about this method is that if even a single bit of the function's input changes, the function produces a completely different result.

Modern hash algorithms

Very new hash algorithms have appeared in recent years, based on the latest standard, SHA-3. Specifically, there are five main new algorithms: BLAKE, Grøstl, JH, Keccak

and Skein. Of all of them, one¹ is the easiest, since it is based on the Merkle-Damgård design.

2.1.5 Authentication with hash functions

The most important purpose of hash functions is message authentication, that is, verifying where a message came from. When a hash function is used for message authentication, it is called a message authentication code (MAC) [31] [32]. The MAC is the most common symmetric technique for message authentication and integrity protection, in protocols such as SSL/TLS and IPsec. Typically, secret keys k are used to parameterize the hash functions [28] [31]. MACs are therefore families of hash functions [32].

The usual way to turn a cryptographic hash function into a MAC is the so-called HMAC [31] [28]. Finally, HMAC can be applied to a hash function h that is built from a compression function f using the Merkle-Damgård method [31]. The simplest way to create a MAC is $M = H(k \parallel H(k \parallel m))$. Given the key and the message, the MAC can be created in the following steps:

1. Hash the key together with the message.
2. Place the key in front of the digest.
3. Hash the previous result again.

The HMAC scheme, on the other hand, is built in the following steps:

1. Append zeros to the end of the key k to create a 64-bit binary string.
2. XOR the string from the previous step with the binary string 00110110; this is repeated 64 times.
3. Append the previous result to the end of the message m and pass it through a hash function.
4. XOR the digest from step 1 with the binary string 01011100; this is repeated 64 times.
5. Append the digest from step 4 to the end of the 64-byte string produced in the previous step.
6. Hash the result of the previous step.

If the 64 copies of 00110110 can be written in hexadecimal as **36** and the 64 copies of 01011100 as **5c**, HMAC can be written as [29] $\text{HMAC}(m, k) = H(k \oplus \mathbf{5c} \parallel H(k \oplus \mathbf{36} \parallel m))$. The following example shows why a MAC is useful. Suppose Alice wants to send a message to Bob. She uses a key known to both of them, creates the MAC from m and k , and sends both the message and the MAC to Bob. When Bob receives them, he computes the MAC of the encoded message m' using k and compares it with the

¹The name of the algorithm is missing in the original.

MAC that Alice sent him. If the two MACs are the same, Bob knows that the message he received is genuine [31].

In a blockchain network, cryptographic hash functions have many applications, and one could call them the foundation on which blockchain is implemented. Within the network, hash functions have the following roles:

- Creating addresses.
- Creating unique identifiers.
- Securing the block data. A publishing node hashes the block data, creating a digest that is stored in the block header [18].
- Securing the block header. A publishing node hashes the block header. If the blockchain network uses the Proof of Work consensus model, the publishing node has to hash the block header with a different nonce (a random number used only once) each time, until the requirements of the puzzle are met. The resulting digest of the current block is then included in the header of the next block [34] [1].

2.2 Elliptic curves

Elliptic curve cryptography (ECC) is the newest member of the three families of established public-key algorithms [32] [28]. ECC is based on the generalized discrete logarithm problem, so DL protocols such as the Diffie-Hellman key exchange can be implemented using elliptic curves [32] [30]. Any polynomial equation that relates y to x can be represented as a curve. By “curve” we mean the set of points (x, y) that are solutions of the equations. For example, the point $(x = r, y = 0)$ satisfies the equation of a circle and is therefore in the set [30]. An elliptic curve is a special kind of polynomial equation [30]. For elliptic curves, the domain is not the real numbers but a finite field [32] [30]. An elliptic curve is essentially a type of polynomial equation known as the Weierstrass equation, which produces a curve over a finite field [32] [35] [36]. In practice, elliptic curves are defined over the field $\mathbb{Z}_p$, where $p > 3$ and p is a prime. Definition of an elliptic curve [30]: the elliptic curve over $\mathbb{Z}_p$, for some prime integer $p > 3$, is the set of elements $(x, y) \in \mathbb{Z}_p \times \mathbb{Z}_p$ that satisfy the equation [30]: $y^2 \equiv x^3 + ax + b \pmod{p}$, where $a, b \in \mathbb{Z}_p$ and $4a^3 + 27b^2 \not\equiv 0 \pmod{p}$. Two basic corollaries of this definition are the following [30] [37]. First, the elliptic curve is symmetric about the x -axis. This follows directly from the fact that, for all values x_i that lie on the elliptic curve, both results, $y_i = \sqrt{(x_i)^3 + ax_i + b}$ and $y'_i = -\sqrt{(x_i)^3 + ax_i + b}$, are solutions. Second, there is one intersection with the x -axis. This follows from the fact that, solved for $y = 0$, it is a cubic equation, which has one real solution (the intersection with the x -axis) and two complex solutions [32] [38].

Example 1. Application of elliptic curves over the field of rational numbers $\mathbb{Q}$. Let $\mathbf{E}$ be the curve with equation:

$$y^2 = x^3 - 2x + 4$$

and let $\mathbf{F}$ be the elliptic curve with equation:

$$y^2 = x^3 - 2x + 1$$

□

In Sage, the `EllipticCurve` command creates elliptic curves. The `plot` function draws the graphs of the curves, as shown in Figure 2.3. Plot 2.3a shows the elliptic curve $y^2 = x^3 - 2x + 1$, defined over the rational numbers $\mathbb{Q}$. Plot 2.3b shows the elliptic curve $y^2 = x^3 - 2x + 4$ over the rational numbers $\mathbb{Q}$.

```
sage: G = EllipticCurve([-2,1])
sage: print G
Elliptic Curve defined by y^2 = x^3 - 2*x + 1 over Rational Field
sage: G.plot(aspect_ratio=1)

sage: E = EllipticCurve([-2,4])
sage: print E
Elliptic Curve defined by y^2 = x^3 - 2*x + 4 over Rational Field
sage: E.plot(aspect_ratio=1)
```

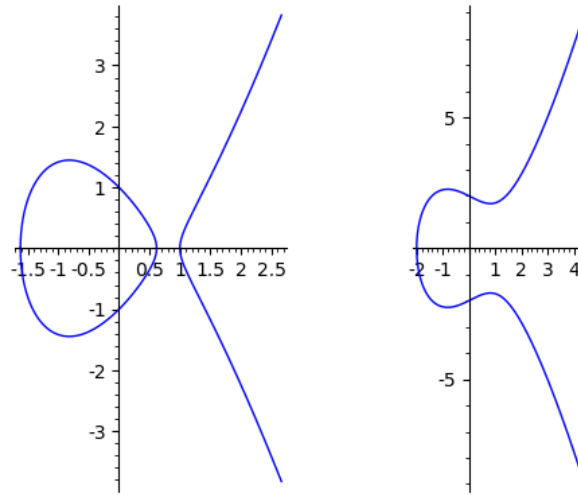


Figure 2.3: Elliptic curves over the set of rational numbers. (a) Elliptic curve $y^2 = x^3 - 2x + 1$. (b) Elliptic curve $y^2 = x^3 - 2x + 4$.

In blockchain, however, the preferred elliptic curves are those defined over a finite field, that is, over a finite number of elements, or in other words over a Galois field (GF) [32] [39]. The elements of finite fields such as $\text{GF}(2^m)$ are represented not as integers but as polynomials, with their coefficients in $\text{GF}(2)$. The polynomials have a maximum degree of $m - 1$, so each element has m coefficients in total [32] [39]. The properties of

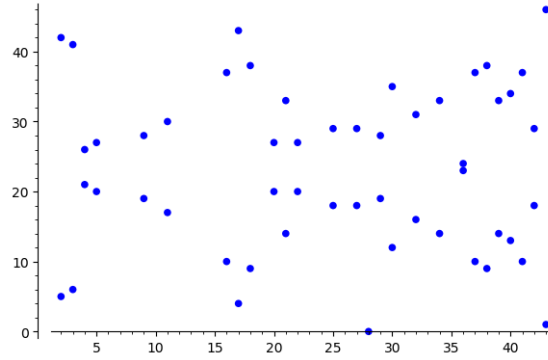
elliptic curves over finite fields do not differ much from those of curves defined over $\mathbb{R}$; only the way they are represented differs.

Example 2. Consider the elliptic curve defined over $\mathbb{Z}_{17}$ by the equation:

$$y^2 = x^3 - 8x + 33$$

□

```
sage: S = EllipticCurve(GF(47),[-8,33])
sage: print S
Elliptic Curve defined by y^2 = x^3 + 39*x + 7 over Finite
Field of size 47
sage: print S.is_on_curve(7,3)
True
sage: print S.cardinality()
54
sage: print S.order()
54
sage: print S.points()
sage: S.plot(pointsize=30)
[(0 : 1 : 0), (2 : 5 : 1), (2 : 42 : 1), (3 : 6 : 1),
(3 : 41 : 1), (4 : 21 : 1), (4 : 26 : 1), (5 : 20 : 1), (5 : 27 : 1),
(9 : 19 : 1), (9 : 28 : 1), (11 : 17 : 1), (11 : 30 : 1), (16 : 10 : 1),
(16 : 37 : 1), (17 : 4 : 1), (17 : 43 : 1), (18 : 9 : 1), (18 : 38 : 1),
(20 : 20 : 1), (20 : 27 : 1), (21 : 14 : 1), (21 : 33 : 1), (22 : 20 :
1),
(22 : 27 : 1), (25 : 18 : 1), (25 : 29 : 1), (27 : 18 : 1), (27 : 29 :
1),
(28 : 0 : 1), (29 : 19 : 1), (29 : 28 : 1), (30 : 12 : 1), (30 : 35 :
1),
(32 : 16 : 1), (32 : 31 : 1), (34 : 14 : 1), (34 : 33 : 1), (36 : 23 :
1),
(36 : 24 : 1), (37 : 10 : 1), (37 : 37 : 1), (38 : 9 : 1), (38 : 38 :
1),
(39 : 14 : 1), (39 : 33 : 1), (40 : 13 : 1), (40 : 34 : 1), (41 : 10 :
1),
(41 : 37 : 1), (42 : 18 : 1), (42 : 29 : 1), (43 : 1 : 1), (43 : 46 :
1)]
```

Figure 2.4: Elliptic curve over the finite field $\mathbb{F}_{47}$.

Point addition The addition of two points of an elliptic curve over $\mathbb{Z}_p$ is defined in the same way as over the real numbers. Let $P = (x_1, y_1)$ and $Q = (x_2, y_2)$ be two points of the elliptic curve $y^2 \equiv x^3 + ax + b \pmod{p}$. The point $P + Q = (x_3, y_3)$, which is also a point of the curve, has the coordinates $x_3 \equiv \lambda^2 - x_1 - x_2 \pmod{p}$, $y_3 \equiv \lambda(x_1 - x_3) - y_1 \pmod{p}$, where:

$$\lambda \equiv \begin{cases} \frac{(y_2 - y_1)}{(x_2 - x_1)} \pmod{p}, & \text{if } Q \neq P \\ \frac{3(x_1)^2 + a}{2y_1} \pmod{p}, & \text{if } Q = P \end{cases}$$

Another very important property of elliptic curves over $\mathbb{Z}_p$ is that the points of the elliptic curve, together with the point $\mathcal{O}$, form a cyclic subgroup. This means that any point on the elliptic curve other than $\mathcal{O}$ is a generator of it. That is, given a point R of the curve, adding R to itself repeatedly runs through all the points of the curve (for example, $2R = R + R, \dots, nR = \mathcal{O}$). The discrete logarithm problem in ECC rests on the idea that, under certain conditions, all the points on an elliptic curve form a cyclic group [32]. On an elliptic curve, the public key is a random multiple of the generator point, while the private key is a randomly chosen integer used to create that multiple [40]. In other words, a private key is a randomly chosen integer, while the public key is a point on the curve [40]. Finding the private key (an integer that ranges over the number of points on the curve) from the public key is a discrete logarithm problem [40]. To set up the discrete logarithm cryptosystem, it is important to know the order of the group. Although finding the exact number of points on a curve is a complex task, we can approximate it thanks to Hasse's theorem, which states [32] [36]:

Theorem 1 (Hasse's theorem). *Given an elliptic curve E modulo p , the number of points on the curve is denoted by $\#E$ and is bounded by: $p + 1 - 2\sqrt{p} \leq \#E \leq p + 1 + 2\sqrt{p}$.*

Hasse's theorem states that the number of points lies close to the prime p [32]. This has important practical consequences. For example, if an elliptic curve has 2^{160} elements, the prime is about 160 bits long [32].

Definition 2. Let an elliptic curve be defined over $\mathbb{Z}_p$. Let R be a point of the curve and Q a point that is a scalar multiple of R . The discrete logarithm problem on the elliptic curve is to determine the solution n , where $1 \leq n \leq \#E$, such that [30] [32]: $nR = Q$.

Here, Q is the public key (a point on the curve) and n is the private key. This means that the public key is a random multiple of the generator, while the private key is the integer used to create the multiple. $\#E$ is the order of the elliptic curve, that is, the number of points in the cyclic group of the elliptic curve [32]. A cyclic group is formed by a combination of points on the elliptic curve and the point at infinity [40]. Blockchain, and more specifically Bitcoin, as well as some Ethereum platforms, use one particular elliptic curve algorithm, namely Secp256k1 [40], which was established by the US National Institute of Standards and Technology (NIST). Secp256k1 was almost never used before Bitcoin became popular, but it is now gaining popularity thanks to its many nice properties. The most commonly used curves have a random structure, but Secp256k1 was constructed in a special, non-random way that allows especially efficient computation. As a result, it is often more than 30% faster than other curves if the implementation is sufficiently optimized. Also, the constants of Secp256k1 were chosen in a predictable way, which greatly reduces the chance that the curve's creator inserted any kind of "backdoor" into the curve [41].

Public- and private-key encryption with elliptic curves

Private key A private key is simply a number chosen at random. It is used to create signatures that prove ownership of the funds used in a Bitcoin or Ethereum transaction. The private key must stay secret, because revealing it to third parties amounts to handing them control of the personal data or funds. It must also be backed up and protected from accidental loss. If it is lost, it cannot be recovered, and anything it holds is lost forever. In Ethereum, a private key can be any nonzero number up to a very large number slightly less than 2^{256} , a 78-digit number, roughly 1.158×10^{77} [18]. Creating a private key means choosing a 256-bit key at random, which is done by hashing a number with many more bits (over 256 bits) using the SHA-256 or Keccak-256 algorithm. A very important detail is that the private key is generated locally, not online.

Public key The public key is computed from the private key using elliptic curve multiplication, which in practice cannot be reversed: $K_{\text{pub}} = k_{\text{pr}} \cdot G$, where k_{pr} is the private key, G is a constant point called the generator point, K_{pub} is the resulting public key, and the symbol $\cdot$ denotes the special elliptic curve multiplication [18]. The generator point is defined as part of the secp256k1 standard. It is the same for all implementations of secp256k1, and all keys derived from this curve use the same point G . Because the generator point is always the same for all Ethereum users, a private key k multiplied by G always gives the same public key K [34].

Digital signatures with ECDSA

The ECDSA standard is defined for elliptic curves over the prime field $\mathbb{Z}_p$ and the Galois field $\text{GF}(2^m)$. Key generation in ECDSA follows these steps [32]:

Definition 3.

1. Use an elliptic curve E with the following characteristics: a) it has a modulus p , b) it has coefficients a and b , c) it has a point A that is the generator point of the cyclic group with prime coordinates q .
2. Choose a random integer d , $0 < d < q$.
3. Compute $B = dA$. The keys are then $k_{\text{pub}} = (p, a, b, q, A, B)$ and $k_{\text{pr}} = (d)$.

This sets up a discrete logarithm problem in which the integer d is the private key, and the result of the scalar multiplication, the point B , is the public key [32]. The cyclic group also has an order q , which should be at least 160 bits in size. An ECDSA signature consists of a pair of integers (r, s) . Each value has the same bit length as q , which makes for fairly compact signatures. Using the public and private keys, the signature for a message x is computed as follows:

1. Choose the ephemeral key, which is a random integer k_E , $0 < k_E < q$.
2. Compute R : $R = k_E A$.
3. $r = x_R$.
4. Compute $s \equiv (h(x) + d \cdot r)(k_E)^{-1} \pmod{q}$.

In step 3, the x -coordinate of the point R is assigned to the variable r . The message x must be passed through a hash function h to compute s [32]. The output of the hash function must be at least as long as q . The process for verifying an ECDSA signature is then as follows [32]:

1. Compute the auxiliary value $w \equiv s^{-1} \pmod{q}$.
2. Compute the auxiliary value $u_1 \equiv w \cdot h(x) \pmod{q}$.
3. Compute the auxiliary value $u_2 \equiv w \cdot r \pmod{q}$.
4. Compute $P = u_1 \cdot A + u_2 \cdot B$.
5. Finally, verify with $\text{ver}_{k_{\text{pub}}}(x, (r, s))$, where:

$$x_P \begin{cases} \equiv r \pmod{q}, \Rightarrow \text{verified signature} \\ \not\equiv r \pmod{q}, \Rightarrow \text{signature not verified} \end{cases}$$

In step 5, x_P denotes the x -coordinate of the point P . Verification accepts a signature (r, s) only if x_P has the same value as the signature parameter, that is, r modulo q . Otherwise, the signature is considered invalid [32].

Proof. We show that a signature (r, s) satisfies the verification condition $r \equiv x_P \pmod{q}$. Consider the signature parameter s [32]: $s \equiv (h(x) + d \cdot r) \cdot k_E^{-1} \pmod{q}$.

This is equivalent to $k_E \equiv s^{-1}h(x) + ds^{-1}r \pmod{q}$. Let the right-hand side of the equation be expressed in terms of u_1 and u_2 : $k_E \equiv u_1 + d \cdot u_2 \pmod{q}$. Since the point A generates a cyclic group of order q , we can multiply both sides of the equation by A : $k_E \cdot A = (u_1 + d \cdot u_2) \cdot A$. Because the associative property holds in groups, $k_E \cdot A = u_1 \cdot A + d \cdot u_2 \cdot A$,

$$k_E \cdot A = u_1 \cdot A + u_2 \cdot B.$$

□

So, if the correct key is used, the key $k_E \cdot A$ equals the sum $u_1 \cdot A + u_2 \cdot B$, which holds [32]. In Ethereum and Bitcoin, the ECDSA model is necessary, since it is the only method used to identify the user (by their private and public key) and to create the user's address [32]. More specifically, the following holds in Ethereum [42]:

Definition 4. *For a given private key p_r , the 160-bit Ethereum address $A(p_r)$ is defined as the rightmost 160 bits of the Keccak hash function applied to the user's corresponding ECDSA public key [42]: $A(p_r) = B_{96\dots255}(\text{KEC}(\text{ECDSAPUBKEY}(p_r)))$.*

Chapter 3

The blockchain model

In this chapter we study the structure and architecture of the Bitcoin blockchain, which was also the first blockchain ever created. We look at the technology from its technical side and do not deal with the economics of Bitcoin or with cryptocurrencies.

3.1 Blockchain layers

Before we start describing the blockchain and how it works, we should first accept that a working system is a set of different layers. To begin with, we can distinguish the application layer from the functionality layer, and also two perspectives: what a system does and how it does it. What an application does at the application layer (the functional side) is the most visible part of a system, because it is what serves the users. How an application does something at the functionality layer (the nonfunctional side) is not considered as important. Three components of the nonfunctional side of a system are also very important. They are [23]:

1. **Data integrity:** The data a system uses are complete and correct, and leave no room for doubt.
 2. **Behavioral integrity:** The system behaves as it should, without logical errors.
 3. **Security:** The system is able to restrict access to its data to authorized users only.
- So most software failures are due to lost information, illogical system behavior or unlawful access to data, and they are caused by compromised integrity.

Just as the TCP/IP and OSI protocols are divided into stack layers, blockchain technology is divided into layers too. The layered approach of the TCP/IP stack is in fact a standard for achieving an open system [43]. Establishing abstraction layers makes the stack easier to understand [43]. Keeping the layers abstracted from one another also makes the system more resilient and easier to maintain [43]. For blockchain, there are as yet no agreed global standards that clearly separate its parts into distinct layers. A layered architecture is certainly needed, but for now this is still “fluid”. As Figure 3.1 shows, the blockchain is divided into five layers in total: the application, execution, semantic, propagation and consensus layers.

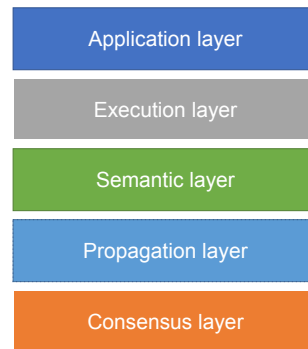


Figure 3.1: The layers of the blockchain.

Application layer

This is the layer where the functions are programmed and an application is built for the users. It usually involves a software development stack (for example, the user interface). Applications that treat the blockchain as a back-end system may need to be hosted on web servers and may require web application development or other similar technologies [43].

Execution layer

The execution layer covers the execution of the instructions created by the application layer, which are applied on all the nodes of a blockchain network [43]. A program or a script has to be run to make sure the transaction is executed correctly. All the nodes in a blockchain network must run the programs independently of one another. Deterministic execution of programs on the same set of inputs and conditions always produces the same output on every node, and this helps avoid inconsistencies [43].

Semantic layer

The semantic layer is a logical layer, because transactions and blocks are organized in it [43]. A transaction, whether valid or invalid, has a set of instructions that passes through the execution layer, but it is validated in this layer [43]. To use Bitcoin, one has to spend one or more previous transactions, and there are no user accounts [43]. This means that when someone makes a transaction, they use the amounts they received in their earlier transactions, and the sum of these must be at least equal to the amount they want to spend. All the nodes must validate this transaction by going through the previous transactions to establish whether it is legitimate [43]. Ethereum, on the other hand, has an account system and does not rely on the previous-transaction logic that Bitcoin uses [43].

A block usually contains a group of transactions and some smart contracts. Data structures such as the Merkle tree are defined in this layer [43]. This layer also defines

how blocks are linked to one another. Every block contains the hash of the previous block, all the way back to the first block ever created, the “genesis” block [43].

Propagation layer

The propagation layer is the peer-to-peer communication layer. It lets nodes discover one another, communicate, and stay synchronized with one another on the current state of the network [43]. We know that when a transaction is made, it is broadcast to the whole network. Likewise, when a node wants to propose a valid block, the block is broadcast at once to the whole network, so that the other nodes can build on it, treating it as the latest block [43]. The propagation of a transaction or block across the network is therefore defined in this layer, and this keeps the whole network stable. By design, most blockchains immediately forward a transaction or block to all the nodes they are directly connected to [43].

Consensus layer

The consensus layer is usually the most fundamental layer of most blockchain networks [43]. The primary purpose of this layer is to get all the nodes to agree on the state of the network’s ledger [43]. There can be different ways of reaching consensus among the nodes, depending on the use case [43]. The security of the blockchain is built into this layer. In Bitcoin and Ethereum, consensus is achieved through suitable incentive techniques called “mining” [43]. For a public blockchain to sustain itself, there must be incentive mechanisms that help keep the network alive and also enforce agreement [43]. Bitcoin and Ethereum use the Proof of Work (PoW) consensus model [43]. Once a block is proposed and propagated to all the nodes, the nodes check that it is a valid block whose transactions are all legitimate and that the puzzle (PoW) was solved correctly. They then add the block to their own copy of the blockchain and “build” on top of it [43].

3.2 Architecture

There are two kinds of system architecture: the centralized system, where all the nodes are connected to one central node, and the distributed system, where all the nodes are connected to one another indirectly. The advantages of a distributed system are [23]:

- **Higher computing power:** The node computers pool their computing power, so together they produce more of it.
- **Lower cost:** Maintaining and repairing a single part of a distributed system is much cheaper than repairing a supercomputer.
- **Greater reliability:** If one node fails or breaks down, the remaining nodes can take over its work. In a centralized system, if the central computer breaks down, the whole system collapses.

- **Ability to scale up naturally:** The computing power of such a system can be increased very easily by connecting more computers, whereas a single computer has to be replaced entirely to increase its computing power.

However, a distributed system also has several drawbacks compared with a standalone computer, such as [23]:

- **Lack of coordination.** Distributed systems have no central entity to coordinate their individual members, so the members must coordinate themselves, which is quite costly.
- **Time lost to communication.** Distributed systems must communicate internally, but this process wastes valuable time and computing power.
- **Network dependence.** Such a system necessarily needs a network over which its internal communication takes place. Every network, however, has its own drawbacks.
- **Greater complexity.** Overcoming the drawbacks of a distributed system listed above requires a great deal of complexity.
- **Security issues.** Since a distributed system requires a network, there is a risk that messages are transmitted insecurely.

In practice, however, modern systems use a combination of both types of system.

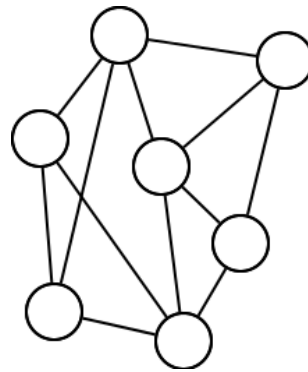


Figure 3.2: Distributed system.

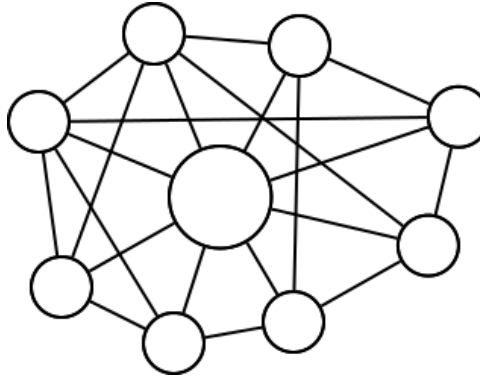


Figure 3.3: Hybrid system.

When we build an application, we can choose either architecture. Both, however, have advantages and disadvantages that affect the functional and nonfunctional sides of the application, and neither delivers full integrity. Blockchain addresses this problem, because it can give a distributed system full integrity and can achieve the nonfunctional side at the “application” level.

3.2.1 Peer-to-peer systems

A peer-to-peer system is a distributed system made up of nodes that share their resources directly. Blockchain is used in these systems to provide integrity. Integrity and trust in the system are the two most important characteristics of peer-to-peer systems [23]. *Integrity* is a nonfunctional aspect of the system. It must be secure and free of errors, and this is very easy to achieve if we know the number of nodes and how trustworthy they are. *Trustworthiness* concerns how far the human factor can be trusted or relied on, since this determines whether a new user should trust the system. To achieve both, however, two very basic risks must be overcome [23]:

- **Technical failures.** That is, hardware and software errors in a system and in its network.
- **Malicious users.** Users who, in pursuing their own ends, harm the system or other users.

3.3 Ownership

For the concept of ownership to exist, three conditions must hold at the same time [23]:

1. The owner must have an identity
2. Proof that the item has been bought
3. And a link between the two.

Looking at it historically, we can see that it is hard for someone to prove that something belongs to them unless they have witnesses who can confirm it. Today there are official

documents that prove that something belongs to someone (e.g. a house, through a notarial deed, a ledger and records).

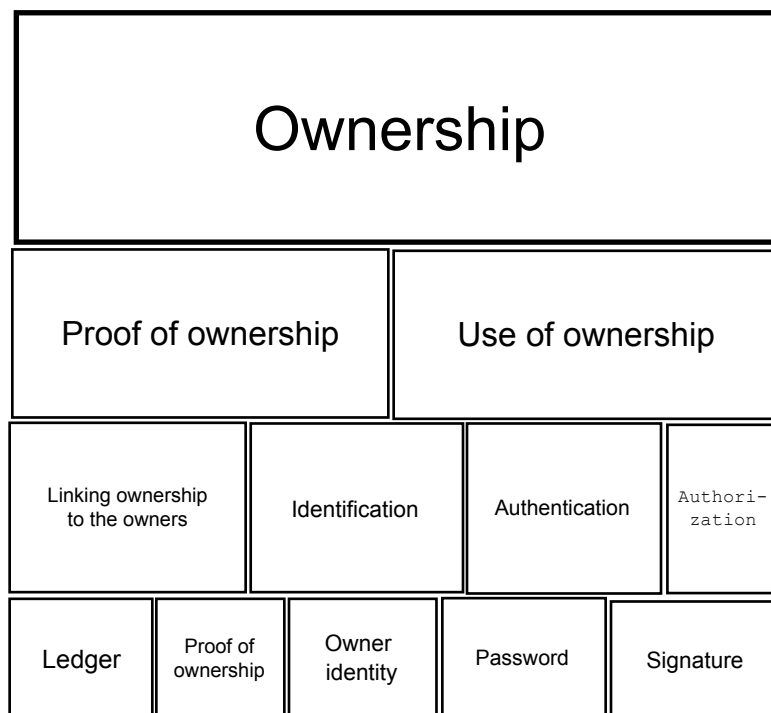


Figure 3.4: Ownership.

In Figure 3.4 above, the entities in the upper levels are more general than those in the lower ones. Each lower entity is directly related to the entities immediately above it. For example, proving ownership requires identification and authentication of the credentials, as well as authorization, to confirm the lawful holder of the property. The lowest entities represent the functionality layer. The three basic concepts of security are the following [23]:

- **Identification.** Identification is when someone claims to be a particular person. It does not prove who they really are, since it involves no proof.
- **Authentication.** The purpose of authentication is to prevent someone from claiming to be someone else. Authentication means that there is evidence of who someone really is. The proof is unique to each person (e.g. an ID card).
- **Authorization.** Authorization is when one user grants another user the right to perform a specific action that only the first user is entitled to. Authorization follows the successful identification and authentication of the user.

Figure 3.5 below shows how proving and transferring ownership relate to a ledger.

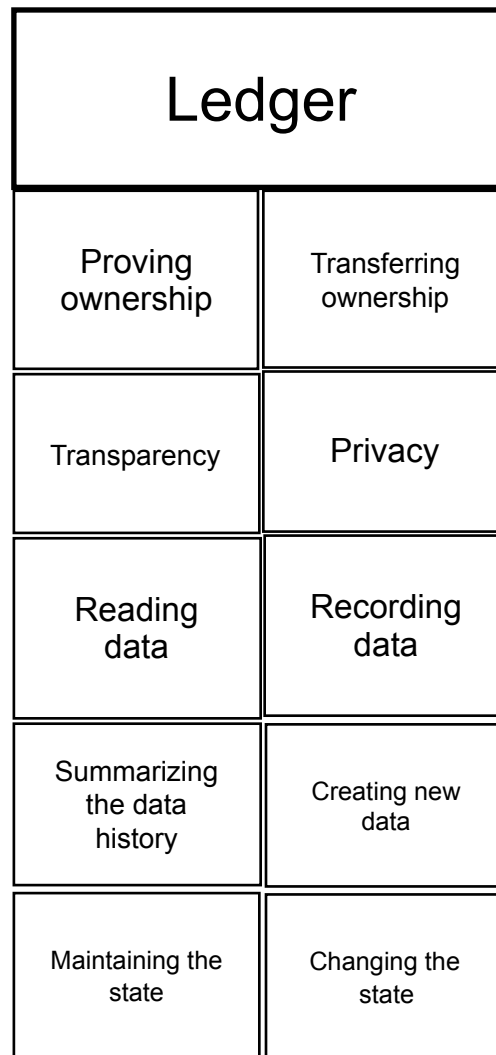


Figure 3.5: The entities of a ledger.

Figure 3.5 above shows a ledger that has to fulfill two opposing roles: one is to prove ownership by reading the data history, and the other is to record any transfer of ownership. So there is a conflict between the transparency of ownership, which is the only way ownership can be proved, and the privacy of the owner's personal data. The same conflict exists in the blockchain, since it is a distributed peer-to-peer system with a ledger structure that anyone can read [23]. Blockchain resolved this conflict between transparency and privacy, and it shares some characteristics with the ledger, for example [23]:

- A ledger is used to keep titles of ownership. It is the same as the structure of the blockchain and stores the data.
- Ledgers are stored as nodes in a peer-to-peer system
- Integrity in this system is the ability of someone to declare what they really own

- Cryptography is needed to create trustworthy means of identification, authentication and authorization, which keep the data secure.

Centralized ledgers have weaknesses: for example, they need copies in case of loss, and they must validate transactions, include all verified transactions and never modify their history. There are also distributed ledgers, which differ from the ledgers described above. Distributed ledgers (DLT) work in a fundamentally different way from a public blockchain. DLTs do not perform mining, since all the participants have already been vetted and are known to the network, and the network does not need mining to exist. There is also no concept of a digital currency [40]. In a public blockchain, access is open to everyone, and the blockchain needs some form of incentive and a network effect to grow [40]. A DLT, by contrast, has no such requirements. Permissioned DLTs can be built with Ethereum in private consortium deployments to run financial systems [40]. The main benefit of distributed ledgers is that they are much faster and easier to use [40].

3.4 The double-spending problem

In a peer-to-peer system, when an exchange takes place between A and B, there is a long delay before all the users learn that B received something from A. During that delay, a malicious A could go and exchange exactly the same item with a C who has not been told about the exchange between the first two. Only one party should own the item, however, and so the double-spending problem arises. This problem breaks down into three sub-problems, which blockchain has solved [23]:

- **Copying digital goods.** A digital good can be copied quite easily without anyone noticing. The solution lies in the nature of ownership, that is, in linking the specific item to its owner. This can be done with a ledger, which allows the item to take part in only one exchange at a time.
- **Peer-to-peer systems.** In a peer-to-peer system that uses ledgers, news that two users have made an exchange can take a long time to spread, and so we have the double-spending problem [23]. Blockchain, however, carries the solution to this problem within its structure and architecture.
- **Integrity violations in peer-to-peer systems.** In a peer-to-peer system the users themselves manage the data, so double spending is easy to carry out in such a system. Here the solution depends on the nature of the peer-to-peer system.

3.5 Transactions

A transaction is the record of a transfer of assets (e.g. cryptocurrency) between two entities. A transaction requires at least the following fields [23]:

- **Amount:** The total amount of digital goods to be transferred
- **Input:** A list of digital assets to be transferred. Each digital asset is unique and can have a different value from other goods. However, existing digital assets cannot create material goods, while digital assets can be divided into new digital assets or combined into fewer ones.
- **Output:** The accounts that will receive the digital assets. Each output includes the value to be transferred to the new holder, the identity of the new recipient, and a set of conditions the recipient must meet.
- **Transaction ID/Hash:** A unique key for each transaction. Some blockchains use an ID, while others use the hash as the key for a transaction.

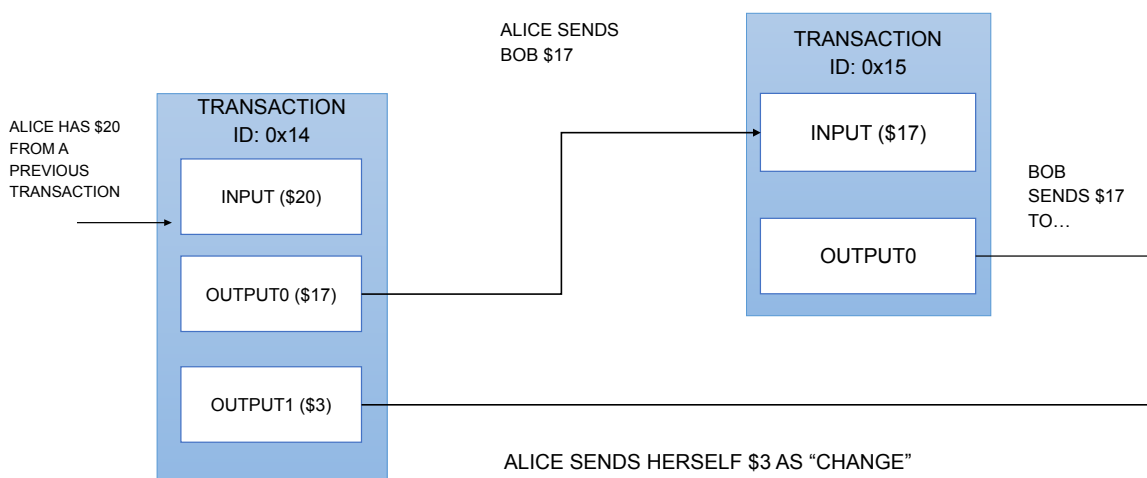


Figure 3.6: Example of a transaction.

Establishing whether a transaction is valid is very important, since someone may claim that a transaction took place when it never did [23].

Asymmetric cryptography

In asymmetric cryptography there are two keys, the public key and the private key, which are mathematically related. Unlike the private key, the public key is known to everyone. Asymmetric cryptography has the following applications in blockchain [23]:

- Private keys are used to sign transactions.
- Public keys are used to denote the address of origin, which allows pseudonymity.
- Public keys are used to verify signatures made with private keys.
- Asymmetric cryptography allows the user who exchanges the good to establish that they hold the private key that matches the public key used to verify the signature.

A user's address is an alphanumeric string derived from the user's public key by passing it, together with other data, through a hash function.

public key $\rightarrow$ hash function $\rightarrow$ address

Users can generate many pairs of private and public keys, which allow several levels of pseudonymity. Addresses act as a user's public identity on the blockchain and are often converted into a QR code for convenience [23]. The private key is generated securely by a random function and is stored in a digital *wallet*.

3.6 Blocks

Blockchain was created together with Bitcoin, so Bitcoin's architecture is the model for blockchain. Each blockchain has its own architecture and components. The block described in this section is the Bitcoin block. Users of a blockchain network submit candidate transactions to the network through software (web applications, smartphone apps, digital wallets, web services, etc.) [20]. The software sends these transactions to one or more nodes in the blockchain network. The distributed transactions then wait in a queue until a mining node adds them to the blockchain. Mining nodes are the subset of nodes that keep the blockchain running and publish new blocks [20].

Transactions are added to the blockchain when a mining node publishes a block. A block contains a set of validated transactions [20]. The "verification" is ensured by the cryptographic signature that the users (those referred to in the transaction's input values) place on the transaction [20]. This verifies that the transacting parties had access to the private key that could sign these particular transactions [1]. The other mining nodes check the validity of every transaction in a published block and will not accept the block if it contains any invalid transactions. Once created, each block is passed through the hash function, which produces a digest that represents the block. The structure of a block is fairly complex. In particular, its fields include the following [20] [1]:

- The block number, also known as the *height*.
- Its hash.
- The hash of the previous block. This is what creates the virtual chain of the blockchain.
- The hash of the Merkle tree root.
- Its size.
- The value of the *nonce* (a number used only once), which is a number the mining node uses to solve the hash puzzle so that it can publish the block.
- A list of the transactions included in the block.

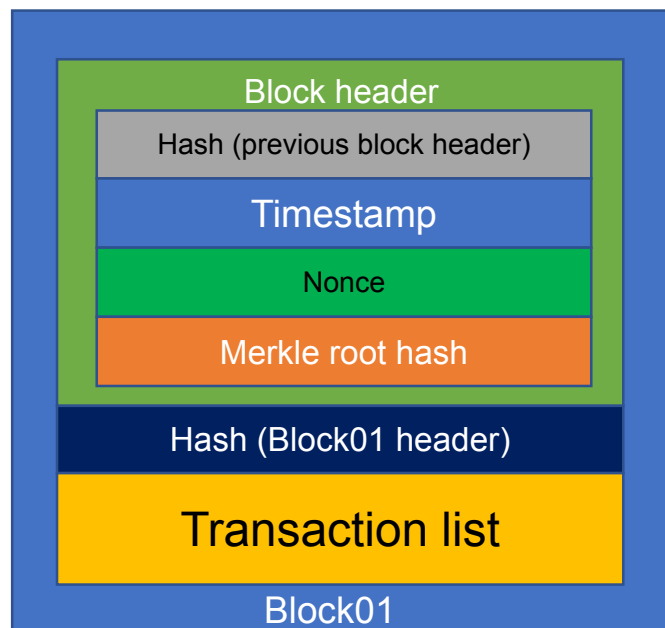


Figure 3.7: Structure of a block.

The block header is 80 bytes, while the average transaction is at least 250 bytes and the average block contains more than 500 transactions. A complete block, with all its transactions, is 1,000 times larger than the block header [34].

A block is usually identified by its header. There are two ways to identify a block. The first and main way is by the digital fingerprint produced by double-hashing the block header with the SHA-256 algorithm [34]. This fingerprint is 32 bytes long and is called the *block header hash*. For example, the block header hash of the first Bitcoin block is 000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f (Figure 4.1). The block header hash is unique, and any node can verify the validity of a block by hashing the block header again [34]. The block hash is not stored on the blockchain network; each node computes it at the moment it receives the block. The node stores the block hash in its own database to speed up block indexing.

The second way to identify a block is by its block height. The first block ever created has height 0 (Figure 4.1). Starting from the first block, each block is added on top, one position above the block immediately before it. While the block hash is unique to each block, two blocks often end up with the same height. This is a problem for the blockchain because it creates a conflict between the blocks. How this is handled is described in Section 4.4.

Merkle trees

Instead of storing the hash of every transaction in the block header, a data structure known as a Merkle tree is used, which holds the hash of all the transactions in the block

[1] [34] [40]. A Merkle tree (also called a binary hash tree) combines the hash values of the data and arranges them into parents and children until it reaches the one and only root of the tree (that is, a Merkle root hash). The root is an efficient mechanism for summarizing the transactions in a block and for verifying that a transaction is present in a block. The hash algorithm used in Bitcoin's Merkle trees is SHA-256, applied twice in a row. A Merkle tree is built from the bottom up. This structure ensures that the data sent to the distributed system are valid. A Merkle tree (Figure 3.8) has the following properties [20]:

- The bottom row represents the data to be hashed; in a blockchain, these are the transaction data.
- The second row from the bottom shows that the data have passed through the hash function and produced the hash.
- Next, the hashed data from the second row are joined one after another, and from each new value a new hash is created and stored in the leaves of the third row from the bottom.
- Finally, the root holds the hash of the combination of its two fields. The root is therefore the hash of all the earlier combinations and hashes performed within the tree.

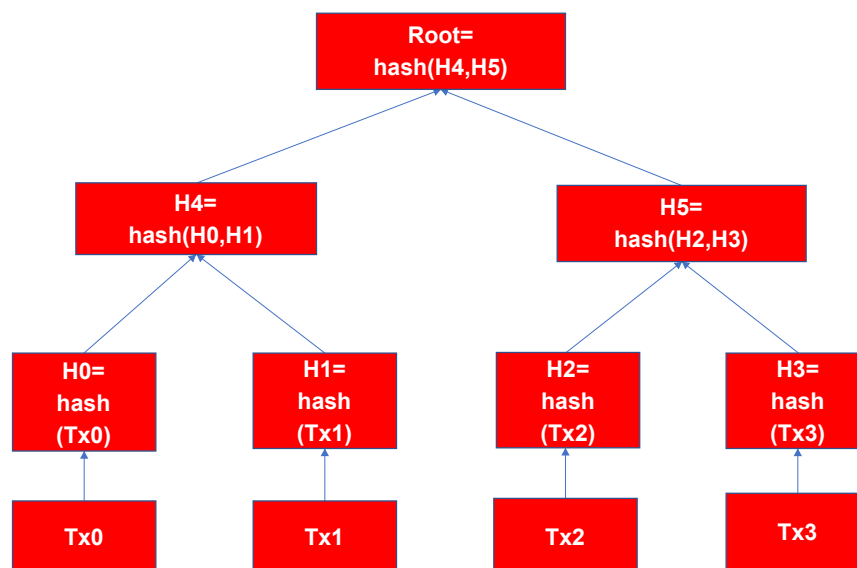


Figure 3.8: Structure of a Merkle tree.

Figure 3.8 shows that the bottom row of leaves of the tree contains the block's transactions (Tx0 to Tx3). The root of the Merkle tree is stored in the block header, as Figure 3.9 below shows.

The block header hash is stored both in the block itself and in the next block. This confirms that the transaction is immutable, since the Merkle root hash would no longer be the same if any change were made to the transactions.

When N data elements are hashed into a Merkle tree, checking whether an element is in the tree takes at most $2 \log_2 N$ computations [34]. The root of the tree, like every leaf of the tree, is 32 bytes in size. To prove that a particular transaction is included in a block, a node only needs to compute $\log_2 N$ 32-byte hashes, which form an *authentication path*, also called a *Merkle path*, linking that transaction to the root of the tree [34]. This lets Bitcoin nodes efficiently produce paths of 10 or 12 hashes, 320 to 384 bytes in size, which can prove that a single transaction is present among thousands of transactions [34].

Merkle trees are also used heavily by SPV (simplified payment verification) nodes. These nodes do not hold all the transactions or all the full blocks; they download only the block headers. To verify that a transaction is included in a block, they use a Merkle path [34].

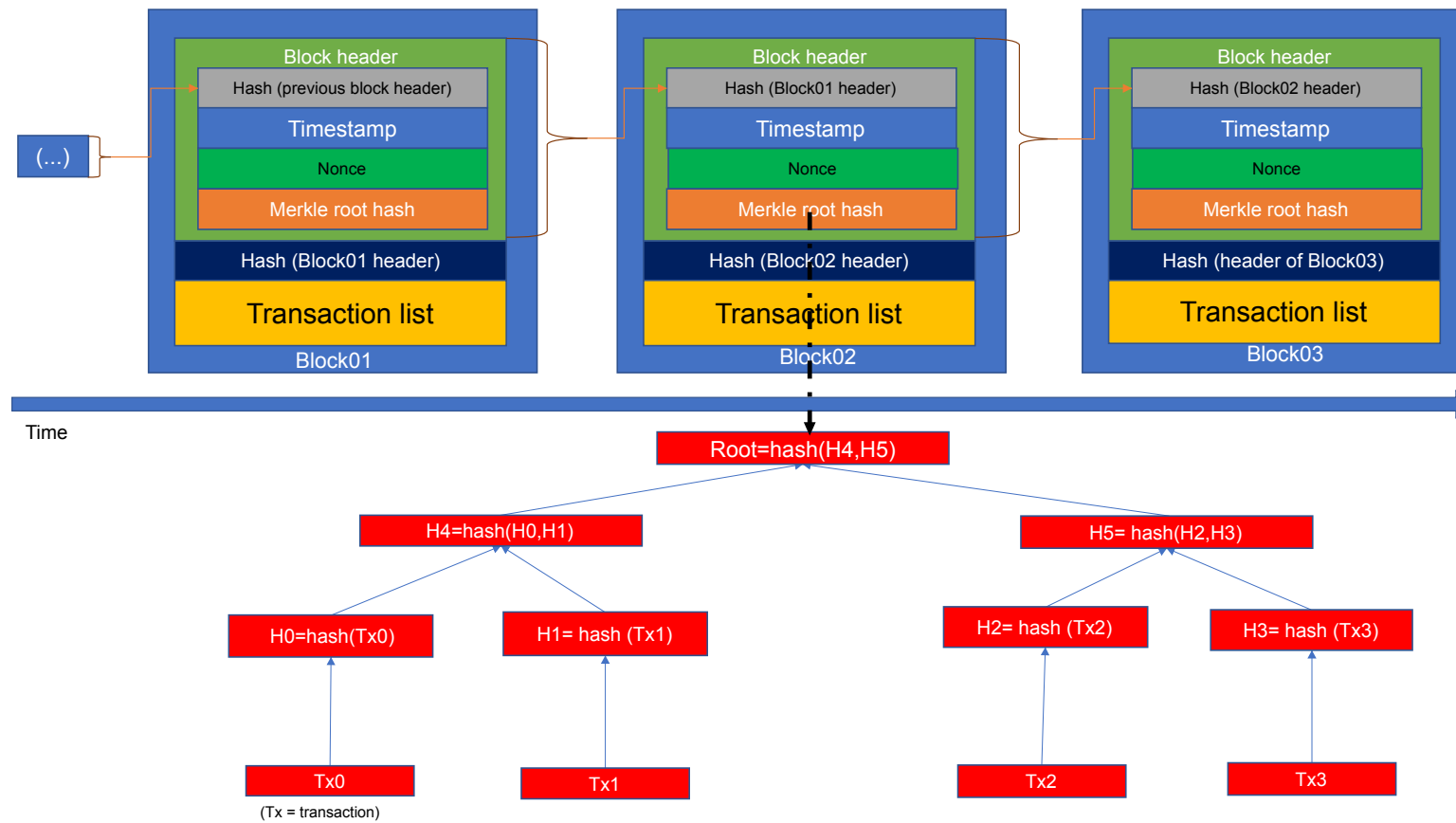


Figure 3.9: The blockchain with the Merkle tree.

3.7 Linking the blocks

Blocks are linked to one another in a chain, one on top of the other. Each block contains the header hash of the previous block, and in this way the blocks form the blockchain [20]. Suppose a block that has already been published is tampered with. The block then produces a new hash value (because a collision is very hard to produce) [1]. This in turn would change the hash values of all the following blocks, since each of them includes the hash of the block before it. This makes it possible to detect and reject altered blocks [1]. Figure 3.10 shows the abstract structure of a chain of blocks.

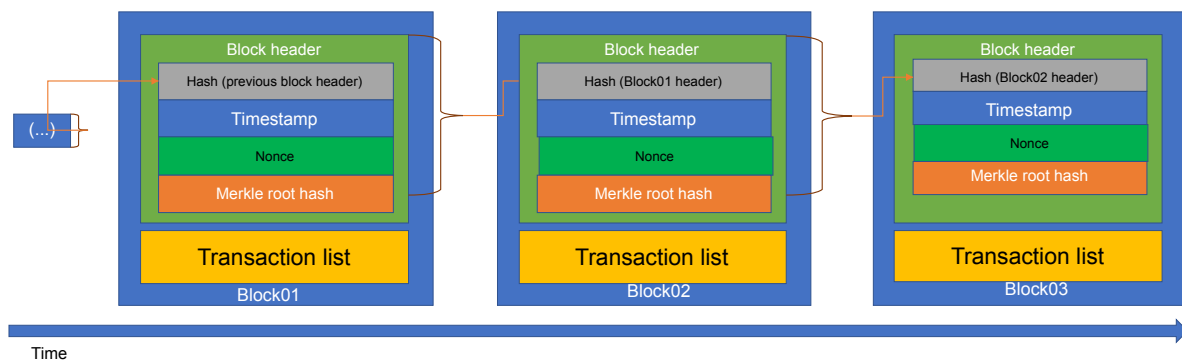


Figure 3.10: A chain of blocks.

In Bitcoin, computing the block hash takes a great deal of computing power and energy. So, in an attack on the blockchain, the attacker would have to recompute the whole chain up to and including the latest block to change the chain. This is practically impossible, first for economic reasons, and second because it is inherently impossible for the attacker to use that much computing power.

3.8 How the blockchain works: Bitcoin

The blockchain is maintained through the consensus of a set of computers that run blockchain software, called mining nodes [20]. There is no central authority that decides which node publishes the next block on the blockchain. Each node keeps a copy of the blockchain and can propose a new block to the other mining nodes [20]. Invalid blocks will be detected and rejected, because a valid block is hard to compute but computationally easy to verify [20]. Mining is a deliberately enormous task that requires large amounts of processing, memory, or both, depending on the blockchain implementation [1].

Any computer that runs the blockchain is considered a *node* of the blockchain. There are two types of node. *Full* nodes store the blockchain data, pass them on to other nodes and validate new blocks. *Light* nodes do not need to store full copies of the blockchain and often pass their data on to the full nodes [20]. A block is validated when

its format is correct, all the hash values in the new block have been computed correctly, the new block contains the hash of the previous block, and every transaction in the block is valid and signed by the appropriate parties [20]. Light nodes are usually found on smartphones and in IoT systems with low computing power and little memory. Any node can propose new transactions, which propagate between the nodes until they are finally added to a block [20].

Figure 3.11 shows a blockchain transaction stored on a mining node in a transaction “pool”, waiting to be assigned to a block.

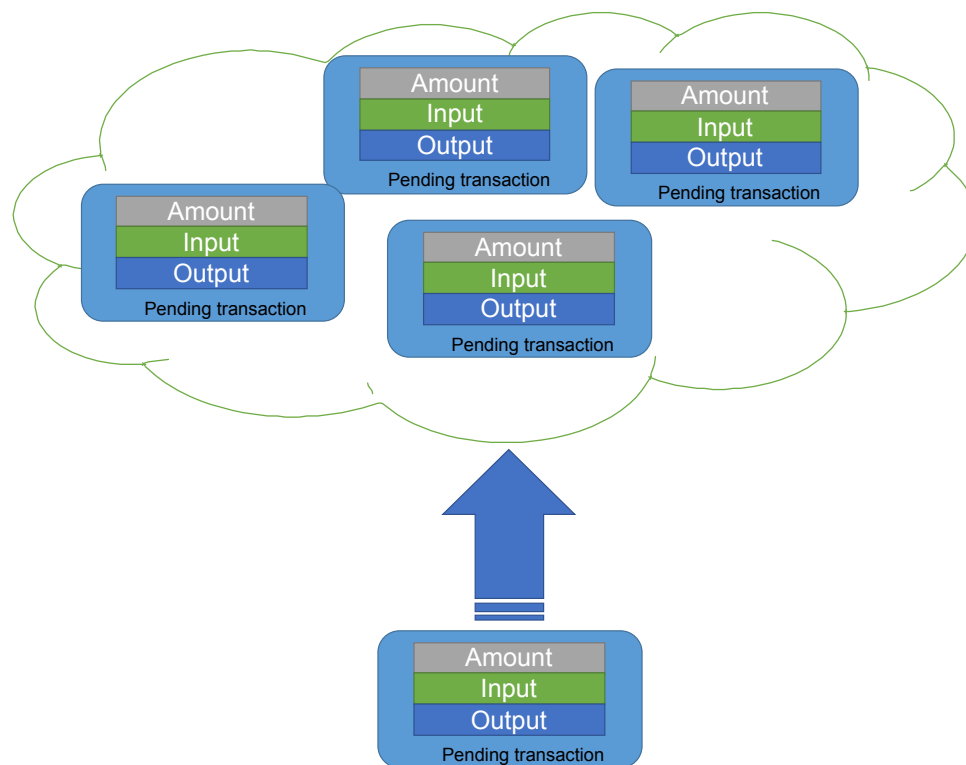


Figure 3.11: A transaction will be added to a group of unused transactions.

When mining nodes create a new candidate block, they place a set of unconfirmed transactions in it [1]. After validating the transactions, a node adds them to the *memory pool* or *transaction pool*, where the transactions wait until they can be included (mined) in a block. Nodes can therefore take a mix of older transactions that have been waiting and newer transactions that offer a higher payment [20]. The mining node checks the validity of every transaction, and the other nodes will reject the block if it includes invalid transactions [20]. At this point, the mining node fills in all the information that the block structure requires except the nonce [20].

Some blockchain systems require time and effort to create the next block [20]. In such systems, the mining node computes many nonces to solve a computationally hard

puzzle [1]. The mining node that wins gets the right to publish the next block. Mining nodes usually try many nonces before they solve a puzzle [20]. Once a puzzle is solved with a particular nonce, the node creates a hash of the block's data and stores it in the block itself. The block is then sent to other nodes for verification. If everything checks out, the nodes accept it as the latest block and move on to the next one [20] [1].

3.9 Forking

Changes to implementations and technology updates can be difficult at the best of times [20] [34]. Changes to the protocol and the structure of a blockchain network are called forks, that is, splits or branchings. They fall into two categories: soft forks and hard forks [20] [34]. In a soft fork, the changes are compatible with nodes that have not been updated, and the chain stays interoperable. In a hard fork, the changes “break” interoperability, because nodes that have not been updated will reject the blocks produced after the changes, and this effectively splits the chain [20] [34]. This can split the network and create multiple versions of it. Such networks can mitigate the problems of forking by requiring software updates [20] [40].

3.9.1 Soft forks

A soft fork is a change to a blockchain implementation that allows interoperability [20] [34]. Nodes that have not been updated can continue to transact with updated nodes [1] [40]. If the nodes do not adopt the update, it will not be followed [20] [34]. An example of a soft fork would be a blockchain deciding to reduce its block size (for example, from 1.0 MB to 0.5 MB) [20] [34]. Updated nodes would adjust the block size and continue to publish blocks as normal [1] [34]. Nodes that have not been updated would still see these blocks as valid, since the change does not break their rules (namely, that the block size must be within the maximum allowed limit). However, if a non-updated node created a block larger than 0.5 MB, the updated nodes would reject it as invalid [20].

3.9.2 Hard forks

A hard fork is a change to a blockchain implementation that does not allow interoperability [20] [34]. At a specific point in time, all the nodes must switch to the updated protocol. They must also upgrade to the new protocol so that they do not reject newly created blocks [20] [34]. Nodes that have not been updated cannot continue to transact on the updated blockchain, because they are programmed to reject any block that does not follow the new specifications [1] [40]. Note also that while most hard forks are intentional, software bugs can cause unintentional hard forks. If a serious flaw is found in the blockchain software, the only solution is to create a hard fork [20] [34]. In cryptocurrencies, if a hard fork splits the blockchain in two, users will hold independent coins on both forks [20] [34]. The old chain therefore cannot be used again, since the

two chains are no longer compatible. When Ethereum underwent a hard fork, most users moved to the new fork, while the old fork was renamed Ethereum Classic and continued to operate independently [1] [40].

Chapter 4

Consensus models

A key element in the smooth running of a blockchain is the consensus model it uses. The consensus model decides who issues a block and when, and all users consent to these rules. When a user connects to a blockchain network for the first time, they automatically accept the rules of that network and append new blocks after the genesis block. The genesis block is the first block created on the network, and it follows the rules of whichever consensus model the network uses.

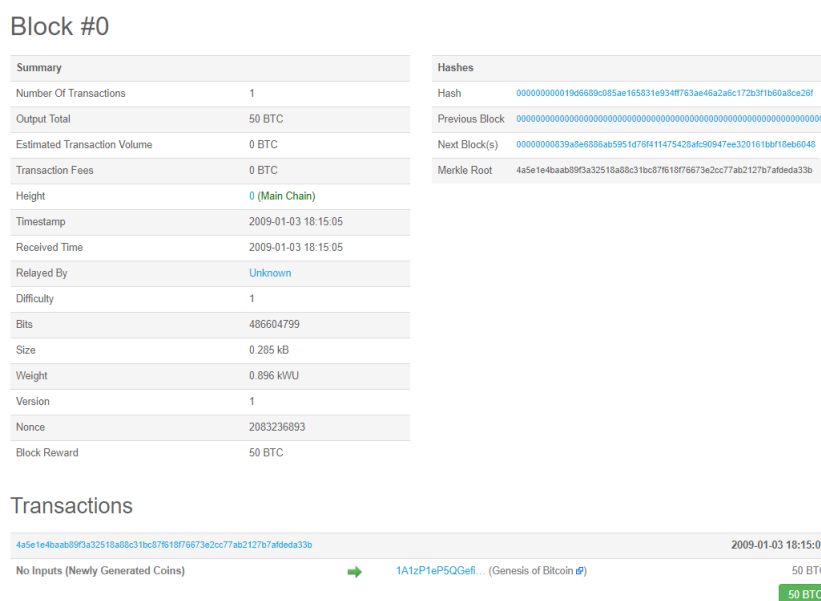


Figure 4.1: The first block.

Every consensus model has the following properties [40]:

- The initial state of the system, together with its rules, is set in the genesis block.
- Users agree to the rules of the consensus model from the moment they append a new block.
- Every block is linked to the digest of the previous block's header (except the genesis block, which has no predecessor).

- Users can verify each block independently.

4.1 The Proof of Work consensus model

In the Proof of Work (PoW) model, the users who publish the next block are the first to solve a computationally hard puzzle [18]. The solution to this puzzle is also the proof of their work [22]. Solving the puzzle is computationally quite hard, but checking a solution is easy [40]. This lets all the other full nodes verify any proposed next block, and any block that fails verification is rejected. The most common puzzle asks for a hash value lower than a given value [40]. The mining nodes then make small changes to the block itself (specifically to the nonce) as they try to find a block hash that meets the requirements [34]. For each attempt, a mining node has to compute the digest of the whole block header, which takes a lot of computing time [1]. The target hash value can be changed over time to adjust the difficulty and so control how often blocks are published. Note also that solving one puzzle does not change the probability of solving other puzzles, now or in the future, because the puzzles are independent of each other [22]. A puzzle has to meet certain conditions to be solvable and valid. First, all puzzles use the SHA-256 algorithm, and the computer must find the hash value that is requested each time and that meets the conditions [20, 22, 34]. For example, let:

$\text{SHA256}(\text{"blockchain"} + \text{nonce}) = \text{a digest that starts with the string } 000000.$

The string “blockchain” is concatenated with the nonce, and the result is hashed into a digest. This process is repeated 10,730,896 times before a hash value starting with six zeros is found (about 54 seconds on an older computer) [34]. As the number of consecutive zeros grows (0000000), so does the difficulty of finding such a value (on the same computer it took 1 hour, 18 minutes and 12 seconds) [1]. When a node has “matured” and is ready to publish, it sends its block, with the validated nonce, to a full node in the blockchain network [34]. The receiving full node checks that the new block meets the conditions of the puzzle, adds the block to its copy of the blockchain and shares it with the other peer nodes [18]. In this way the block spreads quickly across the network in a distributed manner, and the nonce is easy to verify: one hash computation shows that it solves the puzzle [34].

Suppose, for example, that Bob bought a product and paid in bitcoin. A mining node then picks up this transaction from the transaction pool. The mining node keeps accepting transactions until it has gathered a number of them. While it collects them, it also receives new blocks published by other nodes. So if it was trying to mine the 50th block of the blockchain and another node got there first by solving the puzzle faster, its own block will compete with the other nodes for the position of the 51st block. It therefore accepts the 50th block, verifies it and removes that block’s transactions from the transaction pool (even those it had added to its own block). It then adds new

transactions from the updated pool to its block, together with Bob's transaction, and within 10 minutes it computes many nonces in order to solve a computationally hard puzzle. Once it finds the nonce for this block, it immediately publishes the block to the other nodes. They accept it, verify it and remove Bob's transaction from the pool, and so the chain continues. Straight afterwards, the mining node creates an empty block (candidate block), that is, a block without a valid PoW, for position 52 of the blockchain and starts the same process again [34].

In many PoW blockchain networks, the publishing nodes tend to organize themselves into groups called "pools" or "collectives", which solve the puzzles together. They do this because the work can be split between two or more nodes in a collective, which then share the rewards as well as the work. In the following example the work is split across four nodes, and each checks an equal share of the nonce range [20]:

- Node 1: check nonce 0000000000 to 0536870911
- Node 2: check nonce 0536870912 to 1073741823
- Node 3: check nonce 1073741824 to 1610612735
- Node 4: check nonce 1610612736 to 2147483647

The seven-zero example took only 10 minutes and 14 seconds to solve, because the work was now split across four computers that computed hash values in parallel [40].

4.2 The Proof of Stake (PoS) consensus model

Proof of Stake rests on the idea that the more someone participates (that is, the larger their stake in the network), the more that user will want the system to succeed instead of undermining it. The stake is the amount of cryptocurrency the user has invested in a blockchain network [18]. A stake is both a bet and a share: once users have staked their coins, they cannot use them again for a period of time [20]. In this model, the probability that a user publishes a new block is proportional to their stake in the network [20]. This consensus model does not consume resources (time, electricity, processing power) on the scale of PoW, since all the cryptocurrency has been distributed to the users [20]. Some networks have therefore decided not to reward the creation of a new block [1]. A network can manage stakes in several ways [20], but the best known are the following four [20]:

1. Random selection of the publishing user. The network picks a user at random to publish the next block (chain-based PoS). The choice depends on each user's share of the stake: a user who holds a 30% stake has a 30% chance of being chosen to publish the next block.
2. Multi-phase voting. The next block is chosen by a vote held in several phases (Byzantine fault tolerance PoS). The network selects several participating users to

create the next block, and all of them vote on the proposed block. Several rounds of voting may take place before a decision is reached. This method lets all users take part in publishing the next block.

3. **Coin aging systems.** The user is chosen on the basis of time. Once a stake has been “staked” for a set period, it gives its owner an advantage in publishing the next block. The stake’s “staked” time is then reset to zero, and the user cannot use the stake again until a set period has passed. This method lets users with larger stakes publish more blocks without dominating the network, because the probability of success also has an upper limit.

One problem that may arise with this consensus model is known as “nothing at stake”: a user takes part in several competing networks [1]. This automatically raises the user’s chances of winning the rewards of each network, without the user devoting the necessary time to each branch of the network. Under this method, rich users with the largest stakes earn more; however, acquiring the majority of the digital assets is very expensive, or even impossible [20].

4.3 More consensus models

The sections above described the two basic consensus models, which are also the best known and the ones in practical use. There are, however, many more such models, which either have not been used yet or work only under certain conditions. Some examples follow:

- **Proof of Burn (PoB).** With Proof of Burn, or Proof of Destruction, mining nodes do not pay for electricity or computer hardware; they “burn” their coins to earn the privilege of proposing the next block. Proof of Burn was first proposed by Iain Stewart in 2012 [3] and is a mechanism for destroying cryptocurrency irrevocably and provably [44] [3]. The ability to produce convincing proofs turned this practice from a fringe act into a sensible and potentially useful effort [44]. It has since been established that metadata of the user’s choosing can be uniquely attributed to a burn, so that each “burn” can be tailored to a specific purpose [44] [3]. Burning coins means sending them to an address where they are lost for good. The more coins a node destroys, the higher its chances of being selected to mine the next block [44].
- **Proof of Importance (PoI).** Here the model considers both the amount of coins a mining node holds and an assessment of how important its activity is and how many transactions it makes. This variant of the mechanisms above takes a broader view of the “importance” of nodes when it selects the node that may propose the next block. More specifically, Proof of Importance is the blockchain consensus

algorithm used by the company NEM [5]. In this model, every account has an importance score that represents its overall significance to the NEM economy [5]. Accounts with a higher importance score are more likely to harvest a block, that is, to publish a new block [5]. Since all NEM transactions are public, the transaction graph of its economy can be computed exactly [5]. The main innovation of Proof of Importance is therefore that the transaction graph can be used to record and clarify the inflows of any account [5]. Because this network is highly transparent, the information on transfers between accounts can be used to assess how important the accounts are [5]. Finally, the outlink matrix (Figure 4.2), which defines the transaction graph, is important and is used to compute a user’s importance [5]. In essence, the outlink matrix (Figure 4.2) describes the weighted net flow of an account over the last 30 days, so only net transfers count towards that account’s importance [5].

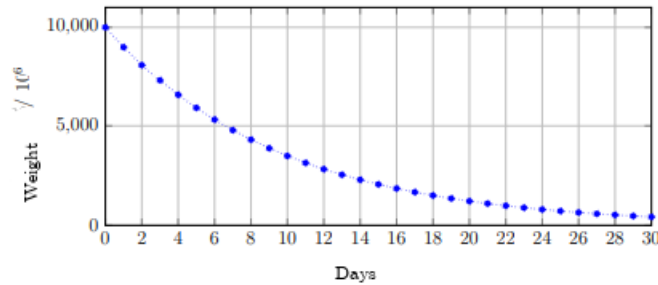


Figure 4.2: The outlink matrix that computes the importance of users in PoI [5].

- **Round Robin.** Round Robin is a consensus model often used for private blockchains, in which the nodes take turns to create blocks [1]. If a mining node is unavailable when its turn comes, a random available node can publish the block [20]. The model ensures that no single node creates the majority of blocks; it has no cryptographic puzzles and needs little energy [20].
- **Proof of Elapsed Time (PoET).** Intel introduced an alternative consensus protocol called “proof of elapsed time”. Each publishing node requests a wait time from a secure hardware time source in its computer system [1]. The secure hardware time source generates a random wait time and returns it to the publishing node’s software [20]. No puzzle has to be solved; a trusted execution environment (TEE) makes sure that blocks are produced at random. The model relies on a guaranteed wait time provided by the TEE [45]. Its drawback is that it needs specialized hardware and trust in a third party [45].

- **Proof of Authority.** Proof of Authority (also called Proof of Identity) relies on confirmed transactions and nodes, through validated accounts known as validators [1, 12]. In other words, publishing nodes must have a proven and verified identity within the network: the other nodes have given them the right (have authorized them) to publish new blocks. The idea is that a publishing node uses its confirmed, authentic identity to publish new blocks. The users of the network directly affect a publishing node's reputation according to how it behaves [12], so it is in the node's interest to keep its reputation high [1]. This algorithm applies only to blockchain networks with a high level of trust between the nodes.
- **Proof of Cooperation.** On 18 July 2017 the new “Proof of Cooperation” algorithm was introduced in FairCoin, and with it an energy-saving blockchain that uses cooperation instead of competition [4]. The consensus rules determine which Collaboratively Validated Nodes (CVNs) must create the next block [4]. Each CVN approves the selected CVN by digitally signing a piece of data that contains its unique identifier [4]. Once the selected CVN has received all the required signatures, it takes pending transactions and forms a new block, which is then stored in the immutable, distributed blockchain database [4]. Creating a block costs almost nothing, so CVNs can run on a Raspberry Pi 3, which draws very few watts and wastes no energy or computing resources. Even if the network of CVNs grows, its power consumption will stay very low [4].

4.4 Ledger conflicts and resolutions

One problem that comes with consensus models is congestion [20]. In many networks, several users may publish new blocks at the same time, so several different versions of the blockchain exist at that moment. Within distributed networks, some systems tend to lag behind or to hold alternative information. This depends on the network latency between nodes and on how close groups of nodes are to each other [1]. An important task of consensus models is to resolve data conflicts. For example, suppose node X creates block(X) with transactions 1, 2 and 3 and then publishes them to other nodes. Node Y then creates block(Y) with transactions 1, 2 and 4 and publishes them to other nodes. This creates a conflict between nodes X and Y, because the first includes transaction 3 and not 4, while for Y the reverse is true [20]. The conflict temporarily produces two different versions of the blockchain, as Figure 4.3 below shows. These versions are not necessarily “wrong”; each was created from the information its node had.

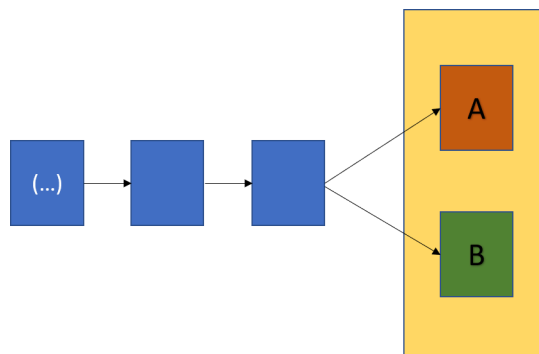


Figure 4.3: Transaction conflict (A and B correspond to block(X) and block(Y) in the text).

If the transactions involved a transfer of cryptocurrency, then both transactions may in this case have been sent and not sent at the same time [1]. Such conflicts are usually resolved very quickly. Most blockchain networks wait until the next block is published and then treat that chain as the “official” one, because it received the next valid block and is the “longest” chain. Every transaction that was in block(X) but is not in the block(Y) chain goes back to the “pool” of transactions waiting to be published [1]. Each node keeps its own local set of transactions waiting to be published, since the architecture has no central server [20]. The conflict was therefore resolved by choosing the chain that took the most energy to produce. This method is found mainly in the *Proof of Work* consensus model; the other consensus models have different mechanisms for resolving conflicts.

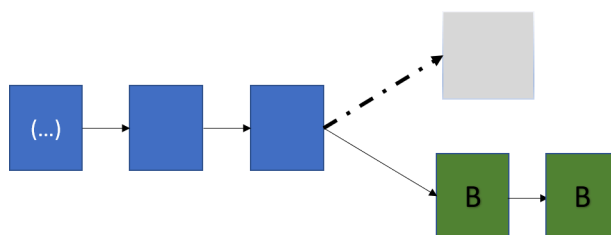


Figure 4.4: Conflict resolution.

Because a block may be replaced, a transaction is not accepted as confirmed until enough further blocks have been created on top of the block that contains the transaction that caused the conflict [20]. Some blockchains also lock specific older blocks into the blockchain software, creating checkpoints, to make sure this can never happen [20].

4.5 Comparison table of consensus algorithms

Algorithm	Attack resistance	Energy saving	Advantages, disadvantages / implementation	Goals	Implementations
PoW	25% of computing power	No	+Hard for malicious actors to carry out a DoS attack. -Computationally expensive. -51% attack on computing power. $\diamond$ Languages: Golang, C++, Solidity, Vyper, LLL	To allow transactions between untrusted participants by solving a very hard puzzle to publish new blocks	Bitcoin, Litecoin, ZCash, Ethereum
PoS	50% of the stake	Yes	+Open to anyone who wants to take part in the system. -Those who hold the stakes control the system. -51% attack by acquiring economic power. -Sybil Attack [46] $\diamond$ Languages: Michel-son	To allow transactions between untrusted participants, with a much lower computational barrier to publishing new blocks	Peercoin, Tezos, Tendermint
PoI	Unknown	Yes	+Fast and saves energy. +Takes into account both the user's stake and how active the user is in the system. -Just as vulnerable as PoS $\diamond$ Languages: Java	To allow transactions between untrusted participants, where users publish new blocks according to their importance.	XEM
Round Robin	Untrusted	Yes	+Easy to learn +Very little computing power. -Requires a high degree of trust between users. $\diamond$ Languages: C++	Block publishing among trusted nodes.	Multichain
PoET	Unknown	Yes	+Low energy consumption. -Uses special hardware to record time, which must not have been tampered with. -Time synchronization is practically impossible in distributed systems [47]. $\diamond$ Languages: Python	To create a more economical consensus model, reducing the financial cost that comes with the high security PoW offers.	Hyperledger Sawtooth Lake
PoB	25% of computing power	No	+More ecological than PoW -Bitcoins must be created in order to "burn" them, so it needs more energy than PoS $\diamond$ Languages: Golang, C++, Solidity, Serpent, LLL	To allow transactions between untrusted users by destroying coins, which requires computing power.	Slimcoin
PoC	Unknown	Yes	+Low transaction tax (transaction fee) and fast transactions. +Low power consumption. -Inherits the disadvantages of the Round Robin model, since it is based on it. $\diamond$ Languages: C++.	To create an environmentally friendly consensus algorithm (300watt/y). It works through cooperation between the nodes	FairCoin
PoA	50% of the online stake	Partially	+Fast confirmation +Allows intensive block production +Can be used on sidechains -Relies on the assumption that the current authorized node has not been compromised -Possible single (central) point of failure. $\diamond$ Languages: Solidity, Java, Python.	To create a centralized consensus algorithm that minimizes block creation and the confirmation rate	POA Chain; systems that use Parity

Table 4.1: Comparison of consensus models [1, 2, 3, 4, 5].

Chapter 5

Decentralized technologies on the Ethereum blockchain

Bitcoin was the first blockchain, but not the last. In this chapter we study the Ethereum blockchain, which was inspired by Bitcoin and proposed in 2015 by Gavin Wood and Vitalik Buterin [42]. It uses the Proof of Work consensus model [18] and is a permissionless blockchain [42]. The Ethereum blockchain uses endpoints and works through a JSON-RPC API, which is supported by the Ethereum Web3 API, that is, by the web3.js library. Figure 5.1 below shows the main elements of the Ethereum structure.

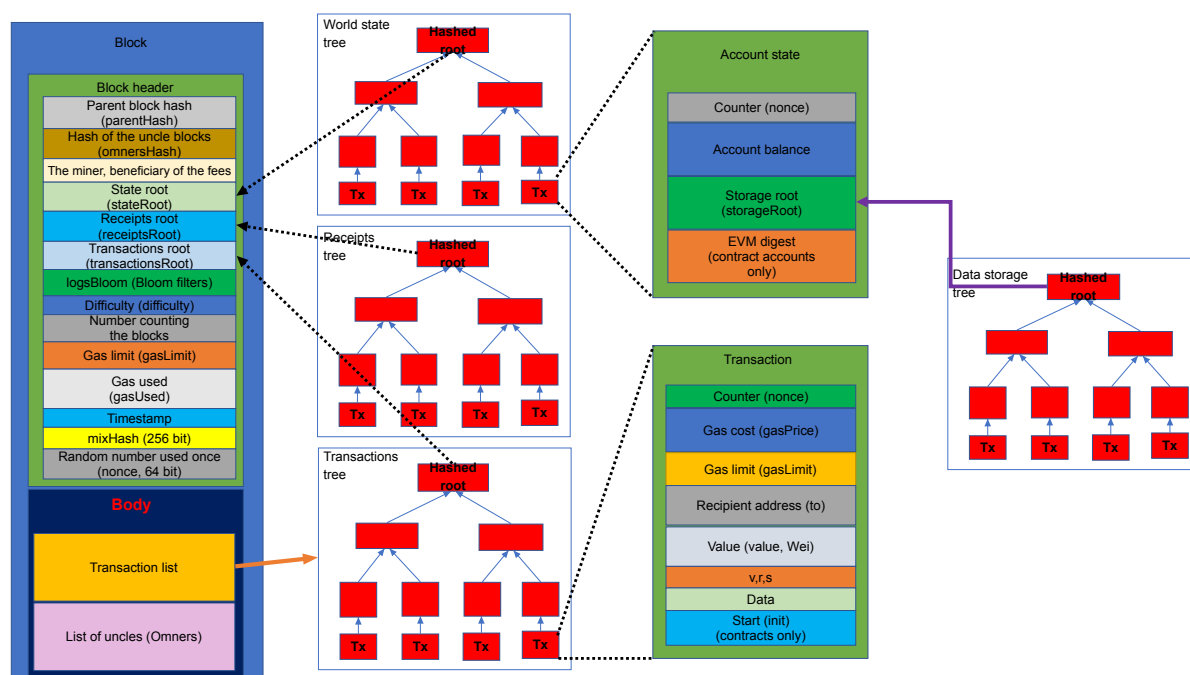


Figure 5.1: The structure of Ethereum.

5.1 Ethereum

The Ethereum network is a peer-to-peer network in which the nodes take part in the blockchain and contribute to the consensus mechanism [18]. Ethereum is split into two

kinds of network. The main network (mainnet) is where everything is published, and it corresponds to the main Ethereum chain. The test network (testnet) is a separate chain, distinct from the main one, which developers usually use to test how their smart contracts work (e.g. Ropsten) [40]. Ethereum is made up of several components. At its core is the Ethereum blockchain, which runs the peer-to-peer network. Next comes the Ethereum client (for example, Geth), which runs on the nodes and connects to the peer-to-peer network; the blockchain is downloaded from that network and stored locally [40] [18]. The local copy of the blockchain is synchronized with the network at regular intervals. Another component of Ethereum is the web3.js library, which allows interaction with the Geth client through the Remote Procedure Call (RPC) interface [40].

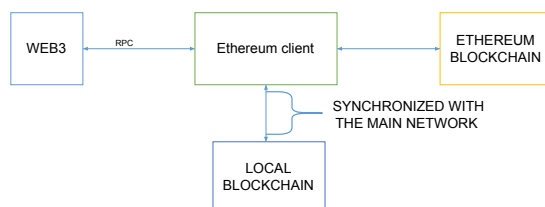


Figure 5.2: The Ethereum “ecosystem”.

5.2 Digital wallets

A digital wallet is an application that acts as the user’s main interface. The wallet controls access to the money in the user’s account, manages keys and addresses, tracks the account balance, and creates and signs the user’s transactions [34].

The wallet contains only the user’s public keys; the cryptocurrency itself is recorded on the blockchain [34]. Users control their cryptocurrency on the network by signing transactions with their keys. The coins are stored on the blockchain as transaction outputs [34]. There are two main types of wallet, depending on whether the keys they contain are related to each other [18]. The first is the nondeterministic wallet, also known as “Just a Bunch Of Keys”, in which each key is generated from its own random number [34]. The second is the deterministic wallet, in which all keys are derived from a single master key, known as the seed. All the keys in this type of wallet are linked to each other and can be generated again from the original seed [18]. To make deterministic wallets easier to use, seeds are encoded as English words, known as mnemonic code words. Anyone who holds these words and knows their order can reproduce the seed [34].

5.2.1 Nondeterministic wallets

In Bitcoin and Ethereum, the first wallets were collections of randomly generated private keys. These wallets have been replaced by deterministic wallets because they are hard to manage (that is, to back up). The drawback of random keys is that a copy of every key must be kept [18]. If the wallet is lost, the funds it held are lost with it. This is in direct conflict with the principle of avoiding address reuse, under which each bitcoin address is used for one transaction only [18]. Reusing the same address reduces privacy, because linking multiple transactions and addresses together weakens pseudo-anonymity. In Ethereum, “type 0” nondeterministic wallets are the hardest to deal with, because they create a new wallet file for every new address, “just in time” [18]. Even so, many Ethereum clients use a keystore file, which is in JSON format and contains an encrypted private key [34] [18]. For example:¹

```
{
  "address": "c10674bfce6a9933db3dd6f4ee895b8910ab5b78",
  "crypto": {
    "cipher": "aes-128ctr",
    "ciphertext": "c5ba1529dffdaf1e2b4439dd41e4e12eb6a1310fbf0761004ab9efba4d27851e",
    "cipherparams": {
      "iv": "669df88f19708a17c930ce45c04ec36e",
      "kdf": "scrypt",
      "kdfparams": { "dklen": 32, "n": 262144,
        "p": 1, "r": 2,
        "salt": "7807232bb72a2cf2a20af95195d6fb3d6e0fdff76e521aa30504ee22a6d506d5"},
      "mac": "ee3ee3af662b4b4508c975df89b3914e0aaac5556c65944f6ca47e5b7319de4e",
      "id": "7a551706-e627-40af-b9c9-0fce3940d4cf",
      "version": 3
    }
  }
}
```

Deterministic wallets

Deterministic, or seeded, wallets contain private keys that are all derived from a common seed through a one-way hash function [34]. The seed is a randomly generated number combined with other data [18]. In a deterministic wallet, the seed is enough to recover all the derived keys, so a single backup taken when the wallet is created is sufficient [40]. The seed is also enough to export or import a wallet, which makes it easy to move all of a user’s keys between different wallet implementations [34].

¹The standard cipher identifier is aes-128-ctr.

5.2.2 Hierarchical Deterministic (HD) wallets (BIP-32, BIP-44)

Deterministic wallets were developed to make it easy to derive many keys from a single “seed” [18]. The most advanced form of deterministic wallet is the hierarchical deterministic (HD) wallet, defined by the BIP-32 standard [34] [18]. HD wallets contain keys derived in a tree structure: a parent key can derive a sequence of child keys, each of which can derive a sequence of grandchild keys, and so on, to infinite depth [18]. HD wallets offer two important advantages over random (nondeterministic) keys [34]:

- The tree structure can express additional organizational meaning, for example when one branch of subkeys is used to receive incoming payments and a different branch is used to receive the change from outgoing payments [34].
- Users can create a sequence of public keys without having access to the corresponding private keys [34]. This allows HD wallets to be used on an insecure server, generating a different public key for every transaction [34].

5.2.3 Seeds and mnemonic codes (BIP-39)

There are many ways to encode a private key so that it can be backed up and recovered [34]. The method now preferred uses a sequence of English words that produce the private key when placed in the correct order [34] [18]. This is known as a mnemonic and is defined by BIP-39. Today most Bitcoin and Ethereum wallets use this standard and can import and export seeds for backing up and recovering accounts using only the mnemonic [34]. For example, the Ethereum private key `2304305F9FCC0F50E8A213AF903C3E416D959B9AF0951EBAE745217D8C4A15A` has the mnemonic `sock pact twice doctor kite track lonely axis able climb crumble hidden copper into parent`.

The advantage of a mnemonic is that the list of known words is fairly easy to work with, because there is redundancy in the way words are written.

Wallet best practices

As wallet technology has matured, some common industry standards have emerged that make wallets broadly interoperable, easy to use, secure and flexible [34]. These standards also let wallets derive keys for several cryptocurrencies, all from a single mnemonic [18]. The common standards are:

- Mnemonic code words, based on BIP-39.
- HD wallets, based on BIP-32.
- The multipurpose HD wallet structure, based on BIP-43.
- Multiple wallets with multiple accounts, based on BIP-44.

The standards have been adopted by a wide range of software and hardware wallets, which makes all of these wallets interoperable [18]. A user can export a mnemonic created in one of these wallets and import it into another wallet, recovering all of their transactions, keys and addresses [34]. Software wallets that support these standards include Breadwallet, Copay, MetaMask and Mycelium [18]. Hardware wallets that support them include Keepkey, Ledger and Trezor.

5.3 Accounts

As in every blockchain, Ethereum has keys and accounts. Before we describe them, however, it is very important to understand the concept of the world state (or simply the state).

World state

The world state is a mapping between addresses (160-bit identifiers) and accounts (account states), and it is the top level of Ethereum [42]. The world state is not stored on the blockchain; it keeps this mapping in a modified Merkle Patricia tree [42]. The trie needs a database that maps byte arrays to byte arrays [42]. This database is called “the state database” [42]. This has several benefits. The root node of the structure depends cryptographically on all the data inside it, so its hash can be used as a secure identifier for the entire state of the system [42]. Also, because it is an immutable data structure, any previous state can be recalled by changing the root hash [42]. Since all these roots are stored in the blockchain, an old state can be restored in this way [42].

Account types

At the lower level, on the other hand, are the Ethereum addresses. These represent accounts, each holding a balance in Ether, a nonce (a counter of the transactions successfully sent from the account), the account’s storage (permanent data storage, used only by smart contracts) and the account’s program code [18].

Accounts are therefore one of the basic features of the network. There are two kinds of account: 1) user accounts, called Externally Owned Accounts (EOAs), and 2) smart contract accounts, called Contract Accounts (CAs) [42, 18]. Accounts of the first kind are very like those in Bitcoin: they hold a balance, here in Ether. They can send transactions, are controlled by private keys, contain no code, store the value of the key, and are always associated with people [40, 18]. Smart contract accounts, the second kind, differ from the first in several ways. They also have a balance in Ether, but they are always associated with some code. They cannot carry out a transaction on their own, though they do execute code themselves. They always keep their state the same, are not associated with people, and contain a key value [40, 18]. Finally, when

the destination of a transaction is a smart contract address, the transaction causes the smart contract to run in the EVM, which changes its state.

5.4 Transactions

Like every other blockchain, Ethereum is a network driven by transactions (a transaction-driven state machine). A transaction is a single, cryptographically signed instruction constructed by an actor outside Ethereum. The ultimate external actor is assumed to be human, but software tools are needed to construct and propagate the transaction [42]. There are two types of transaction: those that result in message calls, and those that create new addresses/accounts with associated program code (known as “contract creation”) [42].

5.4.1 Message transactions

The basic idea is that, on the Ethereum blockchain, a “genesis” state is transformed step by step into a final state through the execution of successive transactions [18]. The final transformation is published to the world state of the blockchain [40]. Suppose, for example, that one user asks another for money, or that the sender simply decides to send money to the recipient. The request can be completed once the recipient has sent their Ethereum address to the sender. The address can be sent either as plaintext or embedded in a QR code [43]. In the second case, the sender simply scans the code with their phone, and the money moves from their Ethereum wallet to the scanned address (once the sender has approved it) [43]. Before the recipient is notified that money has arrived in their wallet, several steps must take place on the Ethereum network [43] [40].

First, once the transaction has been created in the sender’s wallet software, it is published to the Ethereum network and digitally signed by the sender, to prove that the sender owns the Ether [18]. Mining nodes then pick up the transaction, confirm it and include it in the block. Once it has been approved and added to the block, the “Proof of Work” consensus model comes into play [18]. As soon as a miner solves the Proof of Work puzzle, the block, with a new nonce, is broadcast immediately to the other nodes, which then verify it. If all the checks pass, the block is added to the blockchain and the miners are paid according to the work they put in. Finally, the recipient is notified that the money has been added to their wallet [18].

The process above is one form of Ethereum transaction: the message transaction. A message is a packet of data passed between two accounts. It contains data and a value (in Ether). It can be sent either through a smart contract or by an external actor, as a transaction digitally signed by the sender. As Figure 5.1 also shows, a message transaction has the following elements [42] [40]:

1. The sender.

2. The current depth of the message-creation stack, or nonce counter.
3. The recipient (which is a component of the message).
4. The available gas (gas limit).
5. The gas price.
6. v, r, s : values that correspond to the signature of the transaction and are used to identify its sender.
7. The value (in Ether).
8. The input data of the call.

Message calls also produce output data, which is not used when transactions are executed, only when the calls are made from the VM [42]. A message call, then, is the act of passing a message from one account to another. If the destination account has associated EVM code, the virtual machine starts as soon as it receives the message and carries out the required operations [42].

5.4.2 Smart contract creation transactions

The other form of transaction is smart contract creation. Its main similarity with message transactions is that contracts can send messages to other contracts, since messages exist only in the execution environment and are never stored [42]. The main difference, however, is that messages are produced by smart contracts, whereas transactions are produced by external accounts in the Ethereum environment [42]. Contract-creation transactions have the same elements as a message transaction, except that instead of data they have a different field, `init`: a byte array of unlimited size that specifies the EVM code for the account initialization procedure. The `init` code, that is, the initialization, runs only once, when the account is created [42].

When a contract is created, its address is generated from this transaction once the EVM has executed it. These addresses are the rightmost 160 bits of the 256-bit Keccak hash of the RLP (that is, of the account state) [42] [40].

5.5 The block in Ethereum

A block in Ethereum differs slightly from the block format described in Chapter 3. An Ethereum block is a collection of related pieces of information in the block header, together with information about the relevant transactions and the headers of the siblings of the parent block, that is, the uncle blocks (ommers) [42] [40]. Ethereum generally uses the Keccak-256 hash function, which belongs to the SHA-3 family of hash functions, so all the elements that make up a block are hashed with this hash function. As Figure 5.1 also shows, a block is made up of the following parts [42]:

- The hash of the parent block's header.

- The hash of the ommers' header.
- The beneficiary: the 160-bit address to which all the fees collected from successfully mining this block are transferred.
- **stateRoot**, **transactionsRoot** and **receiptsRoot**: the root hashes of the tries that hold, respectively, the state after all transactions have run, the list of transactions, and their receipts.
- **logsBloom**: the Bloom filter made up of index information contained in each entry of the receipt of every transaction in the transaction list.
- **Difficulty**: a scalar value corresponding to the difficulty level of this block.
- **Number**: a scalar value equal to the number of ancestor blocks.
- **gasLimit**: a scalar value equal to the current limit on gas expenditure per block.
- **gasUsed**: a scalar value equal to the total gas used by the transactions in this block.
- The timestamp at the start of the block.
- **extraData**: an arbitrary byte array containing data relevant to this block. It must be 32 bytes or less.
- **mixHash**: a 256-bit hash that proves enough computational power has been spent on this block.
- **nonce**: a 64-bit value which, combined with the **mixHash**, proves that enough computation has been carried out on this block. It is unrelated to the account nonce described in Section 5.3.

A block in Ethereum is considered valid when it meets certain conditions [40]:

- It must be related to the siblings of its parent node (that is, its uncles) and to the transactions. This means that every ommer must really be an uncle and that the uncles' PoW must be valid.
- The block's parent (the previous block) must exist and be valid (the block hash).
- The block's timestamp must be valid. This means that the timestamp of the current block must be greater than the timestamp of its parent block. The block must also have been created less than 15 minutes after its parent.
- If any of these checks fails, the block is rejected [40].

Finalizing a block in Ethereum involves validating the block and distributing the earnings from mining it. For a block to be considered successfully finalized, the following four steps must be carried out [40]:

1. Ommer validation. Validating the headers of the previous blocks checks whether each header is valid and whether the ommer's relationship to the current block stays within the maximum depth of six blocks. A block can have up to two ommers.
2. Transaction validation. This step checks whether the total gas used in the block equals the final gas consumption after the last transaction.
3. Reward payment. The beneficiary's account is updated with the balance of the reward. In Ethereum, the miners of previous blocks also receive a reward, which is $1/32$ of the block reward. Uncles included in blocks also receive $7/8$ of the total block reward. The current reward is 3 Ether.
4. Verification of the state and the nonce.

5.6 EVM

The EVM is the Ethereum system that handles and executes smart contracts. Simple value transfers between two EOAs do not need the EVM, but every other transaction has to be computed by the EVM, after which its state must be updated [18]. At a high level, the EVM can be seen as a global decentralized computer that contains millions of executable objects, each with its own permanent data store [18]. One of the main reasons Ethereum is secure is that its virtual machine is completely isolated [42]. Changes to Ethereum's state are made by the Ethereum Virtual Machine (EVM), a stack-based execution machine (specifically, Last In, First Out (LIFO)) that executes bytecode instructions to move the system from one state to another [18]. The stack is limited to 1024 elements [40]. The virtual machine's word size is 256 bits, so that it can accommodate hash functions such as Keccak-256 as well as elliptic curve computations. The EVM architecture (Figure 5.3) has several addressable components: a) the immutable *program code ROM*, which holds the bytecode of the smart contract to be executed, b) volatile memory, and c) persistent storage, which is part of the Ethereum state [18].

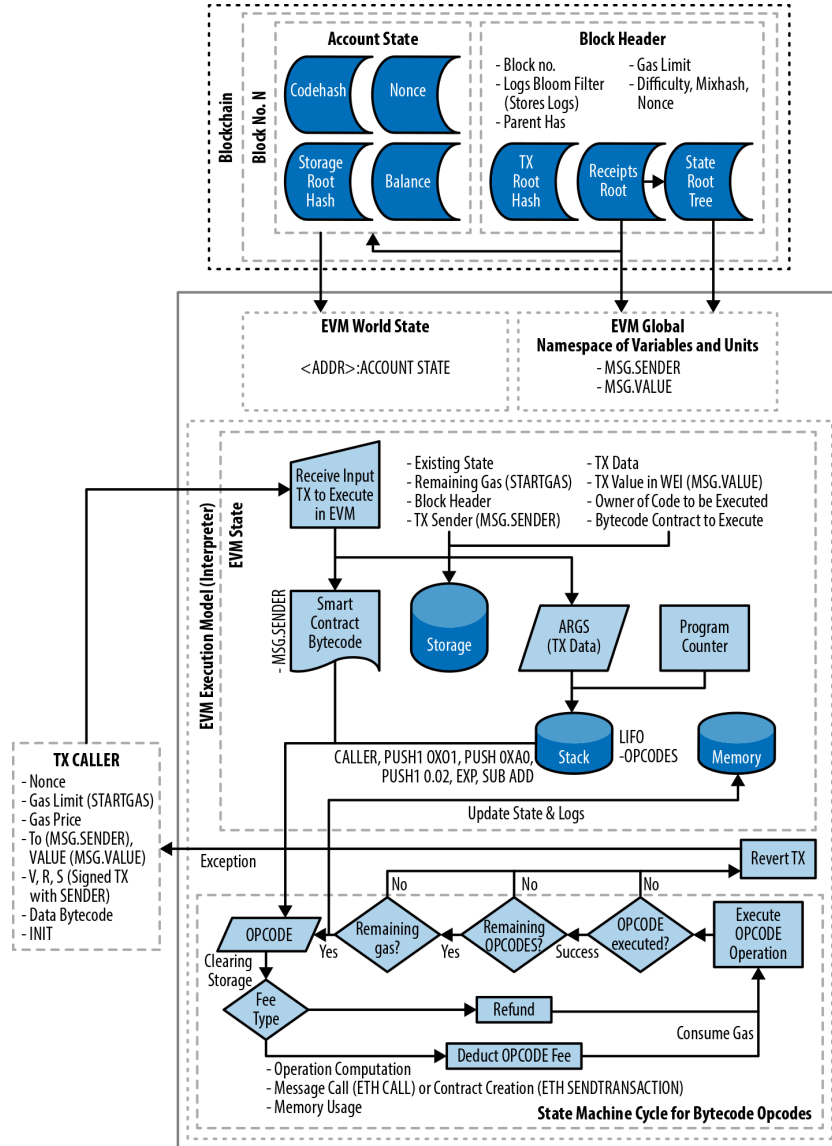


Figure 5.3: A complete diagram of how the Ethereum Virtual Machine works [18].

The *EVM instruction set*, that is, the set of bytecode operations, covers every operation one might need, such as i) arithmetic operations, ii) message calls, iii) access to memory, storage and the stack, iv) control-flow operations, and many others [18]. Besides the usual bytecode operations, the EVM also has access to information about accounts and blocks. The EVM opcodes are the operations it provides, for example `DIV`, which divides two integers, or `BALANCE`, which returns the available balance of a given account [18, 42]. All operations interact with the stack and place their results on top of it [18].

Ethereum is a transaction-based state machine, that is, a system based on transactions. This reflects the fact that external actors (account holders) initiate state transitions by creating transactions [18].

The EVM is a Turing-complete system, but it is also limited by the amount of gas needed to execute any instruction. Infinite loops could therefore lead to denial-of-

service attacks, yet the limit on gas use means these attacks cannot actually be carried out [40]. In practice, Ethereum deals with the problem like this: to deploy and publish a contract, it applies a limit expressed as gas consumption and accumulates the cost of the transactions, so that they are finite. The limit on gas consumption during smart contract deployment is also visible in the diagrams of the study by Tonelli et al. [48]. That study examined 12,094 smart contracts and compared them using general metrics and metrics specific to decentralized applications (calls to and from other addresses, internal calls within the smart contract, gas consumption, cryptocurrency transactions, and bytecode/ABI). All of these metrics appear to reach an upper bound, which is due to the limit on the gas that smart contracts can consume. The smart contract is a very important part of decentralized applications, because it works as their database. All data stored in a contract is stored uniquely at that contract's address, that is, in its state, and is returned from there. Solidity is also compiled into a low-level programming language, namely bytecode, with the help of the EVM.

The EVM also supports exception handling: if an exception occurs, the machine stops immediately and returns the execution error [40] [18]. Code running in the EVM has no access to external resources such as a network or a database, which makes it extremely secure.

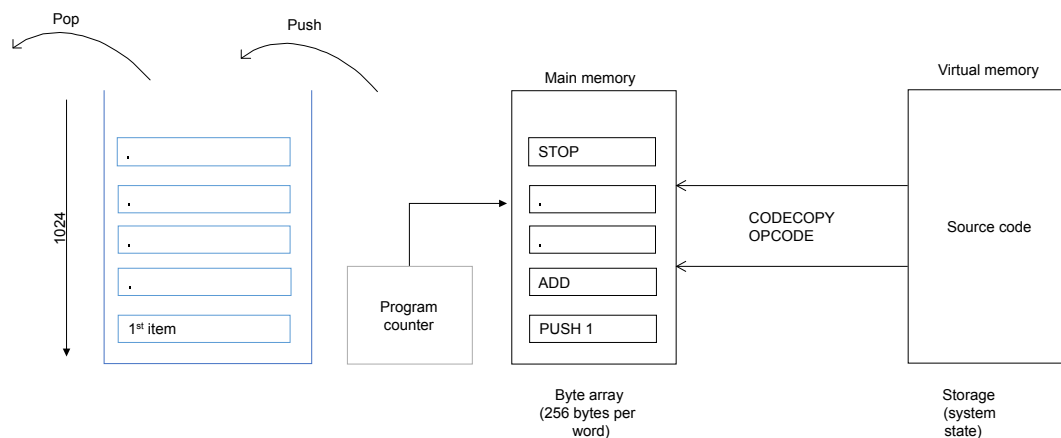


Figure 5.4: An abstract diagram of how the Ethereum Virtual Machine works.

5.7 Smart contracts

The term “smart contract” is an old one with historical significance. In the 1990s, the cryptographer Nick Szabo coined the term and defined it as “a set of promises, specified in digital form, including protocols within which the parties perform on other promises” [18]. The concept of smart contracts has evolved since then. Smart contracts have the following four main characteristics [18]:

- They are computer programs and have no legal standing.
- They are immutable. Once they are deployed on the blockchain, their source code cannot change, which is not the case with other software.
- They are deterministic. The result of executing a smart contract is the same for everyone who executes it.
- They contain EVM operations. Smart contracts operate within a limited execution context: they can access their own state, the context of the transaction that called them, and information about the most recent blocks.

A smart contract is a collection of code and data deployed on a blockchain [1]. Transactions on the blockchain can send data to the public methods that the smart contract offers. The contract runs the appropriate method with the data the user supplied, in order to perform a service [1]. Because the code on the blockchain cannot be changed, it can be used as a trusted third party for financial transactions that are more complex than simply sending funds between accounts [1]. A smart contract can perform calculations, store information and send money to other accounts automatically [1]. In practice, when the mining nodes mine new blocks, they all execute the smart contract at the same time [1]. The user who publishes a transaction to a smart contract often has to pay for the cost of executing the code, on top of the transaction fees [1]. There is a limit on how much execution time a call to a smart contract can consume [20]. If the limit is exceeded, execution stops and the transaction is rejected. This mechanism rewards the “miners” for executing the smart contract’s code, and it also stops malicious users from infiltrating smart contracts and causing a denial of service on the mining nodes [1].

Smart contracts cannot execute on their own or run in the background. Transactions are *atomic*, regardless of which contracts they call: they execute in full and change the world state only when they succeed. Otherwise, the world state is not affected and the smart contract returns to its previous state [18].

In a dApp, smart contracts are used to store the business logic (the program code). In practice, a smart contract replaces a server-side component of a centralized application [18]. One of the main differences is that every computation executed in a smart contract is very expensive and must be kept as small as possible. It is therefore

important to identify which parts of the application need a trusted, decentralized execution platform [40]. Ethereum makes it possible to build architectures out of smart contracts that form a network with one another, reading and writing their own variables, with their complexity limited only by the gas limit [18] [40]. A smart contract can destroy itself if it has been programmed with an opcode, `SELFDESTRUCT`, but apart from being removed completely, its code cannot be changed in any way [18]. Another important drawback of smart contracts is the size of decentralized applications: a very large smart contract can cost a lot of gas to deploy and to use [18]. Some applications may therefore have an off-chain computation process as well as external, off-chain data storage.

ERC20 For our application we chose the ERC20 token smart contract, which is public [49]. It works by creating a number of “pseudo-cryptocurrencies” that are owned and managed by whoever deployed the smart contract and that behave like an ordinary cryptocurrency. These coins are created once and cannot be produced again or transferred without the creator’s approval. All transactions in these coins are tied to the contract’s address; that is, the list of transactions made with this token appears at the contract’s address. Any transfer between the sender’s address and the recipient’s address can therefore be verified, so the transactions are transparent. Finally, the amount of tokens sent to the recipient each time is subtracted from the sender’s balance, and both balances are transparent.

5.7.1 Data storage

Because gas is expensive and the block gas limit is low, smart contracts are not suitable for storing or processing large amounts of data. Most decentralized applications therefore keep their bulky data off the Ethereum chain, in a data storage structure [18]. This structure can be centralized (for example, a typical database), or the data can be decentralized, that is, stored on a P2P platform such as [18]:

- **IPFS.** The Inter-Planetary File System (IPFS) is a decentralized, content-addressable storage system (that is, every content object is hashed with a hash function) that distributes the stored objects among users in a peer-to-peer network. Any file can be retrieved from any IPFS node by its key. IPFS aims to replace HTTP as the protocol of choice for delivering web applications. Instead of sitting on a single server, a web application’s files are stored on IPFS and can be retrieved from any IPFS node.
- **The Swarm platform.** Swarm was created by the Ethereum Foundation as part of the Go-Ethereum tool suite. As with IPFS, any Swarm file can be accessed simply by referring to it through a hash key. Swarm makes it possible to reach a website through a decentralized P2P system instead of a central web server.

Decentralized P2P storage is ideal for storing and distributing large static assets, such as images, videos and the resources of the application's web interface. The processes of a decentralized application also communicate with each other through Whisper, a tool that belongs to the Go-Ethereum tool suite.

5.8 Decentralized applications (dApps)

A dApp is an application that is mostly or entirely decentralized. The parts of an application that may be decentralized are [40]:

- The backend.
- The frontend (user interface).
- Data storage.
- Message communication.

Each of these can be somewhat centralized or somewhat decentralized. For example, a frontend can be developed as a web application running on a central server or as a mobile application running on a device [18]. The backend and the data storage can live on private servers and databases, or a smart contract can be used instead, with the data stored on a P2P network [18]. Building a dApp has many advantages that a typical centralized architecture cannot offer [18]:

1. **Resilience.** Because a smart contract controls the business logic, the dApp backend will be fully distributed and will run on a blockchain platform, so the application will always be available. A dApp deployed on a centralized server, by contrast, will remain available for as long as the platform is running.
2. **Transparency.** The on-chain nature of a dApp lets everyone inspect the code and be more confident about how it works. Any interaction with the dApp will be stored on the blockchain forever.
3. **Censorship resistance.** As long as users have access to an Ethereum node, they will be able to interact with a dApp without interference from any central authority. No service provider, not even the owner of the smart contract, can change the code once it has been published on the network.

In the Ethereum ecosystem as it exists today, there are very few truly decentralized applications, because most still rely on centralized services and servers for some part of their operation [18].

5.8.1 Tools for building a smart contract

Geth

Go Ethereum is one of the three original client implementations of Ethereum (along with C++ and Python). It is written in Go, is fully open source and is licensed under the GNU LGPL v3 [50].

MetaMask

MetaMask is a digital wallet that runs as a web browser extension (Chrome, Firefox, Opera, Brave Browser). A digital wallet is needed, and MetaMask is the most user-friendly one. It is easy to use and convenient for testing, because it can connect to various Ethereum nodes and test blockchains. MetaMask is a digital wallet that requires the user to create an account in it. Once users have created a MetaMask account, or imported an existing account they hold, they can connect to a decentralized application, provided they accept the connection and access message in the MetaMask pop-up window when they enter that application. Creating an account in MetaMask is quite simple. Users choose their own password and must keep the “seed” phrase in a safe place. The phrase is given for security reasons, and it is the only way to restore the account. The MetaMask interface lets the user switch between Ethereum networks, and other networks too. It can connect to the main network, a local network or a test network. MetaMask is responsible only for managing its accounts on Ethereum. The connection to the network nodes goes through Infura, an application that runs in MetaMask’s backend and that the user never deals with [51].

Programming languages

Backend

- Serpent. This is a simple, clean Python-like language. It is no longer used to develop contracts and is no longer supported by the community [18].
- Solidity. It was created by Dr. Gavin Wood. This language has now become almost the standard for writing smart contracts for Ethereum. It is essentially a procedural programming language with a syntax similar to JavaScript, C++ or Java [40] [18]. It is a statically typed language, which means that variables in Solidity are type-checked at compile time [18]. Every variable, whether a state variable or a local one, must be declared with a type at compile time. This is useful because the contract is validated and checked when it is compiled, so some kinds of errors can be dealt with earlier, during the development of the contract, instead of at run time, which is costly [18]. Other features of the language are inheritance, libraries and the ability to define complex data types. Solidity is a contract-oriented programming language: contracts are the equivalent of classes in other object-oriented programming languages [18].

- Vyper. This is an experimental Python-like language being developed with a focus on security, simplicity and auditability in smart contract development [18].

Frontend

The user interface can be built with any widely known programming language used to create websites, for example Django (Python), HTML, CSS (Bootstrap), JavaScript (jQuery, VueJS, Angular, Js-React, Gatsby), PHP, C# and many other tools. However, for the web application to communicate with Ethereum, a JavaScript library, the Web3 library, has to be added to it, together with the necessary functions that this library provides. The vast majority of wallet- and node-related functions are provided by the `web3` object, which belongs to the `web3.js` library. Essentially, `web3.js` is a collection of Ethereum-specific functions. It works as a JavaScript API that is compatible with Ethereum and communicates with users through JSON-RPC. JSON-RPC is a simple, lightweight remote procedure call (RPC) protocol. The specification mainly defines several data structures and the rules for processing them. It is transport-agnostic, because it can be used within the same process, over sockets or over HTTP [18].

Compilers

Compilers convert a smart contract's high-level source code into a form that the Ethereum execution environment can understand. The `solc` compiler converts Solidity from a high-level language into Ethereum Virtual Machine (EVM) bytecode. `solc` is mainly for Linux and macOS systems [18]. There is also an online compiler, Remix, which does not have to be installed locally on the computer [18]. Remix is a powerful open-source tool that helps users write smart contracts in Solidity directly from the browser. The source code of Remix is written in JavaScript. Remix also supports testing, debugging and deploying smart contracts². It is a feature-rich IDE that does not need to use the public version of the blockchain [18]. Remix offers various functions, such as transaction interaction, options for connecting to the JavaScript VM, configuration of the execution environment, debugging of errors that arise in smart contracts, formal verification and analysis [18]. It can be configured to connect to execution environments such as the JavaScript VM, injected Web3 and the MetaMask wallet [18]. Remix also has a debugger for the EVM, a very powerful tool that can be used to trace and analyze EVM bytecode in detail [18].

Testing environment

To check that their smart contracts work correctly, exactly as they intend, developers have to run them on the blockchain network. However, because executing every transaction on the main chain costs money, several parallel chains were created for developers only. On these parallel chains anyone can run their smart contracts exactly as on the

²Source: remix-ide.readthedocs.io/en/latest/

main network, except that the developer bears no financial cost, since they use pseudo-coins that the network in question provides. There are several test networks, among them Ropsten, Rinkeby and Kovan. The best known is Ropsten, which most developers use. MetaMask supports all three of these networks.

Truffle - Ganache

Truffle is an application development environment that makes testing and developing smart contracts on Ethereum easier and simpler [40]. Truffle compiles contracts and links them to an automated environment where they can be tested with the Mocha and Chai frameworks. It also makes it easier to deploy contracts to any private, public or test Ethereum network [40]. These tests are run together with Ganache [18]. Ganache is a local test blockchain that can be used to develop smart contracts. It is available as an application with a graphical user interface that runs on all operating systems (Windows, macOS, Linux), and also as a command-line program [18]. Ganache is the latest addition to the many tools and libraries developed for Ethereum. It is a JavaScript implementation of the Ethereum blockchain, with a built-in block explorer and mining, which makes local testing very easy [18].

5.8.2 Gas

Gas is the Ethereum unit that measures the computation and storage resources needed to perform actions on the Ethereum blockchain through smart contracts [18]. Every operation that a smart contract performs costs a fixed amount of gas, for example [18]:

- Adding two numbers costs 3 gas.
- Computing a Keccak-256 hash costs 30 gas + 6 gas for every 256 bits (32-byte word) of data being hashed.
- Sending a transaction costs 21,000 gas.

Gas is a cornerstone of Ethereum and has a dual role: *i*) as a buffer between the (volatile) price of Ethereum and the reward that miners receive for their work, and *ii*) as a defense against DoS attacks on smart contracts in Ethereum [18]. To prevent an accidental or malicious loop, or any other waste of computation on the network, whoever initiates a transaction must set a limit on the amount they are willing to pay [18]. The gas system therefore discourages attackers from sending “spam” messages, because they have to pay in proportion to the computation, bandwidth and storage resources they consume [18]. Every opcode that is executed has a gas cost, which is subtracted during execution from the limit that the initiator set at the start of the transaction. If the gas runs out before the transaction completes, the transaction is cancelled, an out-of-gas exception message appears, and the amount is refunded through the EVM, apart from the transaction cost, which has gone to cover the miners’ operating expenses [18]. If the EVM reaches the end of execution successfully without running out of gas, the gas

cost paid to the miner as a transaction fee is converted into Ether based on the gas price, using the following function [18]:

$$\text{miner fee} = \text{gas cost} \times \text{gas price}$$

The gas left over from the transaction (essentially the “change”) is returned to the sender through a similar process: it is converted into Ether based on the gas price that was set for the transaction [18]:

$$\text{remaining gas} = \text{gas limit} - \text{gas cost}$$

$$\text{refunded Ether} = \text{remaining gas} \times \text{gas price}$$

Gas cost versus gas price

Gas cost is a measure of the computing power and storage that the EVM uses, while the gas price is measured in Ether [18]. When a transaction is executed, the sender specifies the gas price they are willing to pay (in Ether) for each unit of gas, and the market decides the relationship between the price of Ether and the cost of computing operations. In other words, the following always holds [18]:

$$\text{transaction fee} = \text{total gas used} \times \text{gas price paid (in Ether)}$$

Miners usually choose the transaction that offers the most gas, so that transaction takes priority over others that offer less [18]. In practice, the sender of a transaction sets a gas limit greater than or equal to the amount of gas expected to be used [18]. If the limit is higher than the amount consumed, the sender gets the extra back, because miners are paid only for the work they actually do. Gas cost and gas price differ as follows [18]:

- Gas cost is the number of gas units needed to perform a particular operation.
- Gas price is the amount of Ether that the sender is willing to pay per unit of gas when sending their transaction to the Ethereum network [18].
- The block gas limit is the maximum amount of gas that all the transactions in a block can consume, and so it limits the number of transactions that can fit in a block [18].

Take, for example, 5 transactions whose gas limits have been set to 15,000, 15,000, 20,000, 25,000 and 25,000. If the block gas limit is 85,000, any four of these transactions can fit in a block, while the fifth will have to wait for a future block. The miners decide which transactions go into the block, and they also vote democratically, with a voting factor of 0.0976%, on the gas limit of each block. If a miner tries to include a transaction that needs more gas than the current block gas limit, the network rejects it with the error message “transaction exceeds block gas limit” [18]. According to etherscan.io, the block gas limit on the Ethereum network is 8 million gas, so about 380 transactions (each consuming 21,000 gas) could fit in a block [18].

5.9 Decentralized applications

Hiring employees A company can easily obtain a lot of information about a job candidate within a few minutes from many sources and services, such as the criminal record, and all of this information would be indisputable [52].

Supply chain Blockchain lets anyone check the stages a product went through on its way to the consumer. This makes it possible, by “scanning” a QR code, to confirm the product’s authenticity and quality and to crack down on smuggling [52].

Intellectual property protection Copyright holders can easily use websites such as **binded.com** or **copytrack.com** to upload their material (e.g. photographs) and register their copyright with a simple digital signature. Once an agreement has been reached, they can also allow third parties to reproduce the material lawfully [52].

Weapons tracking Blockchain can help track weapons, because it stores information about a weapon from its production up to its purchase by the last customer, and this information could be made available to the police and to gun sellers [52]. One application, **blocksafe**, is already under development. It has fitted new guns with sensors and a decentralized VPN, which let a gun send data to and receive data from the blockchain. The system can locate the gun and also receive real-time data on where and when the gun was fired, and even at what angle [52].

Storing personal data Separate documents such as the identity card, the driving licence and the tax identification number (AFM) could be replaced by a digital identity based on blockchain technology [52]. The real owner could then access this data through biometric analysis of their features, so in the event of a natural disaster or other catastrophe, their data would remain safe [52].

Games

Play2win

This is a virtual casino implemented with an Ethereum smart contract. It has 1000 active users every day. Players bet Ether on various casino games and can win or lose Ether. The smart contract determines the winners and the losers. In this game, the smart contract defines the random number generation system used to pick the winners. The random number generator algorithm is based on the number of data items in the block, the block’s hash, the transaction confirmation time and the current block difficulty³.

0x- Universe

0x-Universe is a collectible game in which players build spaceships and colonize planets. It has 700 active users in every 24-hour period. Each planet is a digital asset stored on the blockchain. There is a finite number of planets, and each has a unique design and

³More at <https://playtowin.io/>

offers different resources. The game charges a 5% transaction fee every time a planet is bought or sold in its marketplace. The planets are regular ERC721 assets that can be bought and sold on other exchanges, such as Opensea⁴.

Health

EDNA

EDNA members are effectively shop owners: they use the company's cryptographic technologies to secure their genetic data on the blockchain and can sell this data, for a fee, to any researchers who want it. In other words, they use smart contracts to share their data with researchers and get paid for it. Finally, through its operation EDNA also does charitable work to fight sex trafficking⁵.

HEALTHHEREUM

HEALTHHEREUM aims to make the user's experience of electronic healthcare friendlier. The platform enriches provider-patient interactions, makes continuous feedback easier, promotes ongoing patient engagement and enables ownership of medical data. The Healthereum Patient Behavior Management platform is a communication tool that gives healthcare providers a way to solve critical healthcare problems⁶.

Governance

Marriage On Theblock

Marriage On Theblock aims to do away with some of the basic functions of the civil registry office, such as registering or dissolving a marriage. Recording the details of a marriage on the blockchain has many benefits. For example, the spouses can read the certificate and get a copy within a few seconds, even from the comfort of their home. They also do not need to queue at the registry office to submit an application, since this is now done electronically, and they can change their marital status whenever they wish⁷.

Kleros

Kleros is a court system. Smart contracts must designate the judge as their arbitrator. When they do, the contract creators choose how many judges and which court will rule on their contract if a dispute arises. The idea is that they will choose a type of court that specializes in the subject of the contract: a software development contract will choose a software development court, an insurance contract will choose an insurance court, and so on. The Kleros team has developed a series of standard contracts that use Kleros as their dispute resolution mechanism⁸.

⁴More at 0xuniverse.com/

⁵More at edna.life/

⁶More at healthereum.com/

⁷More at www.marriageontheblock.com/

⁸More at kleros.io/en/

Education

Disciplina: Blockchain for Education

Disciplina [53] was founded in 2018 and was the first blockchain to create a unified system both for verifying academic qualifications and for organizing academic activities. The concept consists of a private and a public blockchain, both built on its own technology. The private blockchain has two main roles, the Student and the Educator, and it stores all the processes carried out between them, such as assignments, weekly grades and the final grade. The Educator can be either a single teacher or an entire academic institution. However, as Kuvshinov et al. point out in their paper, the data that the Educator puts on the chain could be tampered with, so a third role is needed: the Witnesses, who take as input the headers of the transactions made by the Educator and publish them on the public blockchain. There is also the role of the Recruiters, the entities that want to collect data about students from educational institutions. They buy this data from the Educators through a secure data disclosure protocol. Academic qualifications are verified without any of these roles having to intervene. The student only needs to submit some information: their own public key and the Educator's, the course x and the hash of its transaction, the Merkle tree path of the private transaction ($P_{\text{priv}} = \text{path}(T_{\text{priv}}, M_{\text{priv}})$), and the block number together with the Merkle tree path of the public transaction. Combining and checking all of the above is enough to verify that the transaction between the Educator and the student is real and genuine.

Verde

As mentioned in all the sections above, the strongest point of blockchain technology is the trustworthiness and authenticity of its transactions. The Verde application uses the Ethereum platform and aims to crack down on the forgery of academic qualifications.

Chapter 6

The Verde decentralized application

As mentioned in the introduction, news agencies quite often report cases of forged degrees discovered in the public sector and elsewhere. Forged degrees are common in Greece, so this illegal practice needs to be tackled in the best possible way, which is prevention. Verde is a verification tool for combating the forgery of university degrees. When a student graduates, the university department will publish the student's grades on the Ethereum network and send them to the graduate's digital wallet, so that the graduate's records cannot be tampered with. In turn, a company, public or private, can easily verify the validity of the former student's degree: once the graduate's digital wallet address is entered into the Verde web application, the application automatically displays all the student details and courses that the university department recorded on the blockchain.

Figure 6.1 shows the logic behind Verde. There are three entities, or levels: 1) the ministry, 2) the academic institutions and 3) the students. The first level is responsible for allocating ECTS tokens to the institutions, which then distribute them to their graduating students. This three-level structure is very important because the central authority decides which universities may distribute ECTS tokens, and so it can itself reverse any violation by an institution or a student. All ECTS tokens therefore have a common authority and a single issuer, which gives the highest possible assurance of data integrity, and therefore of verification. This process was not fully implemented within the scope of the thesis. Instead, the application can be compared to a transaction between levels 2 and 3. It matters who deploys the smart contract on the blockchain, and here the academic institution deploys it instead of the ministry. At the smart contract level, this function has been implemented: the creator of the ECTS tokens allows another account to use and distribute the tokens it sends to that account.

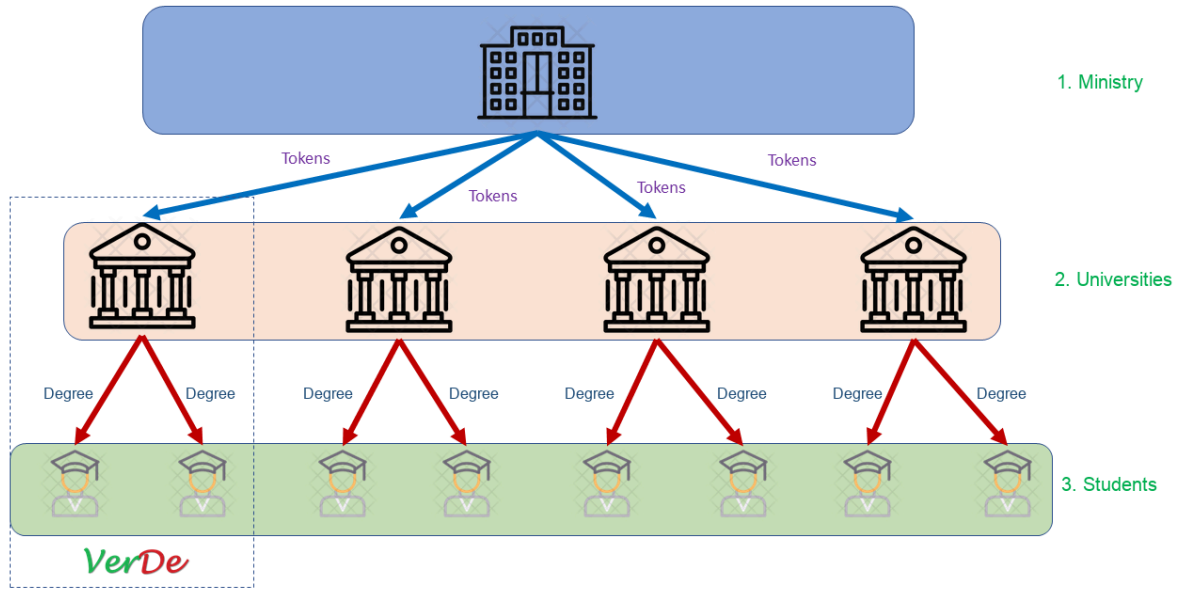


Figure 6.1: The levels of the application.

6.1 The architecture of Verde

This section presents the architecture of the Verde decentralized application. Like all decentralized web applications, Verde has three basic components: (a) the presentation component, (b) the control component, and (c) the component that communicates with the blockchain. The first is the user interface (Figure 6.2), which reads the data the user enters. The third works through the web3.js library and communicates with the Ethereum blockchain [54]. The control component connects the other two and acts as the “brain” of the application [54]. The proposed architecture on which Verde was built is described in detail below.

6.1.1 Purpose of the application

Verde relies on the trustworthiness of the Ethereum blockchain to verify academic qualifications without the academic department having to step in each time. Its front-end consists of two interfaces, as shown in Figure 6.2: 1) the registration interface, which records the data on Ethereum, and 2) the verification interface, which verifies the data. Having two interfaces keeps the registration interface, which each academic department manages separately and exclusively, apart from the verification interface, which is public and therefore open to anyone interested. The back-end connects to the front-end through the web3.js library, which is installed in the interfaces and, together with the smart contract, makes up the application’s back-end (Figure 6.2). In practice, the smart contract provides the application’s functionality and its decentralization.

Connecting to the blockchain, however, requires a digital wallet and a connection to one of the blockchain’s full nodes. The digital wallet holds the user’s funds, and transaction costs are deducted from it (it “filters” the transactions). The full node (which holds whole blocks) keeps up to date with the blocks of the blockchain and also creates new ones. The MetaMask application handles the communication between these two, and it is also part of Verde’s back-end. MetaMask was developed by ConsenSys and is a web-based digital wallet application (a browser extension). The connection to the nodes of the Ethereum network goes through the Infura application, which runs in MetaMask’s back-end. Finally, Verde complies with the GDPR and provides a function for revoking academic qualifications.

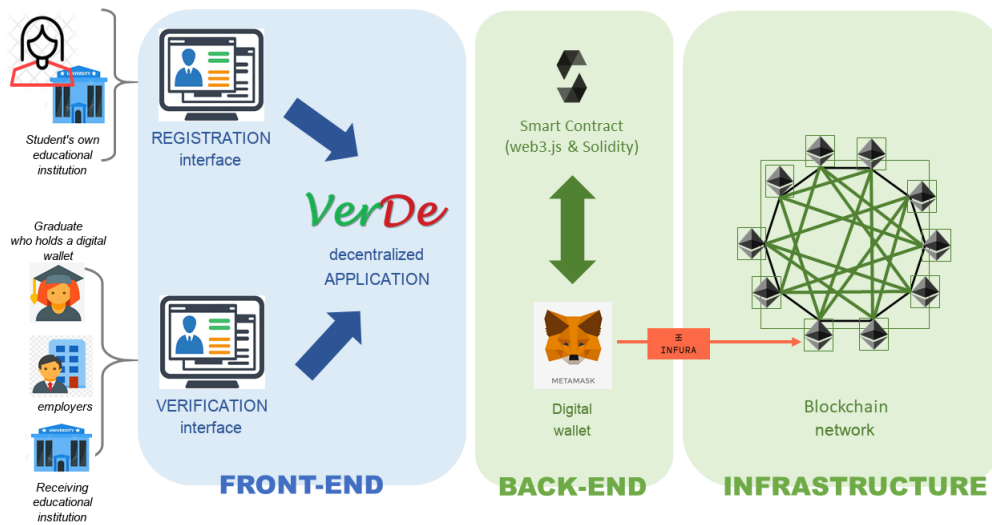


Figure 6.2: The abstract structure of the application.

We built Verde on Ethereum because it is currently the second-largest blockchain network [55] by market capitalization, number of nodes and transaction frequency. It is also public (permissionless), we do not have to set up the nodes ourselves, and a large community supports it.

6.1.2 Architecture of the registration interface

The registration interface sends the student’s encrypted data from the academic department to the Ethereum network, so that the details of the student’s academic qualification can later be decrypted and verified. These details include the student’s full name, father’s name, mother’s name and date of birth, the bare minimum needed to identify the student. They also include information on the department, the courses, the credits (ECTS) and the grade in each course. All of these details are needed to verify the academic qualification and to match it to the student. Verde’s functions must therefore be able to display these details for the qualification to count as verified. Before

they can be displayed, however, they must first be recorded on the blockchain, and this is the job of the registration interface.

A transfer on the blockchain always takes place between a sender and a recipient, here the digital wallet addresses of the academic department and of the student, and the medium is Ethereum. Any academic department can use this interface, as long as it has its own Ethereum digital wallet. Registering the details and transferring them to the blockchain takes eight steps, shown in Figure 6.3 below.

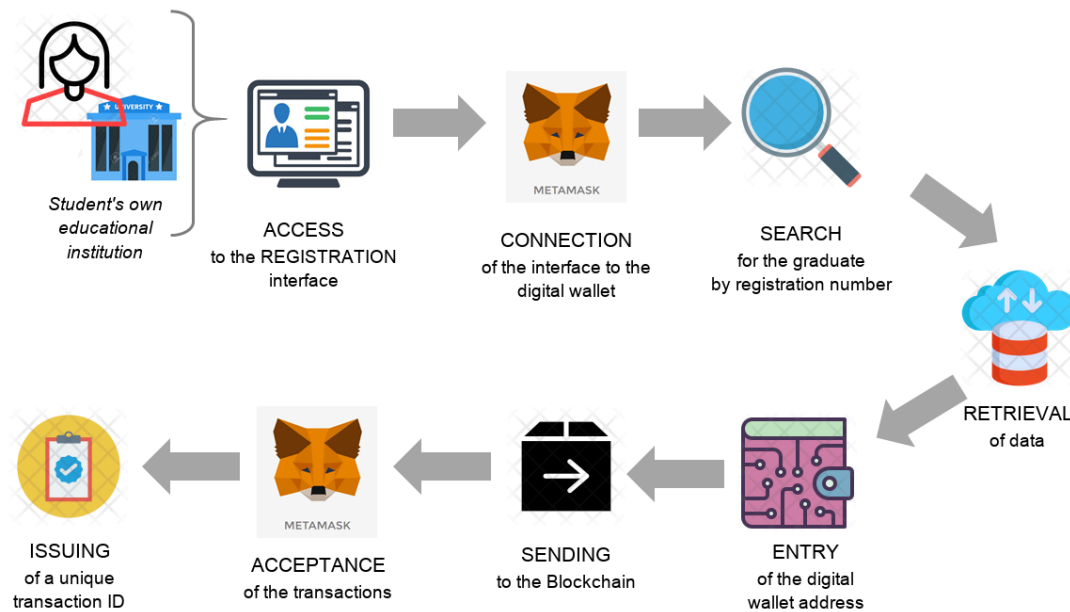


Figure 6.3: Registration of the student's details by the student's academic department.

6.1.3 Architecture of the verification interface

The verification interface is also easy to use and needs only four steps to do its job (Figure 6.4). It works by searching for the graduate's digital wallet address. The search retrieves the encrypted information about the graduate from Ethereum, more precisely from the state of the smart contract, and then decrypts it. This is the information that the registration interface transferred to Ethereum, so what comes back is the graduate's personal details and the details of the courses the graduate completed during their studies. The purpose of this interface is therefore to verify the qualification details of a prospective employee or postgraduate student. Academic departments save operating time and cost in this way, since the Ethereum blockchain has taken over this role. This credential is so strong because data on the blockchain cannot be altered.

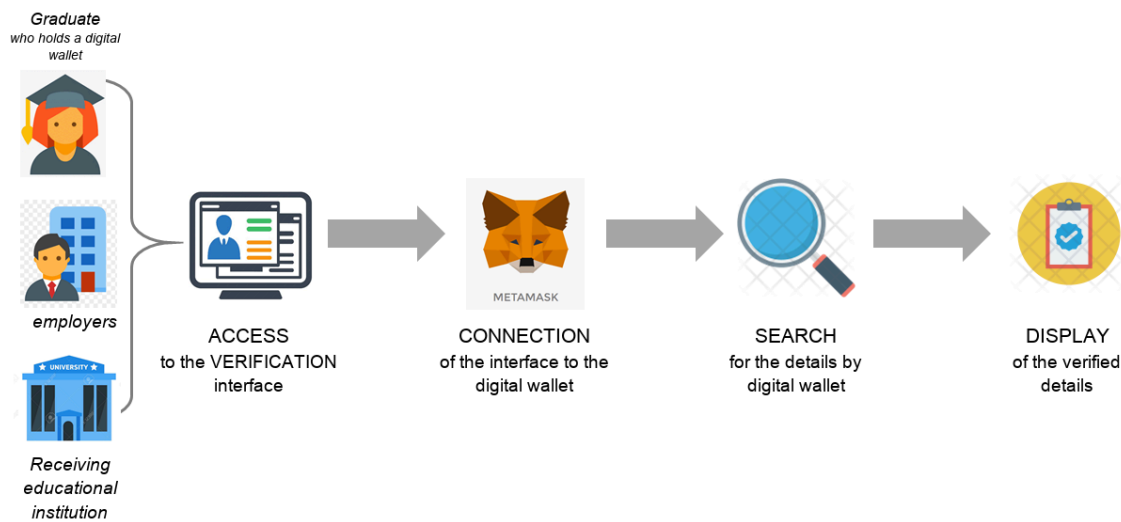


Figure 6.4: Verification of a graduate's academic qualification.

6.1.4 Architecture of the smart contract

The smart contract is the back-end of the decentralized application. Users never come into contact with its code and do not interact with it directly. The contract is very important for the application because it carries out the registration and verification of academic qualifications. It needs the following functions:

- It stores the student's encrypted personal details in the smart contract (first name, surname, mother's and father's names, place of origin, date of birth).
- It stores the student's grades, encrypted courses, grade average and ECTS.
- It creates the "pseudo-cryptocurrencies" called ECTS, so that it can send the student as many of them as the student earned during their studies.
- It returns the student's encrypted details.
- It returns the student's grades, encrypted courses, grade average and ECTS.
- It uses the ERC20 token to create the ECTS. The application creates a very large supply of these ECTS: the integer part of the number has 7 digits and the decimal part has 18. The token is there to double-check the student's details, thanks to its transparency. Once the smart contract is deployed on the blockchain, it cannot be changed. It is therefore important that its functions are strictly defined, that it is tested against every possible case, and that great care goes into its implementation.

6.2 Implementation of Verde

In this section we look at the implementation details of Verde. The interfaces were built with HTML, CSS (Bootstrap), Javascript (jQuery) and PHP. The smart contract is written in Solidity 0.5.7 and was developed in the Remix environment, on the Ropsten network.

6.2.1 Implementation of the registration interface

The registration interface¹ is easy to use and was implemented in PHP. The application uses four separate HTML forms to collect all the details needed to verify a student's academic qualification:

1. The first contains the university logo and the name of the school (Figure 6.5).
2. The second has the fields for the student's personal details and for searching for the student in the university database. When the search is triggered, the PHP code runs a Mysql query that returns all the details of the student with that registration number. If the retrieval succeeds, the fields are filled in with the student's details; otherwise, it returns nothing (Figure 6.5).
3. The third form contains the table of the courses the student passed during their studies (course names, credit units/ECTS, grades). These details come from the search in the previous step and are filled in with the help of PHP (Figure 6.6).
4. The fourth contains the fields for the digital wallet addresses and sends all the data to Ethereum. Each academic institution has its own unique digital wallet address, so the corresponding field is pre-filled. The student's digital wallet address is also unique, but the student supplies it, so that the student's wallet can be updated with the details of the qualification they are about to be awarded (Figure 6.6). Before the sending can complete, the transaction costs must be accepted in MetaMask. Once the last transaction has also completed, its unique identifier (transaction id) is displayed, as shown in Figure 6.7.

¹The application is published at: <https://dry-sierra-28168.herokuapp.com/index.php>

1. Header form of the student's school

CERTIFICATE
THE FOLLOWING DATA CERTIFIED:

Personal Info:

Last Name: last name

Mother's Name: mother's name

Birth Place: birth place

On Register numbers: mtdroo

Name: first name

Father's Name: father's name

Birth Year: birth year

Number: No.

Registration Info:

Register Year: date of register

Type Of Registration: type of registration

Registration documents: documents

Academic ID: registration number

Reg. Semester: semester

Atom. Delt. Epil:

Specialization: department

Search key (registration no.)

search

Button for searching the DB

Figure 6.5: Forms 1 and 2 of the interface and the retrieval of data from the DB.

Class

C.C

ECTS

Grade

Semester A

class name dm ECTS grade

class name dm ECTS grade

class name dm ECTS grade

Semester B

class name dm ECTS grade

class name dm ECTS grade

Semester C

class name dm ECTS grade

class name dm ECTS grade

University administration Address

0xC8BCF9ee3Ba588706532D9e4E52731cd1949aa7

Digital wallet address of the secretariat

Student Address

Student's digital wallet address

Send the student's address (public address)

Send

Button that sends the data to the blockchain via MetaMask

4. Digital wallet addresses form

Figure 6.6: Forms 3 and 4 and the sending of data to the blockchain.

Send

Success TX: 0x9fdb7602c3ac43598add1978d5bac6b7160868e8978f94e95bfd1374f1a1954

Figure 6.7: The ID of a successful transaction (transaction id) in hexadecimal form.

Sending the data to the blockchain was implemented with the web3.js library. Figure 6.8 shows the part of the web3.js code that sends the personal details to Ethereum. The first line calls a smart contract method named `setStudent`, and the first pair of parentheses holds the arguments of this method. The `.send` function transfers/stores the data in the smart contract and charges the transaction cost to the sender's address (`fromAddress`). The rest of the code handles and displays the messages that report whether the transaction with Ethereum succeeded or failed. Figure 6.9 shows the code that sends the student's courses to Ethereum, which works the same way as the previous method.

```
Contract.methods.setStudent(toAddress,lastname,name,fname,mname,birthyear,registryear,specialization).send({from: fromAddress},
function(error, result) {
  if (error) {
    console.log('error: ' + error);
    $('#deposit-result').html('<b>Error: </b>' + error);
  } else {
    $('#deposit-result').html('Success TX: <b>' + result + '</b>');
```

Figure 6.8: Sending personal details to Ethereum.

```
Contract.methods.setStudentGrades(toAddress,alldata).send({from: fromAddress},
function(error,result){
  if (error) { console.log('error: ' + error);
    $('#deposit-result').html('<b>Error: </b>' + error);}
  else { $('#deposit-result').html('Success TX: <b>' + result + '</b>');
```

Figure 6.9: Sending the details of the courses the student passed.

The code in the figures above runs when “Send” in Figure 6.6 is pressed. MetaMask then opens with two pending transactions, each of which asks the user to accept the fee for sending to Ethereum. After approval, the miners on Ethereum carry out the mining process and, if it succeeds, the transaction ID appears at the bottom of the interface (Figure 6.7). Deploying the contract costs about \$13, and storing a student costs \$3 each time. The cost rises and falls with the amount of data we store on Ethereum and the price of Ether at the time, and not with the number of tokens sent to the students.

We also used the Javascript tool `cryptojs` [56] to encrypt the students’ details (course names and personal data) with the AES algorithm, using the student’s digital wallet address as the symmetric key. This way the interface follows the GDPR guidelines. The drawback is that the transaction cost went up (from 0.3 to 3 euros), because more digits have to be stored on the blockchain.

Figure 6.10, the sequence diagram of the interface, gives a clearer picture of what happens while this interface runs.

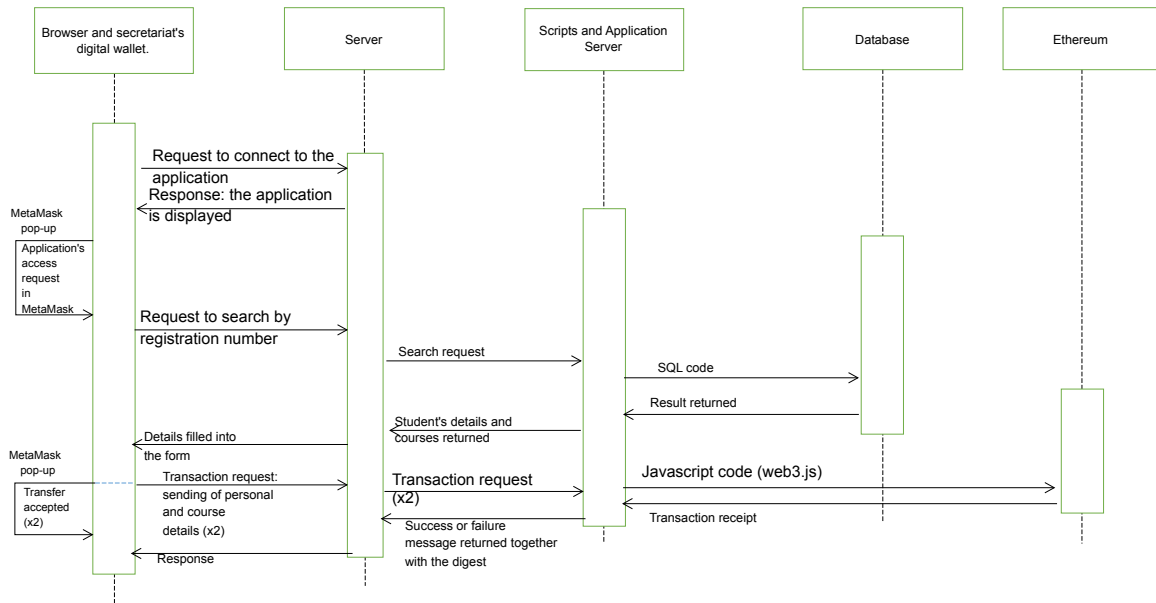


Figure 6.10: Flow diagram of the registration interface.

6.2.2 Implementation of the verification interface

Users can access and use the interface by installing MetaMask in their browser, but this is optional: the interface includes a dedicated tool, `fortmatic.js` [57], which interacts with Ethereum and so takes over the role of MetaMask. MetaMask is used in case the application needs to be deployed on other blockchains as well. The interface also decrypts the student's data with the AES algorithm, using as the symmetric key the digital wallet address that the user enters in the interface.

The design goal was ease of use and easy access. The interface uses two HTML forms: one for searching for a graduate's digital wallet address, and one with the graduate's data, which appears after the search. Verification can also be done with a QR code, thanks to `fortmatic.js` working together with the Javascript code. When the url of the verification interface contains a search parameter whose value equals the student's digital wallet address, that address is placed in the search field, and the search can then run without MetaMask. The QR code is therefore nothing more than the url of the verification interface plus the parameter holding the student's digital wallet address.

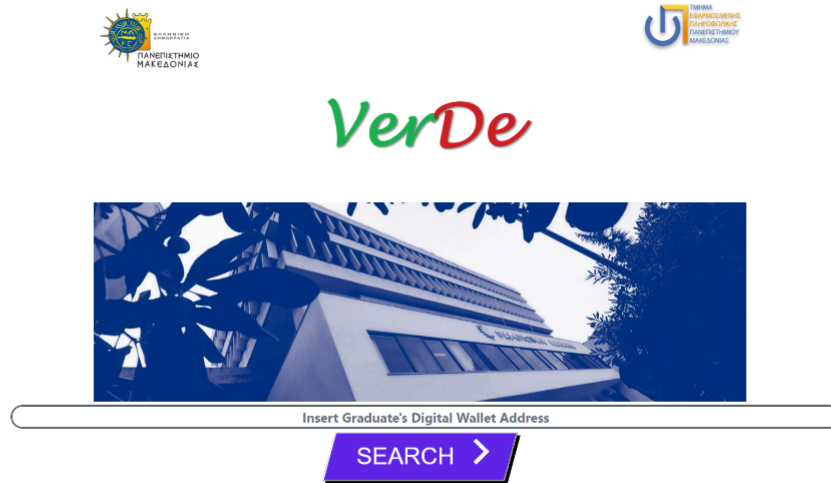


Figure 6.11: The verification interface before “Search” is pressed.

The two main functions of this interface are calling and displaying:

- the graduate’s encrypted personal details, and decrypting them (Figure 6.12);
- the encrypted courses the graduate passed, and decrypting them (Figure 6.13).

```
Contract.methods.getStudent(fromAddress).call(
  function(error, result) {
    if (error) { console.log('error: ' + error); }
    else { console.log(result);
```

Figure 6.12: Displaying the graduate’s personal details.

```
Contract.methods.getGradesOfStudent(fromAddress).call(
  function(error, result) {
    if (error) { console.log('error: ' + error);
    } else {
```

Figure 6.13: Displaying the details of the graduate’s courses.

The two transactions above share the graduate’s digital wallet address (`fromAddress`). They are already stored in the state of the smart contract and are returned from it. The calls that retrieve and verify this information cost nothing, because they do not require any blockchain miner to perform computations. Finally, the interface uses an important tool, obfuscator [58], which turns the Javascript code into a form that people cannot read but the browser still can. This tool partly encrypts the interface’s code, so that a third party cannot work out how the student’s data is decrypted.

6.2.3 Implementation of the smart contract

The smart contract is the back-end that the creators of the application developed. It is the first of the two back-end components and is written in Solidity. The smart contract acts as a regulator of the data that the academic department enters into it through the application (the frontend). It could be likened to the department's database, one that stores the data in a decentralized way. The comparison holds because the data is stored uniquely in the state of a specific address (the Contract Account), produced by the mining of the smart contract, but the contract stores the data on the Ethereum blockchain network and not in a private database. The smart contract consists of two Solidity files: `UomToken`, which represents the user's ECTS, and `StudentGrades`, which contains the code that stores the students' data. Figure 6.14 shows the `UomToken` contract in the Unified Modeling Language (UML). This contract is the parent of the `StudentGrades` contract, so `StudentGrades` inherits the methods declared in `UomToken`. It is part of the ERC20 token standard [49], modified for this application. Most importantly, only the academic department (specifically, the single wallet address of its secretariat) can transfer these units (which stand for ECTS) to the student. The relationship is one-way: the tokens cannot be returned, modified or transferred to a third account². The file `UomToken.sol` contains three main sub-contracts and one library:

- The `ERC20Interface` contract. This contract declares the three functions: one returns the credits the university can still award, one returns the total credits held by a given address, and one transfers credits from the university to the corresponding student.
- The `Owned` contract. It has a constructor, a modifier and two functions for transferring ownership of the coins (usually from the institution to the individual).
- The `UomSupplyToken` contract. As we said, the three functions are declared in the `ERC20Interface` contract; this contract defines them. Besides these three functions, it has a constructor, which holds the main functionality of the smart contract. The constructor sets: a) the name of the token, b) its symbol, c) the number of credits that the (university's) account will hold when the contract starts, d) the transfer of these units to the address that created this smart contract (that is, the address of the university or of the Ministry of Education). Finally, it has other functions, such as `_mint`, which creates new tokens and sends them to the student, `_burn`, which revokes and destroys the student's tokens, and `transferFrom` and `approve`, which are for future use: once the application's

²This property can be seen in lines 106-107 of the code at <https://gist.github.com/gmixoulis/0225779d6b8c75f025cacc2720e701fd>

business logic is complete, a central entity will be able to distribute its coins to the institutions, and they in turn to their graduating students.

- The **SafeMath** library. This library defines the familiar arithmetic operations safely. It is safe because it accepts and returns only integers.

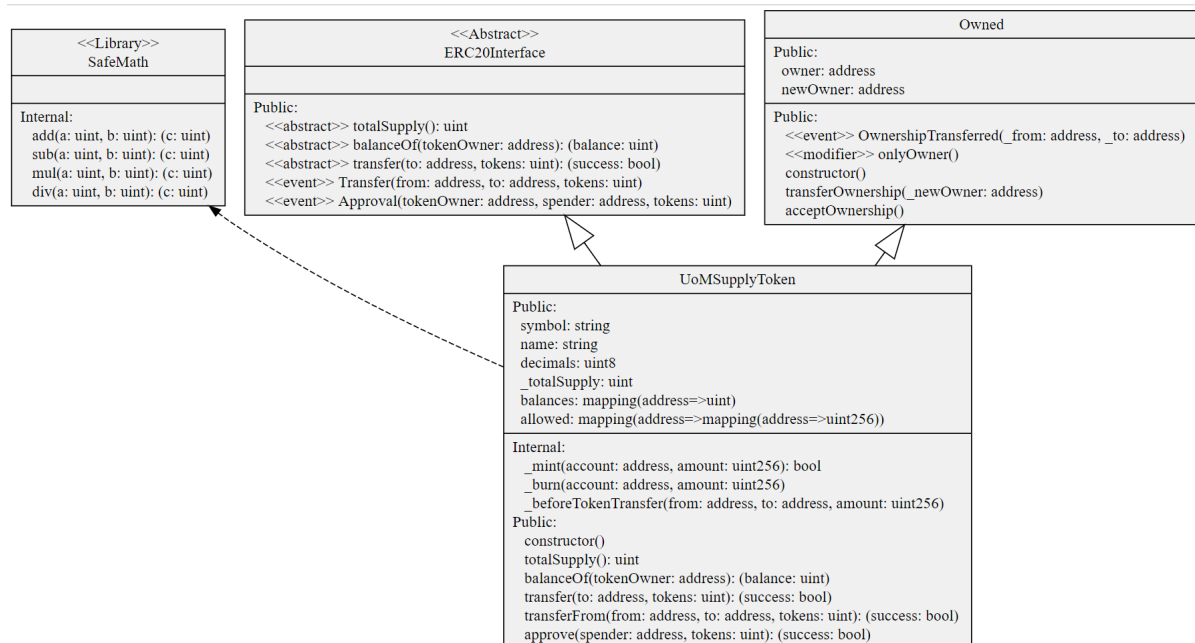


Figure 6.14: The UomToken contract in UML.

The innovation of Verde lies in the file **StudentGrades.sol**. It stores the students' details and their grades in each course, together with the credits that go with each course. Figure 6.15 shows this smart contract in UML. The contract contains seven functions, five data structures of type hashmap (**mapping** = hashmap) and one data structure named **Student**, which holds the student's details. Its functions are the following:

- The **setStudent** function. It takes all the student's details as arguments and returns a boolean value that shows whether the details were stored successfully or the process failed. More specifically, a variable of type **mapping**, **students**, has the graduate's address as its key and the **Student** structure as its value. Through this mapping, the address is bound to all the student's details and to the student's courses, which the next function stores.
- The **setStudentGrades** function. Its arguments are an array with the encrypted names of the courses the graduate passed and an array that holds the graduate's grades together with the credit units of each course. It returns a boolean value that shows whether the function completed successfully. The **Student** data structure contains a variable of type **mapping** named **grades**, and this function stores the

course names and grades in that hash map. It also computes the grade average and the total ECTS earned from the courses. The average is stored in the variable **AvgOfStuddent**, which takes the wallet address as its key and a number (the average) as its value. Finally, the function transfers the total ECTS to the student. It is declared **payable** so that it can send the ECTS to the student.

- The **getStudent** function. It takes the graduate's address as its argument and returns the graduate's details.
- The **getGradesOfStudent** function. It takes the graduate's address as its argument and returns the graduate's grades, ECTS credits and grade average.
- The **IfNotStudentExists** function. It takes the graduate's address as its argument and returns a boolean value (**true** or **false**) that shows whether that address is registered in the contract, specifically in the **studentAccs** array.
- The **mint** function, which will be used only when the institution has run out of the tokens it holds, since it creates new tokens and sends them to the recipients.
- The **revoke** function, which revokes and deletes the student's data from the smart contract.

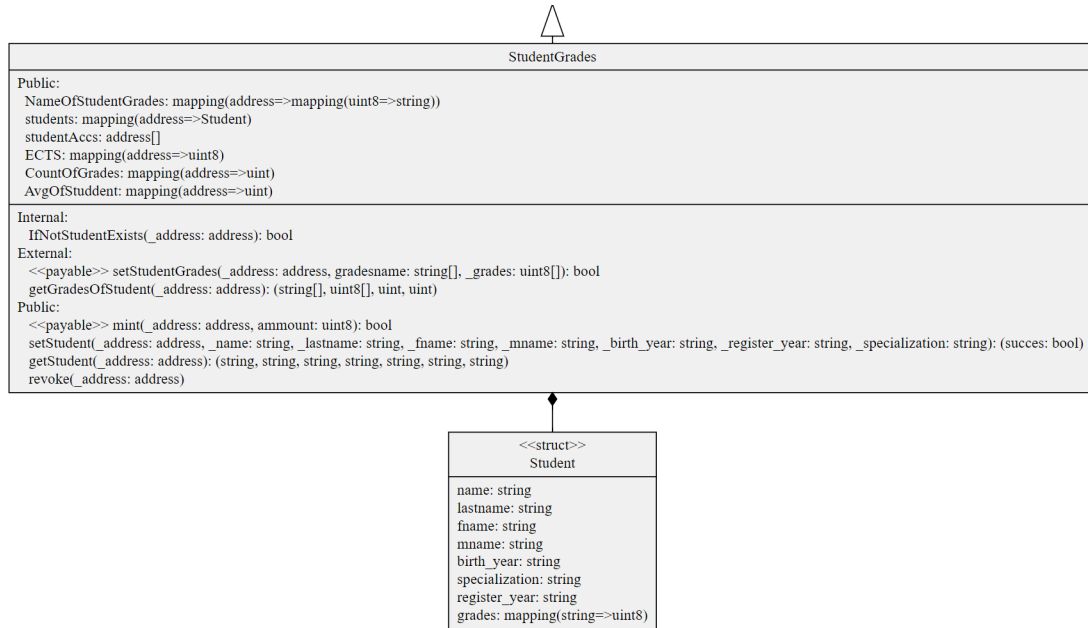


Figure 6.15: The StudentGrades contract in UML.

6.3 Comparing Verde with similar applications

Forging academic qualifications is an unethical and often dangerous act. Being able to verify them quickly and effectively is essential. The Greek state now has legislation in place to deal with forgery. Under document Ref. No. $\Delta\text{I}\Delta\Delta/\Phi.34.1/86/$

οικ.31792/08-12-2014, the Greek public sector urges each body to verify for itself the academic qualifications that job candidates submit as supporting documents. More specifically, the legislation states: *“For this purpose, the services send copies of the candidates’ qualifications, following the attached template, to the competent bodies by fax or by e-mail, so that their authenticity can be established”*. From this we can see that the current legislation is inadequate and quite time-consuming: the process takes considerable time and money to complete, and it is doubtful whether the academic institution will reply at all. The gap left by this legislation created the need for a solution. Solutions came both from institutes and from private parties, and both groups have one thing in common: they used blockchain technology, because it guarantees the validity of the information.

UNIC The first solution came from the University of Nicosia in 2014 [6, 59]. It essentially sends the metadata of the certificate’s PDF to Bitcoin. It then embeds some information in the metadata, such as the digital address of the sender who published the certificate and the hash of the Merkle root of the blockchain in which the certificate’s metadata were stored. The PDF file is verified through a platform the university has built, which checks the file’s metadata. When the law requires a certificate to be revoked, this is done with the Credentialing meta-protocol: a new transaction is published on the blockchain that cancels the certificate’s validity by assigning the appropriate data to Bitcoin’s OPCODE. When a certificate is placed on the platform for verification, any later transactions related to cancelling certificates must also be checked. A cancellation transaction can hold only two certificates at a time.

Blockcerts Blockcerts [7] was founded in 2015. It is a project of the Media Lab Learning Initiative and Learning Machine, which created a platform for verifying academic qualifications. Blockcerts certificates follow the Open Badges standards, a set of specifications and open technical standards developed by the Mozilla Foundation. Blockcerts transfers the degree information to Bitcoin through the `OP_RETURN` field [60], or to Ethereum, depending on the client, and verification works by matching the Merkle Tree Root. Its creators also use Merkle trees to issue certificates other than academic qualifications. Both the institution that issues the certificate and the user must have their own keys to carry out the transaction. An academic qualification is registered and verified as follows. First, the university uses the dedicated Blockcerts tool to convert the student’s degree into JSON format, which it signs with its private key. The tool then creates the hash of the file, sends the hash to the blockchain and sends the signed file to the student. If the university wants to send a batch of degrees, the degrees are placed in a Merkle tree and the path of each degree’s hash is written in the JSON file. To verify a degree, one uploads the file to their website, which essentially looks up that hash in their smart contract. Revocation is handled by searching an

online list kept on their website’s server. Finally, Blockcerts technology has been used in many real-world cases, such as: a) Central New Mexico Community College, b) the National Training Agency, Bahamas, the Institute for Tourism Studies in Malta and the pilot program of the Federation of State Medical Boards (FSMB) [61, 62].

EchoLink EchoLink [8] is an Ethereum blockchain platform that uses a smart contract based on the ERC20 token. Of all the proposals, EchoLink comes closest to Verde: both use the ERC20 token smart contract, follow the same logic in what they set out to do, and run on the Ethereum blockchain. The difference is that EchoLink stores on Ethereum the details of the sender (the academic institution) and of the recipient (the student), the time the certificate was created, the address of the certificate’s PDF file and the recipient’s photo file, and not the data of the degree itself. The files are stored in the InterPlanetary File System (“IPFS”), a protocol and network designed to provide a peer-to-peer method for storing and sharing hypermedia in a distributed file system. The personal data of the sender and the recipient are kept in the company’s database, which raises doubts about how the company handles these data and how well they are protected against attacks. The EchoLink paper does not describe any method for revoking certificates when this is needed.

TrueRec TrueRec [9] was created by SAP and presented to the public in 2017. TrueRec uses a smart contract deployed on Ethereum. The company created a digital wallet for storing professional and academic qualifications. An institution must belong to the TrueRec network to be able to use the application. Registration and verification work as follows: when an institution publishes its academic qualifications through TrueRec, a digital fingerprint is added to the blockchain. TrueRec sends the documents to the user as a `.TRU` file, which the user can easily import into the TrueRec app on their mobile device. Users can share their documents directly from the TrueRec app. Recipients can verify the credentials by comparing the `.TRU` document with its fingerprint on the blockchain. Nowhere is it stated whether TrueRec has a system for revoking degrees.

University of Zurich BlockChain (UZHBC) UZHBC [10] is the solution proposed by the University of Zurich, and it applies only to that institution. It also uses a smart contract deployed on Ethereum. According to Jerinas Gresch et al., registration and verification work as follows. First, the institution must create the digital degree. The back-end then takes the PDF degree as input and generates a hash that corresponds to it. This hash is stored in a smart contract, with no possibility of revocation, and the contract is published on Ethereum. A third party who wants to verify a former student’s details receives the diploma from the student and then uses the institution’s front-end, which takes the digital diploma as input to check its authenticity. The hash is compared with all the other hashes held in the smart contract. If the hash is found,

the verification returns a success message and informs the company that the diploma is authentic. If there is no match, the system likewise returns a failure message.

BCDiploma BCDiploma was created in 2018 by the Blockchain Certified Data (bcd) team [13]. Their idea rests on three smart contracts published on Ethereum and two applications, one for registration (Crypto App) and one for verification (Reader App). The smart contracts are: 1) **SmartValidation**, which ensures that the school's identity has been verified, 2) **SmartIdentification**, which is responsible for issuing the certificate of the school's identity, and 3) **SmartPublication**, which publishes the academic qualification on Ethereum. In practice, the first stores the university's details so that anyone who needs to look them up can find them through this contract. The second creates the university's ID Certificate, that is, the university's permanent key, which the university needs before it can publish on Ethereum. The third publishes the student's details on the blockchain in compressed and encrypted (AES) form, to save on gas costs, once their Crypto App has encrypted them. So each time the Crypto App receives a degree (either as a file or through an API), it sends the encrypted data to **SmartPublication**, which checks the validity of the university's ID and then publishes the data on the network. At the end of this process, the Crypto App creates the diploma's persistence keys and stores them locally at the university, returns the degree's URL, which consists of the key (Diploma's key) and the diploma number, and then deletes these data from the application's memory. A third party who wants to verify a diploma therefore needs to enter the diploma's URL in the Reader App, and the data are decrypted in the application's back-end using the Diploma's key, the persistence key and the university's permanent key. All these keys are passed as parameters to the AES algorithm, which decrypts the student's data. The authors state that the application complies with the GDPR. If a student wants to be removed from the application, the university must delete the persistence keys, but the student's encrypted personal data remain on Ethereum.

GRNET The Greek proposal for “Advanced cryptographic services for digital academic diplomas” by Dimitris Mitropoulos was presented to the public in 2018 under the auspices of GRNET [63, 11], as part of the European Horizon 2020 program, in which 14 other European member states also take part (see, for example, the corresponding Belgian solution [64]). The idea is implemented on the private, permissioned Cardano blockchain and also on the Hyperledger blockchain. The project is notable because it is the first official use case of the Cardano blockchain. It uses a Proof-of-Authority consensus model and, more specifically, the zero-knowledge protocol, and it is being developed with IOHK, the company that created the Cardano blockchain. The zero-knowledge protocol is a method by which one entity can prove to another that it knows a value x , without revealing any information other than the fact that it

knows x [65]. According to Mr. Mitropoulos, authorized entities will be able to access the blockchain (although this is not required), and the aim is to record the fact that a degree was awarded together with the whole history of its use (or revocation). The nodes of the blockchain are the various Greek academic institutions (NTUA, AUEB), and an institution *proves* to a body (such as GRNET) that the graduate really does hold a degree. The blockchain network stores the encrypted metadata. At this stage only the hashes of the academic qualifications are stored on the network, although the project can also store whole degrees in encrypted form.

Cerberus Cerberus [12] is a study by Tariq et al. [12]. The authors built their own private blockchain network with the help of Ethereum Parity, so that they could execute smart contracts. The consensus model they use is Proof-of-Authority. The nodes of the blockchain are the academic institutions and the Observer entity, which monitors every process in the system and checks the system's integrity. The idea behind verification is that neither the verifier nor the person being verified needs to hold keys or a digital wallet on the blockchain; verification uses the QR code printed on the degree. A student's data are registered on the Cerberus network as follows. First, the digital fingerprint of the student's degree is sent to the network in a transaction, which the institution digitally signs through the university registrar on the network. The accreditation body (the entity responsible for maintaining the system) then verifies the validity of the transaction, and the transaction is added to the blockchain. Registration is, however, a little more complex than this. Every detail of the student's degree and each item of personal data is first passed separately through a SHA-2 hash function, and the results form the student-info, which is a leaf of the Merkle tree. Each leaf of the tree essentially corresponds to one degree, and all the leaves share a common root, the batch Merkle root. Qualifications are then verified through a mobile application that checks the QR code. The QR code contains information such as the degree details, the fingerprint of the student's data, the block number, the transaction ID and the addresses of the sibling leaves. The user's mobile application computes all of the above and produces the hash of a Merkle tree root. This hash is looked up on the Cerberus network, and if it is there, the verification has succeeded. Cerberus also has a system for revoking degrees, including bulk revocation. A revocation must be approved by at least two nodes, and the process is carried out by two smart contracts.

Blockchain for Education: Lifelong Learning Passport The project by Kolvenbach et al. [14] uses two smart contracts on the Ethereum blockchain to verify academic qualifications. The first smart contract (`IdentityMgmt`) supports identity management on the Blockchain for Education platform, and the second (`CertMgmt`) manages the life cycle of the academic qualifications issued through the blockchain. The accreditation authority records the public keys of the certification authorities in the

IdentityMgmt contract, so that the student's publishable details can later be registered on the blockchain through it. The academic institution then collects all of the student's data, registers the degree in the document management system and records its fingerprint on the blockchain. The student must register with the document management system, so that the certificates can then be shared with potential employers, who can verify their validity. The other smart contract manages digital academic qualifications, stores degree information and encrypts the degree, and it also provides functions such as revoking academic qualifications. This smart contract manages two different systems, the InterPlanetary Filesystem (IPFS), which is used as public distributed storage for reading information about the certification authorities and also stores the academic qualifications, so that the students' personal data are not published on Ethereum. Registration and verification work as follows. First, the institution logs in to the Web-based groupware system (BSCW) and finds the degree it wants. The degree is then transferred to Ethereum as a result of the **CertMgmt** operation and sent to the student in two forms: as JSON and as a PDF whose metadata contain the sorted JSON form of the degree. The student stores these files locally and also uploads the certificates to their personal folder in BSCW. Verification is done by uploading the PDF or the JSON file to the BSCW system; if the system finds the corresponding hash, the interface returns a success message together with information about the academic institution and the status of the degree.

EduCTX EduCTX is a global credit platform for higher education. The platform is based on the concept of the European credit system for higher education (ECTS). It is a global, decentralized credit and grading system for higher education that creates a globalized environment for students and higher education institutions, as well as for other potential stakeholders such as companies, institutions and organizations [15]. The application is based on a global peer-to-peer distributed network. EduCTX processes, manages and controls ECTX tokens, which represent the credits that students earn for the courses they complete, exactly like ECTS. The higher education institutions are the peers of the blockchain [15]. EduCTX uses a public blockchain platform, namely ARK (which runs on the DPoS consensus model). The application has two different functions: one registers the universities that act as blockchain nodes, and the other assigns credits (ECTX tokens) to students once the professor of a course allows the student to be exempted from it by passing it. An academic qualification is then verified simply by showing the amount of ECTX tokens the student holds. Importantly, each student is anonymous, for reasons of personal data protection. A student has a unique address (their digital wallet), and this address stores and receives the EduCTX tokens, that is, the credit value in ECTS, after the student successfully completes a course. The address is valid only within the EduCTX ecosystem.

All the systems above, however, have many limitations that Verde has managed to overcome. First, the UZHBC solution is restricted to use by its own institution. UZHBC, like the solutions of UNIC (Block.co), TrueRec and BCDiploma, also requires the user to keep safe a file that is the only means of verifying their diploma. If that file is lost, the ability to verify the diploma is lost with it. Most of these systems keep students' data private. EchoLink is the only exception: it runs its own platform and lets registered users view the data. Most proposals do not solve the problem of the storage space the blockchain takes up (scalability on the client side): in those that use a permissioned blockchain, such as GRNET and Cerberus, their own servers carry the whole data volume and constantly need more memory, so relatively modern technology is needed to run the blockchain network. Only Blockcerts solves the problem of scalability on the blockchain side, that is, the transaction cost, since it has a system for sending degrees to the blockchain in bulk. Proposals that use the IPFS storage system (such as Blockchain for Education and EchoLink) risk losing part of the academic qualification file. IPFS stores files in a P2P environment, so if a node deletes the files it holds and no other node has a copy of that piece, the degree is in practice deleted. The only way to keep these files on IPFS is the Pinning method [66], which requires paying a fee. A system for revoking degrees is also very important. Only a few applications have dealt with this, such as GRNET, Cerberus, Block.co, BCDiploma and Blockchain for Education. None of them, however, permanently deletes the user's data from the blockchain.

Verde has managed to overcome most of the limitations above. First, verification uses the student's own details in the verification interface, so the student does not need to keep any stored document to prove their academic qualifications. Our application follows the GDPR guidelines, since it sends the users' data encrypted with AES. The application's logic also rests on the fact that the Fourth Industrial Revolution includes blockchain technology, so in the future everyone will hold a digital wallet. The application builds on this prediction and sends the degree details and the tokens to the personal address of the student's digital wallet. If someone is one of the *crypto-skeptics* and does not want to keep a digital wallet, the application has anticipated this and can also work with a QR code printed on the student's diploma. Neither side, then, has to hold special keys to take part in the network. It is also very important that Verde is web-based: the institution does not need to buy any special equipment, and its servers do not need more memory. In fact, the opposite is true: once all of the student's data are published on the blockchain, the institution no longer needs to keep them in its database, and the only thing it still has to store is the address of the student's digital wallet. Our application works as follows: a central authority distributes tokens to the academic institutions and schools, and they in turn can give them to their graduates. Each school is therefore obliged to show on its website the official address of the digital

wallet it uses to publish degrees on the blockchain, so that the data can be verified and shown to be genuine (no such application was developed within the scope of this thesis, but the tokens' smart contract does provide this function). Verification does not stop at the data, however, since the tokens are transferred to the students from a common address. Anyone can therefore check the token transaction in the student's digital wallet and so verify that the academic qualification is genuine.

6.3.1 Comparison table of the proposals

Table 6.1 below compares the proposals above. Its rows are the proposals and its columns are various properties of decentralized applications. In more detail, the columns are:

- The blockchain on which the proposal runs.
- The consensus model the blockchain uses.
- The type of blockchain, that is, whether it is public or not.
- Whether the proposal follows the GDPR rules, that is, whether it publishes the data on the blockchain in encrypted form.
- The transparency of the process for registering and verifying degrees, how this transparency is achieved, and how the papers analyze these processes.
- The safeguard for verifying degrees. This refers to an application's ability to verify academic qualifications in two or more ways. For example, Verde shows the student's data in the interface, which is the first way of verifying, and it also shows the tokens in the form of ECTS, which is the second way of verifying, since all the tokens share a common smart contract, and both the transactions and the entity that issues the tokens are easy to check. Finally, the accreditation body of Cerberus oversees every publication by the institution and checks its validity, which again works as a safeguard.
- The accreditation of the system. More specifically, accreditation is formal recognition by a competent, recognized body, called an accreditation body. In the proposals above, accreditation means that an authority allows institutions to take part in the network or the application, and that this authority verifies the institutions' details.
- How diplomas are verified, and whether verification needs the address of the student's digital wallet or a file the student holds.
- The application's ability to revoke degrees.
- How user-friendly the interface is.
- How easy it is to access the application and its user guide.

- Adaptability is the application's ability to adapt to the resources the institution has. For example, whether the application can run in an environment with very low computing resources and obsolete computers and systems, and under the institutions' current conditions (that is, that the institution has all of its students' data stored in its database, and how handwritten, non-digitized degrees can be published on the blockchain), and whether the application's functionality can be reused for a similar task (e.g. storing employee records). This last characteristic can be achieved with smart contracts.

Proposals	Characteristics			Security properties			System properties			Usability		
	Blockchain	Consensus model	Blockchain type	GDPR	Transparency	Verification safeguard	Accreditation	Verification	Revocation	User experience	Accessibility	Adaptability
UNIC	Bitcoin	PoW	Permissionless	Yes	Yes	No	No	Using a file	Yes, partially	Very good	Very good	No
Blockcerts	Ethereum, Bitcoin	PoW	Permissionless	Yes	Yes	No	No	Using a file	Yes, partially	Very good	Very good	Yes, partially
EchoLink	Ethereum	PoW	Permissionless	No	Yes	No	No	Using a key	No	Good	Good	Yes
TrueRec	Ethereum	PoW	Permissionless	Yes	Yes	No	Yes	Using a file	No	Good	Good	No
UZHBC	Ethereum	PoW	Permissionless	Yes	Yes	No	No	Using a file	No	Very good	Very good	No
BCDiploma	Ethereum	PoW	Permissionless	Yes	Yes	No	No	Using a file	Yes, partially	Very good	Very good	Yes, partially
GRNET	Cardano	PoA	Permissioned	Yes	Yes	No	Yes	?	Yes, -	?	?	No
Cerberus	Ethereum (Parity)	PoA	Permissioned	Yes	Yes	Yes	Yes	Using a file	Yes, partially	Very good	Very good	No
Blockchain4Edu	Ethereum	PoW	Permissionless	Yes	Yes	No	Yes	Using a file	Yes, partially	Very good	Very good	Yes, partially
EduCTX	ARK	DPoS	Permissioned	Yes	Yes	No	Yes	Using a file	No	Very good	Very good	No
Verde	Ethereum	PoW	Permissionless	Yes	Yes	Yes	Yes	Using a key	Yes, fully	Very good	Very good	Yes

Table 6.1: Comparison of the proposals [6, 7, 8, 9, 10, 11, 12, 13, 14, 15].

6.4 Comparing development platforms for Verde

In this section we look at how Verde can also be deployed on other blockchain platforms that include an EVM for executing smart contracts. For this study we chose the RSK platform, which is a sidechain of Bitcoin.

6.4.1 RSK

In the course of this research it was natural to ask which platform is best for creating smart contracts and building decentralized applications, which offers the most user-friendly (and developer-friendly) environment for developing applications, and, in the end, which is the most economical solution. There are a great many blockchain platforms we could have studied, such as Counterparty, EOS, or private blockchains such as Hyperledger Besu/Sawtooth/Burrow. During this thesis most of these were tried out experimentally, but the application was not deployed on them. The platform we chose is RootStock (RSK), a sidechain of Bitcoin that was created in 2015 and launched its mainnet in 2018. RSK is a platform for executing smart contracts that use bitcoin as their native asset, which extends bitcoin's reach to dApps. The RSK network has some improvements over Bitcoin, such as faster transactions and better scalability. It grew out of two technologies, QixCoin (from the founders of RSK) and Ethereum. RSK inherits many basic concepts from Ethereum, such as the account format, the VM interface and web3. Since January 2019, miners with at least 40% of Bitcoin's hash rate have been merge-mining RSK. So that users can use Bitcoin both inside and outside RSK, the platform has a two-way peg function that lets it interact with Bitcoin. When Bitcoins are transferred to the RSK blockchain, they are converted into Smart Bitcoins (ticker RBTC). Smart Bitcoins are the equivalent of Bitcoins on the RSK blockchain, and the user can convert them back into Bitcoin at any time, at no extra cost apart from the standard transaction fees. RBTC is the currency RSK uses to pay miners for processing transactions and smart contracts. To be clear, RSK does not issue a new currency: all RBTC are created from Bitcoins. RSK has four main features:

- The deterministic, Turing-complete virtual machine RVM. The RVM is a version of the EVM: the two are similar at the opcode level, so the RVM can also execute Ethereum smart contracts. Smart contracts are executed by all the full nodes of the network.
- The two-way peg of the Bitcoin sidechain. A sidechain is a blockchain that has adopted the cryptocurrency of another blockchain, almost always in one direction only, and transfers are made by storing proofs of the transactions. A two-way peg exists when two cryptocurrencies can be exchanged freely and automatically, without negotiating their price. The logic of this function is quite simple: when an amount of cryptocurrency is exchanged from BTC to RBTC, it is locked

on the Bitcoin network and unlocked on the RSK network, and the same holds in the opposite direction. This function is carried out with the help of a third party (STTP), called the Federation, because the Bitcoin network is not Turing-complete.

- A distinctive consensus model, merged mining. Merged mining is a technique that lets miners mine other cryptocurrencies at the same time, at almost zero marginal cost. The same mining infrastructure that mines Bitcoins also mines RSK simultaneously. This means that, as RSK rewards miners with additional transaction fees³
- A low-latency network for fast transfers. This network resembles the way Bitcoin's soft fork, the Lightning network, works. Although it offers fast off-chain transactions, it also offers much faster on-chain transactions through the DECOR+ and FastBlock protocols, which make it possible to produce a block every fifteen seconds on average, so that there is no incentive for centralized, selfish mining.

Finally, two more very important features of RSK are: a) private transactions, in the same sense as Bitcoin, that is, pseudonymity, and b) better scalability than Bitcoin, since it uses the LTCP protocol, which makes each transaction one fiftieth of the size of the same transaction on Bitcoin.

6.4.2 Comparison with Ethereum

Since we chose a platform quite close to Ethereum, we can compare several of their functions and decide which platform is better for deploying Verde. To deploy our application on RSK, we only need to connect MetaMask to the RSK testnet⁴ and deploy the smart contract through remix.ethereum.org on the RSK testnet. The first differences show up as soon as the smart contract is deployed on the network. Figures 6.16, 6.17 and 6.18 below are taken from the MetaMask interface while the following actions were running: deploying the contract, sending the student's details and sending the student's courses. One RBTC equals one BTC, so deploying the contract, at 0.09 RBTC, costs 746.30 euros. Sending the student's data costs about 0.031 RBTC in total, that is, 257 euros. The transaction times differ too: according to the MetaMask interface, sending the student's data to Ethereum takes about 3 minutes, whereas on RSK it takes about 8 minutes.

³The sentence is unfinished in the original.

⁴More details here: <https://developers.rsk.co/wallet/use/metamask/>

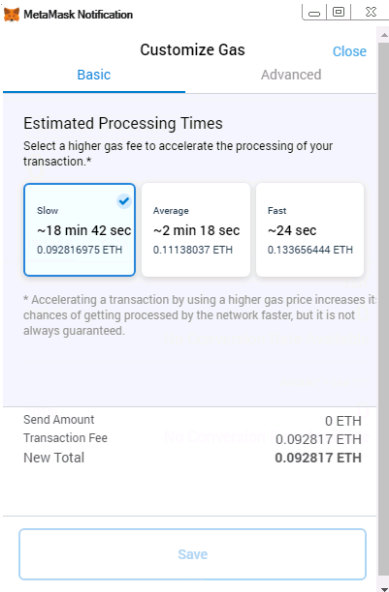


Figure 6.16: Contract deployment.

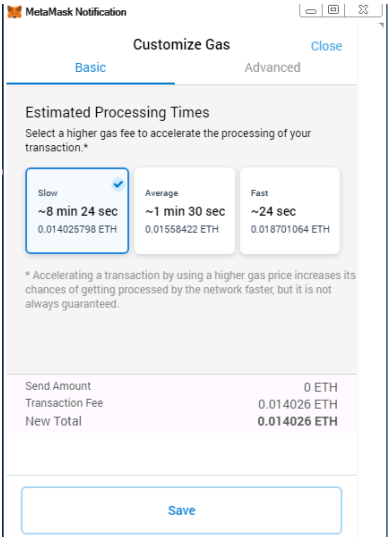


Figure 6.17: Sending the student’s details.

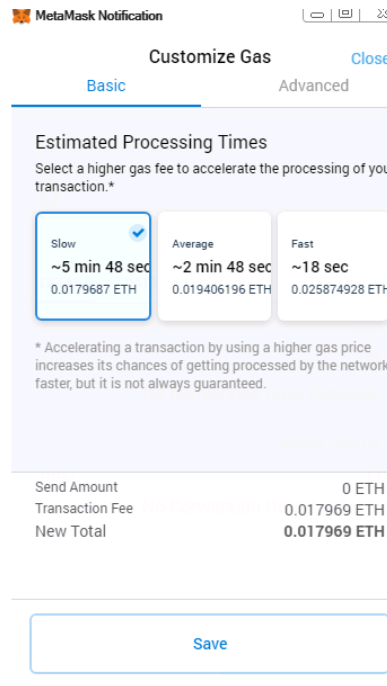


Figure 6.18: Sending the student's courses.

We judged how user-friendly each block explorer (etherscan.io, explorer.rsk.co) is by how easily one can reach certain categories, such as the ERC 20 tokens category, which RSK does not offer, and by the look of the page.

From the above we conclude that Ethereum is the better choice for Verde, except in two categories where it falls behind RSK: a) the type of chain, since RSK works as a side-chain of Bitcoin, which means it saves computing power on the main chain, unlike applications based on Ethereum, and b) the size of the blockchain, which concerns full nodes.

Blockchains / Characteristics		Ethereum	RSK
General	Cryptocurrency	ETH	RBTC
	Company	Ethereum Foundation	RSK Labs
	Platform	Ethereum	Bitcoin
Blockchain	Consensus model	PoW (Casper-PoS expected)	Merged-mining
	Transactions / sec.	15-25	15-25
	Block time (s)	10-20	15-30
	Chain type	main	side
	Programming language	Solidity	Solidity
	Total size	> 1.5 TB	~ 2 GB
Verde	Deployment cost	~ 14 euros	~ 746 euros
	Transaction cost	~ 6 euros	~ 257 euros
	Estimated transaction time	3 minutes	8 minutes
Usability	Transaction verification website	etherscan.io	explorer.rsk.co
	User-friendliness	Very good	Good

Table 6.2: Comparison of development platforms for Verde [16, 17].

Chapter 7

Conclusions

7.1 Conclusion

This thesis presented blockchain technology, its structure and its components, the cryptography behind blockchain (with the help of the open-source software Sage), the uses of blockchain today, and an example of preventing the forgery of university degrees. The thesis also had a teaching purpose, since the literature on the subject in Greek is still at an early stage. It explained what blockchain is, what a blockchain looks like in practice and why it is so safe to use, and it described the widely known blockchains that exist today, which are worth more than 100 billion dollars on the market¹. The topic was studied both in theory and in practice, through the development of the Verde application. The aim was to give a well-rounded understanding of blockchain networks and to show that blockchain is a technology that will bring a new technological revolution. A further aim was to use comparison to examine consensus models, applications for verifying academic qualifications (compared with Verde) and the development platform of our application.

The thesis is of interest both to cryptography and mathematics and to computer science and engineering. To sum up, we first set out to present the architecture of blockchain and the technologies it rests on, and then to build a decentralized application, to show that blockchain is a practical, easy-to-use technology rather than a purely theoretical one. Alongside this, we used open-source code to demonstrate the cryptographic building blocks of blockchain. This thesis can be used as a handbook for studying and researching blockchain technology. It analyzed the cryptography of blockchain in Sage, its structure and its layers, and the Solidity code used to develop the decentralized application, and it carried out a comparative study of key blockchain technologies. In this way the thesis explained the theory behind the technology, together with the mathematics that comes from cryptography and its implementation as a program in a computing environment, to give the fullest and most practical understanding of them.

¹Source: coinmarketcap.com/currencies/bitcoin/

7.2 Future research directions

This study focused on blockchain technology in general. Blockchain is a new technology with a future, because it will survive and thrive in the long term. It is very important that, alongside the growth of IoT systems, big data technology and artificial intelligence, quantum computers are also being developed. Quantum computers will turn computer science into a new science, very different from what exists today. They will make many of the cryptosystems we know today obsolete, because large integers will be factored in seconds. The RSA cryptosystem will therefore no longer be usable, and the same will happen to elliptic curves. Quantum computing will change the picture in informatics and cryptography, but the same does not apply to blockchain technology, whose core is based on the SHA-256 cryptographic algorithm. More specifically, Grover's algorithm² can be used to find a collision in a hash function in a number of steps equal to the square root of its original length. So a ciphertext of length 2^{256} will in effect have a length of 2^{128} [67]. It will still be secure, since a ciphertext of length 2^{128} takes a long time to decrypt.

Further study aims to broaden this work and to assess, from a programming point of view, how far the Verde application can be improved so that it can stand on its own as a commercial application, and whether it is safe enough for government use. First, the application should be examined in terms of smart contract security, to see how tamper-proof and resistant to attacks it is. Smart contract security is a very new field of study and is still at a very early stage [68]. One example of a weakness in the Verde smart contract is the pair of functions that register the students' details and their courses: one of them can be filled with data without the other being updated accordingly. So if, after the first OK that sends the student's details to the blockchain, the second transaction is cancelled, the verification of the data will be invalid. Weaknesses have also been found in the ERC20 token smart contract, and these should be investigated in the future [69]. There are also weaknesses at the interface level. The interfaces encrypt and decrypt the students' data with a specific tool that is managed by a third party unknown to us. In addition, the tool that obfuscates the Javascript code of the interface, so that nobody can easily guess how the students' data is decrypted, is not the best solution: someone who really wants to can still analyze the code and turn it back into readable code. The solution to this problem is to add a second key (for example the user's private key) that only the user holds. Anyone who wants to verify the user's degree will then need both the public and the private key, because the data will be encrypted with the digest of these two keys.

²More information: en.wikipedia.org/wiki/Grover's_algorithm

As for blockchain technology itself, there is much that could be studied and researched further in the future. Blockchain draws on a wide range of ideas and principles from software engineering and computer science, such as hash functions, data structures, data storage, cryptography, network architectures, communication between computers and computational puzzles. Each of these concepts and technologies has been, and still is, a subject of research. As a result, different versions of blockchain can be created simply by using different hash functions, different cryptographic methods for generating keys, or different computational puzzles for proving work [23].

The main problem this technology faces is the scalability of the chain. The chain held by full nodes has already reached a size of around 3.6 TB³. This is far from negligible, and the size grows linearly every day. Research on the problem is already under way, with solutions such as parallel chains that reduce the volume of data on the main chain [70]. One solution is side-chains, so that only the strictly necessary information is recorded on the main network (for example RSK and Plasma [71]).

The violation of personal data, and the doubt blockchain raises over whether it is fully lawful, are a serious concern for legal experts. In information science, users' privacy must be protected, and here blockchain technology leaves room for improvement: it relies on the idea that everyone has access to everyone else's transaction history, so that the next transaction can be validated. Private blockchains restrict read access, so they can no longer be used by everyone to establish ownership from the history of transaction data [23]. Another approach is the *zero-knowledge proof*, which lets anyone prove that their transactions are correct without having full access to the data [23].

Finally, what matters most to the nodes of a blockchain network is the consensus model that all of them must agree on and follow. The best model is still under research and study, since it has not yet been found. At first, Proof of Work seemed a good practice: it works in a network where no node trusts any other, and it distributes the rewards fairly to those who actually did the work of carrying out a transaction. The problem is the excessive energy consumption this model causes, because the machines must run constantly at high output. It therefore leaves a very large environmental footprint on a planet already burdened by climate catastrophe. The alternative to this model is Proof of Stake, which looks like good practice compared with Proof of Work, but it still faces challenges, described in this study, that need further investigation. Paxos [72] and Raft [73] are entirely different algorithms for reaching consensus, developed long before blockchain appeared [23]. One major challenge with alternative consensus mechanisms is that they are often conceptually more complex and therefore harder to prove formally. If one of them has a flaw at the game-theoretic level, that flaw could destroy the blockchain and so shake trust in it [23].

³The up-to-date version of this figure is at <https://etherscan.io/chartsync/chainarchive>

The future holds a great deal, and the only certainty is that blockchain technology will be part of it. The most developed countries have already started to adopt the idea of blockchain and to put it into practice in many cases. A typical example is Germany, which issued an official announcement on the next steps it will take to build an economy that includes cryptocurrencies⁴.

Further research is therefore needed to resolve all the issues that arise in blockchain, leading to a technology that can be widely used and made as efficient as possible.

⁴More details on the announcement: www.bmwi.de/Redaktion/EN/Publikationen/Digitale-Welt/blockchain-strategy.pdf

Appendix A

Terminology

English term	Greek term used in the original	Comments
Blockchain Technology	Τεχνολογία Επάλληλων Μπλοκ	
nonce	Τυχαίος αριθμός μοναδικής χρήσης	
Proof of Stake	Απόδειξη Συμμετοχής	
Consensus Model	Μοντέλο Συναίνεσης	
Proof of Burn	Απόδειξη Καύσης ή Καταστροφής	
Proof of Importance	Απόδειξη Σπουδαιότητας	
Ether	Αιθέριο	Similar to the dollar
Ledger	Κατάστιχο	
Mining	Εξόρυξη	
Wallet	Ψηφιακό Πορτοφόλι	
Smart Contract	Έξυπνο Συμβόλαιο	
Gas	Αέριο	
Hash Functions	Συναρτήσεις Διασποράς	
Ethereum Virtual Machine	Εικονική Μηχανή Ethereum	
Decentralized Application	Αποκεντρωμένη Εφαρμογή	
Centralized Application	Κεντροποιημένη Εφαρμογή	An application built around a center
Cryptocurrency	Κρυπτονόμισμα	
Forking	Διακλάδωση	

Table A.1: Translation table.

References

- [1] Dylan Yaga, Peter Mell, Nik Roby, and Karen Scarfone, “Blockchain technology overview,” *NISTIR 8202*, January 2018.
- [2] Nutthakorn Chalaemwongwan and Werasak Kurutach, “State of the art and challenges facing consensus protocols on blockchain,” 2018.
- [3] SlimCoin, “A peer-to-peer crypto-currency with proof-of-burn,mining without powerful hardware,” 2014.
- [4] <https://fair-coin.org/en/faircoin-2-revision-one-most-promising-cryptocurrencies>, “Faircoin whitepaper.”
- [5] nem.io, “Technical reference,” 2018.
- [6] Konstantinos Karasavvas, “Revoking Records in an Immutable Ledger: A Platform for Issuing and Revoking Official Documents on Public Blockchains,” in *2018 Crypto Valley Conference on Blockchain Technology (CVCBT)*, Zug, June 2018, pp. 105-111, IEEE.
- [7] Marco Baldi, Franco Chiaraluce, Migelan Kodra, and Luca Spalazzi, “Security analysis of a blockchain-based protocol for the certification of academic credentials,” *arXiv:1910.04622 [cs]*, Oct. 2019, arXiv: 1910.04622.
- [8] Steve X Chen and EchoLink Team, “Building A High-Trust Economy,” p. 22.
- [9] “Meet TrueRec by SAP: Trusted Digital Credentials Powered by Blockchain,” July 2017, news.sap.com.
- [10] Jerinas Gresch, Bruno Rodrigues, Eder Scheid, Salil S. Kanhere, and Burkhard Stiller, “The Proposal of a Blockchain-Based Architecture for Transparent Certificate Handling,” in *Business Information Systems Workshops (BIS 2018)*, Witold Abramowicz and Adrian Paschke, Eds., Lecture Notes in Business Information Processing, vol. 339. Springer International Publishing, Cham, 2019, pp. 185-196.
- [11] Dimitris Mitropoulos, “Grnet.”
- [12] Aamna Tariq, Hina Binte Haq, and Syed Taha Ali, “Cerberus: A Blockchain-Based Accreditation and Degree Verification System,” *arXiv:1912.06812 [cs]*, Dec. 2019, arXiv: 1912.06812.
- [13] “Bcdiploma.”

- [14] Wolfgang Gräther, Sabine Kolvenbach, Rudolf Ruland, Julian Schütte, Christof Torres, and Florian Wendland, “Blockchain for Education: Lifelong Learning Passport,” 2018, Publisher: European Society for Socially Embedded Technologies (EUSSET).
- [15] M. Turkanović, M. Hölbl, K. Košič, M. Heričko, and A. Kamišalić, “Eductx: A blockchain-based higher education credit platform,” *IEEE Access*, vol. 6, pp. 5112-5127, 2018.
- [16] Sergio Demian Lerner, “Rsk bitcoin powered smart contracts,” 2019.
- [17] “Smart Contract Platforms Comparison: RSK vs Ethereum vs EOS vs Cardano,” blockgeeks.com.
- [18] Andreas M. Antonopoulos and Gavin Wood, *Mastering Ethereum*, O’Reilly, 2018.
- [19] Leslie Lamport, “The part-time parliament,” *ACM Trans. Comput. Syst.*, vol. 16, no. 2, pp. 133-169, May 1998.
- [20] Dylan Yaga, Peter Mell, Nik Roby, and Karen Scarfone, “Blockchain technology overview,” *NISTIR 8202*, October 2018.
- [21] Leslie Lamport, Robert Shostak, and Marshall Pease, “The Byzantine Generals Problem,” *ACM Transactions on Programming Languages and Systems*, vol. 4, no. 3, pp. 382-401, July 1982.
- [22] Satoshi Nakamoto, “Bitcoin: A peer-to-peer electronic cash system,” 2008.
- [23] Daniel Drescher, *BLOCKCHAIN BASICS A NON-TECHNICAL INTRODUCTION IN 25 STEPS*, Apress, 2017.
- [24] “Clinic director operated for years without a degree” (Greek: «Dieythynths klinikhs xeiroyrgoyse gia xronia xwris ptyxio»), <http://newpost.gr/ellada/500068/dieythynths-klinikhs-xeiroyrgoyse-gia-xronia-xwris-ptyxio>.
- [25] “Nurse’s salary repayment over a forged degree goes to the Court of Audit | Kathimerini” (Greek: «Sto elegktiko synedrio h epistrofh apodoxwn noshleytrias gia plasto ptyxio | Kathimerini»), <https://www.kathimerini.gr/1063512/article/epikairothta/ellada/sto-elegktiko-synedrio-h-epistrofh-apodoxwn-noshleytrias-gia-plasto-ptyxio>.
- [26] “The stethoscope does not make the doctor | Kathimerini” (Greek: «To stithoskopio den kanei ton giatro | Kathimerini»), <https://www.kathimerini.gr/770855/article/epikairothta/ellada/to-sth8oskopio-den-kanei-ton-giatro>.
- [27] Eleftheria, “Messinia: 10 months in prison for having a forged degree!” (Greek: «Messinia 10 mines fylakisi giati eixe plasto ptyxio!»), <https://eleftheriaonline.gr/local/koinonia/dikastiko/item/125650-messinia-10-mines-fylakisi-giati-eixe-plasto-ptyxio>.

- [28] Wade Trappe and Lawrence C. Washington, *Introduction to Cryptography with Coding Theory*, Pearson.
- [29] Hans Delfs and Helmut Knebl, *Introduction to Cryptography - Principles and Applications*, Springer.
- [30] V.A. Katos and G.Ch. Stephanides, “Cryptography and cryptanalysis techniques” (Greek: «Τεχνικές κρυπτογραφίας και κρυπτανάλυσης»), *Zygos Publications* (Greek: Εκδόσεις Ζυγός), 2003.
- [31] Alasdair McAndrew, *Introduction to Cryptography with Open-Source Software*, CRC Press, 2011.
- [32] Christof Paar and Jan Pelzl, *Understanding Cryptography*, Springer, 2010.
- [33] W. Trappe and L.C. Washington, *Introduction to Cryptography with Coding Theory*, Pearson.
- [34] Andreas M. Antonopoulos, *Mastering Bitcoin*, O’Reilly.
- [35] Darrel Hankerson, Alfred J Menezes, and Scott Vanstone, *Guide to elliptic curve cryptography*, Springer Science & Business Media, 2006.
- [36] Lawrence C Washington, *Elliptic curves: number theory and cryptography*, CRC press, 2008.
- [37] Kenneth Ireland and Michael Rosen, *A classical introduction to modern number theory*, vol. 84, Springer Science & Business Media, 2013.
- [38] Vicente Munoz, “Everyday cryptography: fundamental principles and applications [book review],” 2013.
- [39] William Stein, *Elementary number theory: primes, congruences, and secrets: a computational approach*, Springer Science & Business Media, 2008.
- [40] Imran Bashir, *Mastering Blockchain*, Packt, 2018.
- [41] <https://en.bitcoin.it/wiki/Secp256k1>, “Secp256k1.”
- [42] Gavin Wood, “Ethereum - a secure decentralised generalised transaction ledger byzantium version,” 2019.
- [43] Bikramaditya Singhal, Gautam Dhameja, and Priyansu Sekhar Panda, *Beginning Blockchain*, Apress, 2018.
- [44] Kostis Karantias, Aggelos Kiayias, and Dionysis Zindros, “Proof-of-burn,” 2019.
- [45] “PoET 1.0 Specification - Sawtooth v1.0.5 documentation,” <https://sawtooth.hyperledger.org/docs/core/releases/1.0/architecture/poet.html>.
- [46] S. Zhang and J. Lee, “Double-spending with a sybil attack in the bitcoin decentralized network,” *IEEE Transactions on Industrial Informatics*, vol. 15, no. 10, pp. 5715-5722, 2019.

- [47] Leslie Lamport, “Time, clocks, and the ordering of events in a distributed system,” 1978.
- [48] Roberto Tonelli, Giuseppe Destefanis, Michele Marchesi, and Marco Ortu, “Smart Contracts Software Metrics: a First Study,” *arXiv:1802.01517 [cs]*, Feb. 2018, arXiv: 1802.01517.
- [49] “ERC20 Token Standard - Ethereum Smart Contracts - BitcoinWiki.”
- [50] “Go Ethereum,” <https://geth.ethereum.org/>.
- [51] “Introduction metamask docs,” <https://docs.metamask.io/guide/#why-metamask>.
- [52] Monica Rozenfeld, “Five surprising applications of blockchain technology.,” 13 Aug. 2018.
- [53] Kirill Kuvshinov, Ilya Nikiforov, Jonn Mostovoy, Dmitry Mukhutdinov, and Vladislav Podtelkin, “Disciplina: Blockchain for Education,” p. 17.
- [54] Daniel Zinca and Vlad-Andrei Negrean, “Development of a Road Tax Payment Application using the Ethereum Platform,” in *2018 International Symposium on Electronics and Telecommunications (ISETC)*, Timisoara, Nov. 2018, pp. 1-4, IEEE.
- [55] “Ethereum (ETH) price, charts, market cap, and other metrics.”
- [56] “brix/crypto-js,” May 2020.
- [57] “Fortmatic,” docs.fortmatic.com.
- [58] “javascript-obfuscator/javascript-obfuscator,” May 2020.
- [59] “Blockchain Certificate Validation.”
- [60] “blockchain-certificates/cert-issuer.”
- [61] “Customer Story FSMB | Hyland Credentials.”
- [62] “Blockchain for Vocational Training Certification:,” www.hylandcredentials.com.
- [63] “Cardano Blockchain’s First Use Case: Proof of University Diplomas in Greece.”
- [64] “European self sovereign identity framework.”
- [65] “Zero-knowledge proof,” Apr. 2020.
- [66] “IPFS Documentation,” docs.ipfs.io.
- [67] Vasileios Mavroeidis, Kamer Vishi, Mateusz D. Zych, and Audun Jøsang, “The Impact of Quantum Computing on Present Cryptography,” *ijacsa*, vol. 9, no. 3, pp. 405-414, 2018, arXiv: 1804.00200.
- [68] Sarwar Sayeed, Hector Marco-Gisbert, and Tom Caira, “Smart Contract: Attacks and Protections,” *IEEE Access*, vol. 8, pp. 24416-24427, 2020.

- [69] “sec-bit/awesome-buggy-erc20-tokens,” May 2020.
- [70] Kyle Croman, Christian Decker, Ittay Eyal, Adem Efe Gencer, Ari Juels, Ahmed Kosba, Andrew Miller, Prateek Saxena, Elaine Shi, Emin Sirer, Dawn Song, and Roger Wattenhofer, “On scaling decentralized blockchains (a position paper),” in *Financial Cryptography and Data Security (FC 2016 Workshops)*, Feb. 2016.
- [71] Georgios Konstantopoulos, “Plasma cash: Towards more efficient plasma constructions,” 2019.
- [72] Leslie Lamport, “The part-time parliament,” *ACM Trans. Comput. Syst.*, vol. 16, no. 2, pp. 133-169, May 1998.
- [73] Diego Ongaro and John Ousterhout, “In search of an understandable consensus algorithm,” in *USENIX ATC '14*, pp. 305-319, 2014.